Numerical Derivatives with Numerixx

A textbook and reference for finite differences and Ridders' extrapolation in C++

Kenneth Troldal Balslev

October 2026

About this document

This document is a textbook and a reference for `nxx::deriv`, the numerical-derivatives module of Numerixx, a header-only C++23 library of numerical algorithms. The module computes the first to the fourth derivative of a function that a program can evaluate but cannot differentiate: a property model, the output of a solver or a simulation, a correlation in someone else's code. It is written for engineers and scientists who are comfortable with calculus and with C++ but are not specialists in numerical computation. The fundamentals are meant to be easy to follow; the topics a user does not need to know, such as proofs, floating-point detail and the library's implementation decisions, are in numbered sidebars that the surrounding text never depends on.

How to read it

Part I teaches. Its seven chapters build the methods up from one idea, Taylor's theorem, and are meant to be read in order: where derivatives arise and what the library offers (Chapter 1); the forward and the central difference and their errors (Chapter 2); why a smaller step is not always better, and how the step is chosen (Chapter 3); more points and one-sided rules (Chapter 4); second to fourth derivatives (Chapter 5); Ridders' extrapolation and its one-sided forms, the methods that estimate their own error (Chapter 6); and the library in use, with worked engineering examples and the pitfalls (Chapter 7). One example runs through Part I, the heat capacity of nitrogen from its enthalpy, whose derivative is known, so that every error quoted could be measured.

Part II is the reference: every public name (Chapter 8), every weight and constant (Chapter 9), and what accuracy and cost to expect, where the methods stop working, and what the library guarantees (Chapter 10). The appendices hold the floating-point facts and the derivations the sidebars cite; a notation table and a glossary follow them.

A reader in a hurry has two shortcuts. Section 1.5, "If you read nothing else", chooses a method for the five situations that cover most uses and gives the two checks every call needs. And the quick-reference page (page 83) is the interface on one sheet: the calls, the result, the failures, the method choice, the checks and the pitfalls that matter most.

The evidence

Every number in this book that comes from a computation was produced by the library itself, apart from a few attributed figures of its design record and tests: a generator program in the book's sources runs `nxx::deriv` and writes the values, the data behind every plot and the bodies of the tables, and the text quotes them by name; at every run it also checks the weight tables and the nitrogen example against independent evaluations. Every code listing is quoted from a compiled program that runs as a test of the library and checks what the text claims about it, with two exceptions:

the declarations of Chapter 8 are transcribed from the library's header and checked against it at every build, and the CMake snippet of Section 7.1 is quoted from a file the book's build does not configure. The data behind every plot and table, and the program that computes them, are in the repository under `docs/deriv-book/data` and `docs/deriv-book/tools`.

The version

This document describes `nxx::deriv` as of commit `90d83378863c` (in full in the snippet of Section 7.1) of the Numerixx repository, branch `dev`. The library's public headers, its tests and its design record are the ground truth: where this book and the code disagree, the book has the bug.

How this document was prepared

This document was prepared with the use of AI, under the direction of its author, who approved every stage and is responsible for its content. Anthropic's Claude drafted and reviewed it in named authoring and reviewing roles; OpenAI Codex and Cline with DeepSeek V4.1 Flash reviewed the result independently. It was updated for the library's Phase 5 in the same way, drafted and reviewed by Claude, with an independent review by OpenAI Codex. What makes it trustworthy is checked by the build, not taken on a model's word: every number, plot and table was computed by running the library, every code listing is compiled and run as a test, and every reference was checked against the work or a library catalogue.

Contents

About this document i

I Fundamentals 1

1 Why numerical derivatives 2
- 1.1 Derivatives without a formula . . . 2
- 1.2 The alternatives in one page . . . 2
- 1.3 What the library offers . . . 4
- 1.4 A first example . . . 4
- 1.5 If you read nothing else . . . 6

2 Finite differences from Taylor's theorem 8
- 2.1 The secant and the tangent . . . 8
- 2.2 Centring cancels a term . . . 9
- 2.3 Order of accuracy . . . 10
- 2.4 Exactness on polynomials . . . 12
- 2.5 What this chapter did not say . . . 12

3 Rounding and the choice of step 13
- 3.1 Floating-point numbers in brief . . . 13
- 3.2 The two errors and the V . . . 13
- 3.3 The best step and the best accuracy . . . 15
- 3.4 The default step . . . 16
- 3.5 When the default step is wrong: the scale of f . . . 17
- 3.6 When f is less accurate than ε . . . 19
- 3.7 Float, double and more digits . . . 21
- 3.8 A checklist for the step . . . 21

4 Higher-order and one-sided stencils 22
- 4.1 Stencils: nodes, weights and the moment conditions . . . 22
- 4.2 The order ladder . . . 24
- 4.3 The price . . . 24
- 4.4 One-sided rules . . . 26
- 4.5 Choosing P . . . 27

5 Second to fourth derivatives 28
- 5.1 The rules for N = 2 to 4 . . . 28
- 5.2 Why accuracy falls with the order . . . 29
- 5.3 The default step per order and its sensitivity . . . 30
- 5.4 The float warning . . . 31
- 5.5 Ridders for higher orders . . . 31
- 5.6 The nitrogen example: $\mathrm{d}c_p/\mathrm{d}T$ . . . 31

6 Ridders' extrapolation 33
- 6.1 Richardson's idea . . . 33
- 6.2 The tableau in one picture . . . 33
- 6.3 Reading `abs_error` . . . 35

6.4 When not to trust it . . . 37
6.5 Cost . . . 38
6.6 Stencil or Ridders . . . 38
6.7 One-sided Ridders . . . 39

7 Using the library 42
7.1 The call . . . 42
7.2 The result . . . 43
7.3 Choosing a method . . . 45
7.4 Worked examples . . . 45
7.5 Is my derivative good enough? . . . 50
7.6 User-defined and multiprecision types . . . 51
7.7 At compile time . . . 52
7.8 Pitfalls . . . 53
7.9 What the library does not do . . . 54

II Reference 55

8 API reference 56
8.1 Synopsis . . . 56
8.2 `derivative` . . . 56
8.3 `Estimate<T>` . . . 59
8.4 `error` and `order` . . . 59
8.5 The methods . . . 60
8.6 The precision warning . . . 60
8.7 `nxx::real` and `nxx::math` . . . 61
8.8 Rejected calls . . . 62

9 Tables 63
9.1 How to read the tables . . . 63
9.2 Central rules . . . 63
9.3 Forward rules . . . 64
9.4 Default steps and model accuracy . . . 64
9.5 Ridders' constants . . . 65

10 Accuracy, cost, limits and guarantees 67
10.1 Accuracy . . . 67
10.2 Ridders' error estimate . . . 68
10.3 Cost . . . 69
10.4 Limits . . . 69
10.5 Guarantees . . . 70
10.6 How it is verified . . . 72

A Floating-point essentials 73
A.1 Representation . . . 73
A.2 Machine epsilon, the unit roundoff and the ulp . . . 73
A.3 The ends of the range . . . 74
A.4 Why $x + h$ is not $x + h$ . . . 74
A.5 Cancellation . . . 74
A.6 Other types . . . 74

B Derivations 75
B.1 Taylor expansions and the moment conditions . . . 75

B.2 The Lagrange formula for the weights 76
B.3 Richardson extrapolation 77
B.4 The error model and the best step 78

Notation and glossary **79**

References **82**

Quick reference **83**

List of sidebars

1.1 When automatic differentiation or the complex step is the better tool . . . 3
2.1 Where the constant 1/6 comes from . . . 9
3.1 Dividing by the distance actually evaluated . . . 15
3.2 The error model behind the default step, and why its constant is dropped . . . 16
3.3 Machine epsilon, ulps and the rounding of $x + h$ . . . 17
4.1 The moment conditions and the Lagrange generator . . . 23
4.2 `Central<4>` is Richardson extrapolation in disguise . . . 25
5.1 Why the fourth derivative is so sensitive . . . 29
5.2 The three-point f'' from the actual offsets . . . 32
6.1 Burden's difference form of the tableau . . . 40
6.2 How Ridders stops, and how often its estimate holds . . . 40
6.3 Aliasing: when the samples lie . . . 41
7.1 Expression templates and user-defined real types . . . 51
7.2 Overflow guards and constant evaluation . . . 52

Part I

Fundamentals

Chapter 1

Why numerical derivatives

Engineering models are full of derivatives that nobody writes down. A heat capacity is the slope of an enthalpy, a Jacobian is the slope of a set of balances, and a sensitivity is the slope of a result with respect to a parameter. When the function is a formula on paper, the derivative is a matter of calculus. When the function is a program, it often is not. This chapter says where such derivatives arise, which tools exist for them, and what the library Numerixx offers; it ends with a first computation and a half-page summary for the reader in a hurry.

1.1 Derivatives without a formula

A function in engineering software is usually a piece of code that takes a number and returns a number. Typical cases are these.

- *Property models.* A heat capacity $c_p = \mathrm{d}H/\mathrm{d}T$ from an enthalpy $H(T)$, a compressibility from an equation of state, a vapour-pressure slope for the Clausius–Clapeyron equation. The model is a correlation inside a property library, often in several pieces.
- *Simulators.* The outlet temperature of a heat exchanger as a function of a feed flow, computed by a simulation that iterates to convergence.
- *Implicit functions.* A friction factor defined by an equation it must satisfy, or a flash temperature defined by an energy balance: the value comes from a solver, and no formula for it exists.
- *Sensitivities.* How much a reaction rate changes per kelvin, or per unit of activation energy: the numbers a design review asks for.
- *Newton's method.* Solving $g(T) = 0$ needs the slope $g'(T)$ at every iteration.

In every case the function can be *evaluated* at any point we like, but its derivative is not given. Deriving it by hand from the code is tedious and error-prone, and impossible when the code is not ours.

The way out is old and simple. The derivative $f'(x)$ is the slope of the tangent to the curve of f at x, and the slope of a line through two nearby points of the curve, a *secant*, is close to it (Fig. 1.1). Evaluate f at x and at $x + h$, divide the change of f by h, and we have an estimate of $f'(x)$: a *finite difference*. The secant through $x - h$ and $x + h$, placed symmetrically around x, is much closer to the tangent than the one through x and $x + h$, as the figure shows. Chapter 2 explains why, and Chapter 3 why the step cannot simply be made as small as we like.

The *step* h is the distance between x and the points at which f is evaluated.

1.2 The alternatives in one page

A finite difference is not the only way to a derivative, and not always the best.

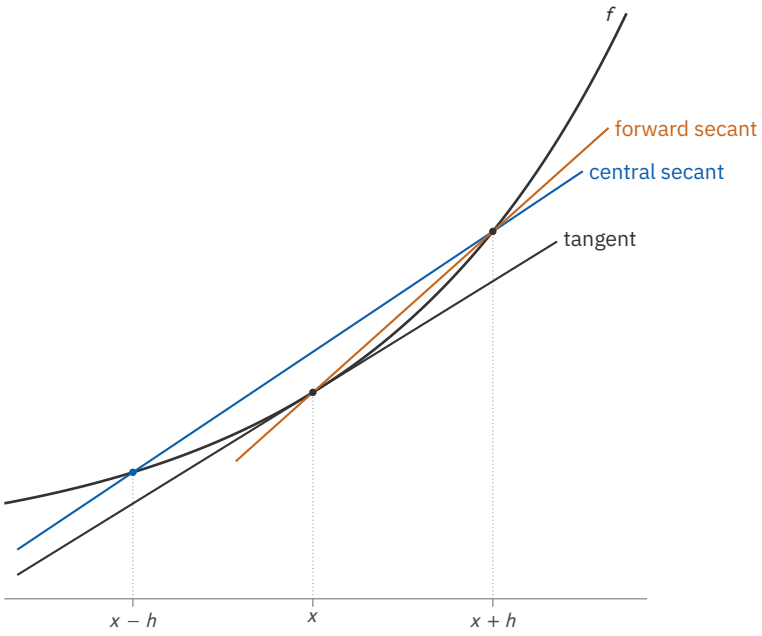


Figure 1.1: The tangent at x and two secants. The central secant, through $x - h$ and $x + h$, is nearly parallel to the tangent; the forward secant, through x and $x + h$, is visibly steeper. A diagram, without data.

- *The analytic derivative*, by hand or by computer algebra, is exact and cheap to evaluate. Use it when the formula is at hand and short enough to differentiate without mistakes.
- *Automatic differentiation* runs the program with a special number type that carries a derivative along with every value, and is exact to rounding. It needs the source code, written for a generic number type.
- *The complex step* evaluates f once at a complex argument and reads f' from the imaginary part of the result, again exact to rounding. It needs an f that accepts complex numbers and is analytic.
- *A finite difference* needs only values of f at real points. It works on any function the program can evaluate: a library call, a solver, legacy code. Its price is accuracy (about two thirds of the digits of the number type, Chapter 3) and two or more evaluations of f.

This book is about the last: the tool for the function that is a black box. When the first three are open to you, they are better, and the sidebar below says when (Sidebar 1.1).

Sidebar 1.1. When automatic differentiation or the complex step is the better tool

Forward-mode automatic differentiation replaces each number v in the computation by a pair (v, v') and each operation by its rule of calculus: $(u, u') \cdot (v, v') = (uv, u'v + uv')$, and so on. Seeding x with $(x, 1)$ gives $(f(x), f'(x))$ at the end, exact to rounding, at a cost of a small multiple of one evaluation. It needs the function's source, written for a generic number type (a C++ template, for instance), and a library that supplies the pair type. [1]

The *complex step* uses $f'(x) \approx \operatorname{Im} f(x + \mathrm{i}h)/h$. [2] No two nearly equal values are subtracted, so the step can be taken far below the limit that rounding sets for a finite difference, and the result is exact to rounding with one evaluation. It needs f implemented for complex arguments and analytic near x: an absolute

value, a comparison or a table lookup inside f breaks it. It is closely related to forward-mode automatic differentiation.[3] The complex step is a special case of Lyness and Moler's earlier contour-integral methods, which also give higher derivatives of analytic functions;[4] Squire and Trapp brought the simple first-derivative form back into use.

`nxx::deriv` takes any callable `fn` that maps a real number to a real number, and assumes nothing else: not its source, not a complex extension, not even smoothness beyond what the error terms of the later chapters need. That is the case the other tools do not cover.

1.3 What the library offers

`nxx::deriv` is the numerical-derivatives module of Numerixx, a header-only C++23 library. Everything goes through one function, `derivative`, with these choices.

- *Six methods.* `Central<P>`, the default, samples f on both sides of x; `Forward<P>` and `Backward<P>` sample on one side only, for a function defined on one side; `Ridders` adapts its step and also estimates its own error, and its one-sided forms `ForwardRidders` and `BackwardRidders` do the same on one side of x (Section 6.7). The first three are *stencils*: `P` is the accuracy order of Chapter 2.
- *Derivatives of order 1 to 4*, f' to $f^{(4)}$.
- *Any real type*: `float`, `double`, or a user type with more digits such as those of Boost.Multiprecision.
- *No exceptions from the library.* A failure is a value: the result is a `std::expected` that holds either an estimate or the error `non_finite`. The library throws nothing itself; an exception thrown by `fn` passes through unchanged, and `derivative` is not `noexcept`.
- *A default step*, so that a call needs only the function and the point.

A *stencil* samples f at a fixed pattern of points around x and combines the samples with fixed weights. Ridders' method decides its points as it goes.

The calls themselves are the subject of Chapter 7; Chapter 8 lists every one.

1.4 A first example

The example that runs through Part I is the heat capacity of a gas from its enthalpy. For nitrogen the NIST Chemistry WebBook gives the molar enthalpy in the Shomate form, a fit to the NIST-JANAF thermochemical tables.[5,6] We use the range 500 K to 2000 K and the point $T_0 = 1000$ K. The function returns $H(T)$, the enthalpy relative to its value at 298.15 K, in J/mol, so that $\mathrm{d}H/\mathrm{d}T$ is c_p in J/(mol K) directly. The whole program is short:

The *Shomate form*: a fitted polynomial in $T/1000$ plus a $1/T$ term, one set of coefficients per temperature range.

```
// The heat capacity of nitrogen at 1000 K: cp = dH/dT.
#include "shomate.hpp" // the book's H(T), namespace book

#include <numerixx/deriv.hpp>

#include <print>

int main()
```

```
{
    // H(T) in J/mol for T in K, so dH/dT is in J/(mol K)
    auto const H = [](double T) {
        return book::shomate_H(book::nitrogen, T);
    };
    double const T0 = 1000.0; // K, inside the fit's range

    auto const d = nxx::deriv::derivative(H, T0);
    if (d) {
        std::println("cp = {:.10f} J/(mol K)", d->value);
    } else {
        std::println("no estimate: error::non_finite");
    }
    return d ? 0 : 1;
}
```

It prints $c_p = 32.691\,729\,999\,2$ J/(mol K). The heat capacity is one call: without a method or a step, `derivative` uses `Central<2>`, the central difference of Fig. 1.1, with its default step, here 0.0061 K; Chapter 3 explains where it comes from.

The result `d` holds either an estimate or the reason why there is none. `if (d)` is true when there is an estimate, and only then is `d->value`, the derivative, read; the `else` branch handles the failure. The test is not a formality. A call fails, with the error `error::non_finite`, for example when `fn` returns an infinite or NaN value at a point the method samples, or when the step is too small to move x (Chapter 3). Every listing in Part I reads a result this way.

The first include is the book's own header, not the library's: `book::` is the namespace of the book's examples, and `shomate.hpp` defines the fit once for all of them, with the WebBook's coefficients and its expression for H:

```
// One range of a Shomate fit: the WebBook's eight
// coefficients and the temperatures the fit is valid on.
struct Shomate
{
    double A, B, C, D, E, F, G, H;
    double T_min, T_max; // K
};

// Nitrogen (N2), NIST Chemistry WebBook, fit of January
// 2009 to Chase 1998; retrieved 2026-10-23. Three ranges
// that share their end points.
inline constexpr Shomate nitrogen_low{
    28.98641, 1.853978, -9.647459, 16.63537, 0.000117,
    -8.671914, 226.4168, 0.0, 100.0, 500.0 };
inline constexpr Shomate nitrogen{
    19.50583, 19.88705, -8.598535, 1.369784, 0.527601,
    -4.935202, 212.3900, 0.0, 500.0, 2000.0 };
inline constexpr Shomate nitrogen_high{
    35.51872, 1.128728, -0.196103, 0.014662, -4.553760,
    -18.97091, 224.9810, 0.0, 2000.0, 6000.0 };
```

and the enthalpy itself, in J/mol:

```
// H(T) - H(298.15 K) in J/mol for T in K: the WebBook's
// kJ/mol expression times 1000. Valid on [T_min, T_max];
// evaluable anywhere above 0 K.
template<nxx::real Real>
constexpr Real shomate_H(Shomate const &c, Real const &T)
{
    Real const t = T / Real(1000);
    return Real(Real(1000)
        * (Real(c.A) * t + Real(c.B) * t * t / Real(2)
            + Real(c.C) * t * t * t / Real(3)
            + Real(c.D) * t * t * t * t / Real(4)
            - Real(c.E) / t + Real(c.F) - Real(c.H)));
}
```

How good is the estimate? For this function we know the answer. The WebBook also publishes the fit's $c_p(T)$, the exact derivative of $H(T)$, which gives $c_p(T_0) = 32.691\,730\,000\,0$ J/(mol K). The estimate is off by 2.4×10^{-11} relative. This closed form is the *answer key* of the book. We differentiate a function whose derivative we know so that every error can be measured; in practice you would differentiate one whose derivative you do not know, and Chapter 7 does so with a friction factor that comes from a solver. The closed form is exact for the *function*, not for nitrogen: at 1000 K the tables give 32.697 J/(mol K) (Part II, p. 1621),[6] from which the fit differs by 1.6×10^{-4} relative. That is a matter for the property data, not for the derivative.

Ridders' method adapts its step and reports an estimate of its own error in `abs_error`:

```
using nxx::deriv::Ridders;
auto const r = nxx::deriv::derivative<Ridders>(H, T0);
if (r) {
    std::println("cp = {:.12f} +- {:.1e} J/(mol K)",
                 r->value, *r->abs_error);
}
```

After 14 evaluations of H, against the stencil's two, it returns $c_p = 32.691\,730\,000\,0$ J/(mol K) with an error estimate of 2.7×10^{-12} J/(mol K). The actual error is 2.0×10^{-12} J/(mol K), or 6.0×10^{-14} relative, so the estimate is honest here: it is larger than the actual error. Chapter 6 shows how the method works and how far its estimate can be trusted.

`abs_error` is a `std::optional`, empty for the stencils; the `*` in `*r->abs_error` reads it.

1.5 If you read nothing else

Table 1.1 chooses a method for the five situations that cover most uses; the full decision table is Table 7.1 in Chapter 7.

Accuracy. At the default step a first derivative in `double` has about 11 correct digits, a fourth derivative about 5: the medians of Table 10.1, measured on e^x. Higher derivatives lose digits fast (Chapter 5).

Cost. A stencil costs a fixed number of evaluations, two for `Central<2>` and f'; Ridders' method typically 14 for f' and never more than 64 (Table 10.3).

Did it work? Test the result before you use it, as in the program of Section 1.4: `if (d)`, then `d->value`, and an `else` that handles the failure.

Is it right? A value can be wrong without any warning. Three situations cause most such errors:

- f varies on a scale far from $\max(|x|, 1)$ in your units, such as a narrow transition or a small mole fraction (Section 3.5);
- f is the output of a solver, or otherwise less accurate than the number type (Section 3.6);
- f has a kink or a range break within a few steps of x (Section 7.8).

A stencil reports no error estimate: if in doubt, repeat the call with the steps $2h$ and $h/2$ and count the digits that agree (Section 7.5). For Ridders' method, read the estimate before you report a value: accept the result `res` only if `res && res->abs_error && *res->abs_error <= atol + rtol * std::abs(res->value)` (Section 7.2). Take `rtol` $= 10^{-n}$ for a value reported to n significant figures and `atol` as the error you can accept in the derivative's units, which is what decides near a zero of f'; inside an iteration no check is needed. A passed check says that the estimate is small, not that the value is right where the estimate is blind, as at a kink.

Where to read on. Why the central difference beats the forward one: Chapter 2. Why a smaller step is not better, and when to pass your own: Chapter 3. More points and one-sided rules: Chapter 4; higher derivatives: Chapter 5; Ridders' method: Chapter 6; the calls and the pitfalls: Chapter 7.

Situation	Method `M` and step	Cost and accuracy
Inside an iteration; f smooth and cheap	`Central<2>`, the default step	2 evaluations; about 11 digits in `double`
A value to report, with an error estimate	`Ridders`, the default initial step	typically 14, at most 64; about 14 digits
f defined on one side of x only	`Forward<2>` or `Backward<2>`, the default step; with an error estimate, `ForwardRidders` or `BackwardRidders`	3 evaluations; about 1.8×10^{-11} relative on e^x. With the estimate: typically 10, at most 33; about 1.7×10^{-13}
f varies on a scale w far from $s = \max(\|x\|, 1)$	`Central<2>`, $h = \varepsilon^{1/3} w$	2 evaluations; as in the first row
f accurate to $\eta > \varepsilon$ only (a solver output)	`Central<2>`, $h = \eta^{1/3} s$	2 evaluations; about $\eta^{2/3}$ relative

Table 1.1: The method in five situations. Each row is a call `derivative<M>(fn, x)`, or `derivative<M>(fn, x, h)` with the step given; ε is the machine epsilon of the type (2^{-52} for `double`); w is the distance in x over which f changes appreciably (Section 3.5); η is the relative accuracy of f, how many digits of it you can trust, for example 10^{-8} for a solver converged to a relative tolerance of 10^{-8} on its output (Section 3.6). Evaluations are those of `fn` for f'. The full table, with the evidence for each row, is Table 7.1; the quick-reference page at the end of the book repeats this one.

Chapter 2

Finite differences from Taylor's theorem

Figure 1.1 showed that the secant through $x - h$ and $x + h$ lies closer to the tangent than the secant through x and $x + h$. This chapter says by how much, with one tool: Taylor's theorem. It gives the two simplest finite differences, the error each makes, and a rule that predicts how that error changes with the step. All of it can be done with pencil and paper, and we measure every prediction on the heat capacity of nitrogen.

2.1 The secant and the tangent

The slope of the secant through x and $x + h$ is the *forward difference*

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}. \tag{2.1}$$

How far is it from $f'(x)$? Taylor's theorem answers that. If f has a continuous second derivative between x and $x + h$, then

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2} f''(\xi)$$

for some point ξ between x and $x + h$.[7] The first two terms are the tangent line; the last is the *remainder*, the price of stopping the series there. We do not know ξ, but we know that it lies in the interval. Subtract $f(x)$ and divide by h:

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{h}{2} f''(\xi). \tag{2.2}$$

The forward difference is $f'(x)$ plus an error of $hf''(\xi)/2$. This error is the *truncation error*: the error of the formula itself, with every number computed exactly, so called because it comes from truncating the Taylor series. The picture says the same thing: a secant differs from the tangent because the curve bends, and the bending is f''.

For the enthalpy of nitrogen, $H'' = \mathrm{d}c_p/\mathrm{d}T$ is positive at T_0, where the closed form gives $0.005\,744\ \mathrm{J/(mol\,K^2)}$ (the heat capacity still rises at 1000 K), so the forward difference comes out too high, by about half of H'' times the step.

The library calls this rule `Forward<1>`. The number in the name is its accuracy order, defined in Section 2.3. A rule is written in the library, and in this book, as a set of *nodes* n_k, the offsets of the sample points in units of h, and their *weights* w_k: the estimate is $\sum_k w_k f(x + n_k h)/h$. Table 2.1 gives the nodes and weights of the two rules of this chapter.

Table 2.1: The two rules of this chapter, as $f'(x) \approx \sum_k w_k f(x + n_k h)/h$. The complete tables are in Chapter 9 (Table 9.1).

	`Forward<1>`	`Central<2>`
nodes n_k	$0, 1$	$-1, 1$
weights w_k	$-1, 1$	$-\frac{1}{2}, \frac{1}{2}$
error	$\frac{h}{2} f''$	$\frac{h^2}{6} f'''$

`P` in `Forward<P>` and `Central<P>` is the accuracy order, not the number of points: `Central<2>` uses two points for f', `Forward<1>` two.

2.2 Centring cancels a term

Now expand f on both sides of x, one term further. With f smooth enough,

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(x) + \cdots$$
$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(x) - \cdots$$

The terms with even powers of h are the same in both lines, and the terms with odd powers differ only in sign. Subtracting the second line from the first therefore removes $f(x)$, f'' and $f^{(4)}$ and doubles the rest. Divide by $2h$:

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(x) + \frac{h^4}{120}f^{(5)}(x) + \cdots \tag{2.3}$$

This is the *central difference*, the library's `Central<2>` and its default rule. The term with h has gone: the bending f'', which spoils the forward difference, affects the two sides equally and cancels. What is left starts with h^2 and involves f''', the rate at which the bending changes. With the remainder in Lagrange's form instead of the infinite series, the result reads

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(\xi), \qquad x-h < \xi < x+h, \tag{2.4}$$

which holds whenever f''' is continuous on the interval; the sidebar below proves it and shows why the next term is of order h^4 (Sidebar 2.1).

The sign of the error is the sign of f''': the central difference overestimates f' where $f''' > 0$ and underestimates it where $f''' < 0$. For nitrogen at 1000 K, $H''' = \mathrm{d}^2c_p/\mathrm{d}T^2$ is negative (the heat capacity rises ever more slowly), so the central difference comes out slightly low.

Note what the central difference does not use: the value $f(x)$ has weight zero (Table 2.1). It costs two evaluations of f, the same as the forward difference, and is far more accurate for small h. That is why it is the default.

There is a second way to see the cancellation. The mirror image of the forward difference is the *backward difference* $(f(x) - f(x-h))/h$, the library's `Backward<1>`, the rule for a function known only below x. By the same expansion its error is $-hf''(x)/2 + O(h^2)$, the same size as the forward difference's, with the opposite sign. The central difference is exactly the mean of the two,

$$\frac{f(x+h) - f(x-h)}{2h} = \frac{1}{2}\left(\frac{f(x+h) - f(x)}{h} + \frac{f(x) - f(x-h)}{h}\right),$$

so their errors of order h cancel, and what remains is of order h^2.

$O(h^2)$, "of order h^2": terms of size h^2 and smaller, which shrink at least as fast as h^2 (Section 2.3).

Sidebar 2.1. Where the constant 1/6 comes from

Taylor's theorem with the remainder in Lagrange's form says: if f has $n+1$ continuous derivatives between x and $x+h$, then

$$f(x+h) = \sum_{j=0}^{n} \frac{h^j}{j!}f^{(j)}(x) + \frac{h^{n+1}}{(n+1)!}f^{(n+1)}(\xi)$$

for some ξ strictly between x and $x+h$; the same holds with $-h$ in place of h.[7] Take $n=2$ on both sides of x:

$$f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2}f''(x) \pm \frac{h^3}{6}f'''(\xi_\pm),$$

with ξ_+ in $(x, x+h)$ and ξ_- in $(x-h, x)$. Subtract and divide by $2h$:

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6} \cdot \frac{f'''(\xi_+) + f'''(\xi_-)}{2}.$$

The mean of $f'''(\xi_+)$ and $f'''(\xi_-)$ lies between the two values, and a continuous function takes every value between two of its values (the intermediate value theorem). So the mean equals $f'''(\xi)$ for some ξ in $(x-h, x+h)$, which is Eq. (2.4). The $1/6$ is $1/3!$ from the cubic term, unchanged because the two cubic terms add.

Why the next term is h^4: in the series for $f(x+h) - f(x-h)$ every even power of h cancels, so only odd powers $h, h^3, h^5, \ldots$ remain; after the division by $2h$ only even powers $h^0, h^2, h^4, \ldots$ are left. The error of the central difference is a series in h^2. The forward difference has no such symmetry, and its error contains every power of h.

2.3 Order of accuracy

Both error terms have the form constant times a power of h times a derivative of f: $\frac{1}{2}h^1 f''$ for the forward difference and $\frac{1}{6}h^2 f'''$ for the central one. In general a rule for the N-th derivative has a truncation error whose leading term is

$$\pm C\, h^P f^{(P+N)}(x),$$

with $C > 0$ its size and a sign fixed by the rule (+ for both rules here; Chapter 9 gives it for every rule), and P is its *accuracy order*. The forward difference has accuracy order 1 and the central difference accuracy order 2; we write that their errors are $O(h)$ and $O(h^2)$. The statement is about small steps: as h tends to 0, the error divided by h^P stays bounded, and the leading term describes the error better and better. For larger steps the next terms matter too; for the central difference the next one is of order h^{P+2}, here h^4, as Eq. (2.3) shows.

The *derivative order* N says which derivative is computed; this chapter has $N=1$ throughout. Chapter 5 treats $N=2$ to 4.

The practical meaning is a prediction. While the leading term dominates, halving the step divides the error by 2^P: it halves the error of the forward difference and quarters that of the central difference. To gain a factor of 10 in accuracy, the forward difference needs a step 10 times smaller, the central difference a step only $\sqrt{10} \approx 3.2$ times smaller.

The prediction can be checked by hand. Take $f(x) = x^3$ at $x=1$, where $f'(1) = 3$ and $f'' = f''' = 6$, with the steps 0.1 and 0.05:

	$f(1+h)$	$f(1-h)$	forward, error	central, error
$h = 0.1$	1.331	0.729	3.31, 0.31	3.01, 0.01
$h = 0.05$	1.157625	0.857375	3.1525, 0.1525	3.0025, 0.0025

Halving the step divides the forward error by a little more than 2 and the central error by exactly 4. The error terms predict the sizes too: $hf''/2 = 3h$ gives 0.3 and 0.15 for the forward difference, the small rest being the next term, h^2; and $h^2 f'''/6 = h^2$ gives $1/100$ and $1/400$ for the central difference, exactly, because the fourth and higher derivatives of a cubic vanish.

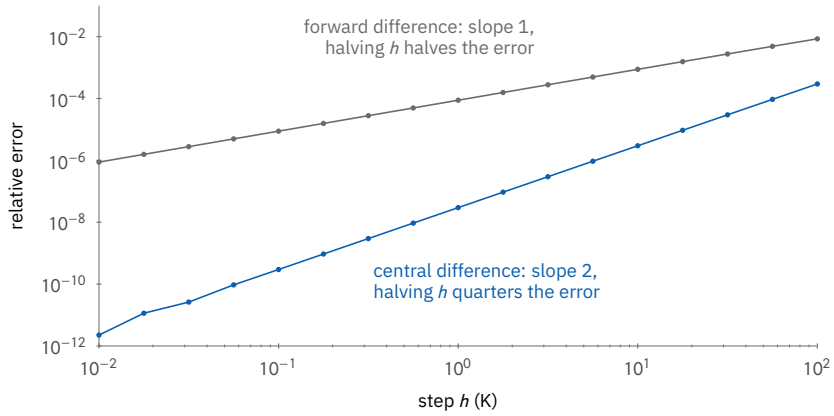


Figure 2.1: Relative error of `Forward<1>` and `Central<2>` against the step h for the heat capacity of nitrogen, $\mathrm{d}H/\mathrm{d}T$ at $T_0 = 1000\,\mathrm{K}$, for large steps. On logarithmic axes the errors fall on lines of slope 1 and 2: halving the step halves one error and quarters the other. Below 0.01 K the central difference stops improving; why is the subject of Chapter 3.

How small must h be for the leading term to dominate? Small compared with the distance over which f changes its character, so that the higher derivatives in the next terms do not outweigh the powers of h they come with. For the enthalpy of nitrogen, a smooth function of $T/1000\,\mathrm{K}$, steps of many kelvin are already small in this sense. Most engineering functions vary on a scale comparable to the size of their argument; Chapter 3 builds the library's default step on that observation, and shows what to do when it fails.

Figure 2.1 tests the prediction on the nitrogen example, $H(T)$ at $T_0 = 1000\,\mathrm{K}$, with the closed form of c_p as the answer key. On logarithmic axes an error Ch^P is a straight line of slope P, and the two measured curves are such lines, of slopes 1 and 2. At every step shown, the central difference is more accurate by orders of magnitude.

The same check takes a few lines of code. The listing computes both differences at steps of 2 K and 1 K and prints the ratio of their errors:

```
using nxx::deriv::derivative;
using nxx::deriv::Central, nxx::deriv::Forward;
double const cp = book::shomate_cp(book::nitrogen, T0);

// steps of 2 K and 1 K: the second is half the first
auto const f2 = derivative<Forward<1>>(H, T0, 2.0);
auto const f1 = derivative<Forward<1>>(H, T0, 1.0);
auto const c2 = derivative<Central<2>>(H, T0, 2.0);
auto const c1 = derivative<Central<2>>(H, T0, 1.0);
if (!f2 || !f1 || !c2 || !c1) return 1;

// absolute errors in J/(mol K), and their ratio
double const ef2 = std::abs(f2->value - cp);
double const ef1 = std::abs(f1->value - cp);
double const ec2 = std::abs(c2->value - cp);
double const ec1 = std::abs(c1->value - cp);
std::println("forward: {:.2e} then {:.2e}, ratio {:.3f}",
             ef2, ef1, ef2 / ef1);
std::println("central: {:.2e} then {:.2e}, ratio {:.3f}",
             ec2, ec1, ec2 / ec1);
```

The forward ratio comes out close to 2 and the central ratio close to 4. At such steps the rounding errors of Chapter 3 are negligible, and the next terms of the series shift the ratios only slightly.

2.4 Exactness on polynomials

The error terms give a second, exact check that needs no computer. The forward difference has the error $hf''(\xi)/2$, which vanishes when $f'' = 0$: the forward difference is exact for every straight line, as it must be, since a secant of a line is the line. The central difference has the error $h^2 f'''(\xi)/6$, which vanishes when $f''' = 0$: it is exact for every quadratic. With pencil and paper, for $f(x) = a + bx + cx^2$,

$$\frac{f(x+h) - f(x-h)}{2h} = \frac{2bh + 4cxh}{2h} = b + 2cx = f'(x)$$

for every h. For a cubic it is not exact: $f(x) = x^3$ gives $3x^2 + h^2$, and the excess h^2 is the error term $h^2 f'''/6$ with $f''' = 6$. In general a rule of accuracy order P for the N-th derivative is exact for polynomials of degree up to $P + N - 1$ (Chapter 4). "Exact" here means in exact arithmetic; a computer still rounds the values, which is the subject of the next chapter.

The structure of Eq. (2.3) also shows where more accuracy can come from. Write $D(h)$ for the central difference at step h. Its error is $\frac{1}{6}h^2 f''' + O(h^4)$, so the error of $D(h/2)$ is a quarter of that, and the combination

$$\frac{4D(h/2) - D(h)}{3} = f'(x) + O(h^4)$$

cancels the h^2 term. This is the four-point rule `Central<4>` of Chapter 4 in disguise (Sidebar 4.2), and repeated, the idea of Ridders' method (Chapter 6).

2.5 What this chapter did not say

Three assumptions stand behind every formula of this chapter.

- *Exact arithmetic.* The values of f are rounded, and Fig. 2.1 stops at 0.01 K because below that step the central difference stops improving. Why, and how to choose the step, is Chapter 3.
- *Two points.* More points buy a higher accuracy order, and one-sided rules of higher order exist (Chapter 4); higher derivatives follow in Chapter 5.
- *Smoothness.* The error terms need f'' or f''' to exist and be continuous on the whole interval the rule samples. Where f has a kink or a jump inside that interval, the error terms say nothing, and the result can be wrong without any warning (Section 7.8).

Chapter 3

Rounding and the choice of step

Chapter 2 predicts that a smaller step gives a smaller error: halve h, and the error of the central difference falls by four. A computer does not follow the prediction for long. Pass the step yourself, as the third argument of `derivative`, and compare a moderate step with a tiny one:

```
// the step, in K, as the third argument
for (double const h : {1e-2, 1e-8}) {
    auto const d = derivative(H, T0, h);
    if (d) {
        std::println("h = {:.0e} K: rel. error {:.1e}",
                     h, std::abs(d->value - cp) / cp);
    }
}
```

The step of 10^{-8} K is worse than the step of 10^{-2} K, by orders of magnitude. This chapter explains why, finds the best step, and shows how the library's default step is built, when it is wrong, and what to do then.

3.1 Floating-point numbers in brief

A `double` stores a number with 53 binary digits, about 16 significant decimal digits. Numbers are therefore spaced: just above 1 the next `double` is $1 + \varepsilon$, where

$$\varepsilon = 2^{-52} \approx 2.2 \times 10^{-16}$$

is the *machine epsilon* of the type. The spacing grows in proportion to the numbers themselves: near 1000 the doubles are 1.1×10^{-13} apart. Every arithmetic operation rounds its exact result to the nearest representable number, which changes it by at most half a spacing, a relative error of at most $\varepsilon/2$.[8,9] A value of f computed by a few dozen operations is therefore wrong by a few units of ε relative, sometimes less, sometimes more. We model that with one number: each value of f carries an error of about $\varepsilon|f|$. Nothing more about floating point is needed to follow this chapter; Appendix A has the details, and Sidebar 3.3 in Section 3.4 what the spacing of the numbers near x means for a step.

`std::numeric_limits<T>::epsilon()` in C++.

3.2 The two errors and the V

Figure 3.1 shows the error of the central difference for the heat capacity of nitrogen over fifteen decades of step. From the right, the error falls along a line of slope 2, as Chapter 2 predicts, until about 0.01 K; from there to the left it *rises* again, along a line of slope −1, and at the smallest steps the result has no correct digit at all. The curve has the shape of a V.

The right branch is the truncation error of Eq. (2.4). The left branch is rounding. The central difference subtracts $f(x - h)$ from $f(x + h)$, two numbers that agree in more and more of their leading digits as h shrinks.

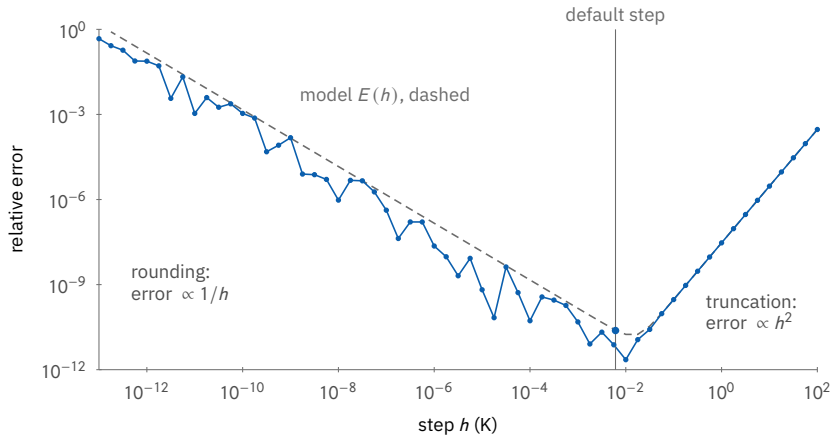


Figure 3.1: The V. Relative error of `Central<2>` for the heat capacity of nitrogen at $T_0 = 1000\,\mathrm{K}$ against the step h, over the whole range of steps, with the model $E(h)/c_p$ of Eq. (3.1) dashed. Right of the bottom truncation dominates and the error falls as h^2; left of it rounding dominates and the error grows as $1/h$. The default step sits near the bottom.

The subtraction itself is exact, but it removes the digits the two values share and leaves only those in which they differ, and those include the rounding errors of both values. The rounding errors were tiny compared with f; they are not tiny compared with the difference. This loss of digits is called *cancellation*.

A pocket calculator that keeps six significant digits shows the whole V in four lines. Differentiate e^x at $x = 1$, where $f'(1) = e = 2.718\,28\ldots$, rounding each value of f to six digits:

h	$f(1+h)$	$f(1-h)$	estimate	error
0.1	3.00417	2.45960	2.72285	+0.0046
0.01	2.74560	2.69123	2.71850	+0.00022
0.001	2.72100	2.71556	2.72000	+0.0017
0.0001	2.71855	2.71801	2.70000	−0.018

From $h = 0.1$ to 0.01 the error falls by a factor of only 21, not the 100 that truncation alone would give: at $h = 0.01$ the truncation error is about 4.5×10^{-5}, and the rest of the error in that row is already rounding. Below that step the error rises. At $h = 10^{-4}$ the two values share their first four digits; their difference, 0.000 54, has only two digits left, and the last of them is rounding. A `double` keeps about 16 digits instead of six, so the same happens at smaller steps, but it happens.

Both errors can be written down. If each value of f is off by up to about $\varepsilon|f|$, the difference $f(x+h) - f(x-h)$ is off by up to about $2\varepsilon|f|$, and after the division by $2h$ the estimate of f' by up to about $\varepsilon|f|/h$. Adding the truncation error of Eq. (2.4) gives the *error model* of the central difference,

$$E(h) = E_\mathrm{t}(h) + E_\mathrm{r}(h) = \frac{h^2}{6}|f'''(x)| + \frac{\varepsilon|f(x)|}{h}. \tag{3.1}$$

The first term falls with h and the second grows; their sum is the V. The dashed line in Fig. 3.1 is this model for the nitrogen example, and the measured curve follows it, with one difference: on the rounding branch the measured errors scatter below the model. The rounding errors of the two values have random signs and partly cancel, so the model gives their typical largest size, not their actual size at a particular step.

The model gives an *absolute* error, in the unit of f'; the figure divides it by c_p. It leaves out a second rounding, of the points $x \pm h$ themselves, which

the library removes for the central difference by dividing by the distance actually evaluated (Sidebar 3.1).

Sidebar 3.1. Dividing by the distance actually evaluated

The points $x + h$ and $x - h$ are computed, so they are rounded: the function is evaluated at $x_+ = \mathrm{fl}(x + h)$ and $x_- = \mathrm{fl}(x - h)$, each within half a spacing of the intended point, so their distance can differ from $2h$ by up to about $\varepsilon|x|$. Dividing by the nominal $2h$ would add a relative error of about $\varepsilon|x|/(2h)$. At the default step $h = \varepsilon^{1/3}|x|$ that is $\varepsilon^{2/3}/2$: as large as the accuracy aimed for.

The two-node rules (`Central<2>`, `Forward<1>` and `Backward<1>` for f') therefore divide by $x_+ - x_-$ as evaluated. The quotient is then the exact slope of the secant through the two points where f was actually evaluated, and only the rounding of the values of f remains. The subtraction $x_+ - x_-$ is itself exact when $x_+ \le 2x_-$ (for positive x), by Sterbenz's lemma (Theorem 4.3.1):[10] two floating-point numbers of the same sign within a factor of two of each other have an exactly representable difference (Higham's Theorem 2.5).[8] That holds roughly for $h \le |x|/3$. For larger steps the subtraction rounds once, a relative error of at most $\varepsilon/2$, which is harmless.

Rules with more nodes divide by a fixed multiple of the nominal step instead. For them the actual distance would not help: for `Central<4>`, dividing by the distance between the outer nodes would quadruple their point rounding (its coefficient rises from 1/12 to 4/12) and leave the inner nodes' untouched. They keep a rounding term from the points, about $(S/2)\,\varepsilon|xf'|/h$, with S the sum of the absolute weights (Chapter 4).

3.3 The best step and the best accuracy

The bottom of the V is where the model is smallest. Setting the derivative of Eq. (3.1) with respect to h to zero, $h|f'''|/3 - \varepsilon|f|/h^2 = 0$, gives the *best step*

$$h^* = \left(\frac{3\varepsilon|f|}{|f'''|}\right)^{1/3}, \qquad E(h^*) = \frac{3}{2}\frac{\varepsilon|f|}{h^*}. \tag{3.2}$$

Both depend on f and f''', which we do not know; their dependence on ε is what matters. Suppose f varies on a length scale w, in the units of x, so that each derivative is about the previous one divided by w: $|f'| \approx |f|/w$ and $|f'''| \approx |f|/w^3$. Then

The *scale* w of f: the distance in x over which f changes appreciably.

$$h^* \approx 3^{1/3}\varepsilon^{1/3}w, \qquad \frac{E(h^*)}{|f'|} \approx \varepsilon^{2/3}.$$

For a `double`, $\varepsilon^{1/3} = 6.1 \times 10^{-6}$ and $\varepsilon^{2/3} = 3.7 \times 10^{-11}$: at its best step the central difference keeps about 10 of the 16 digits (Nocedal and Wright,[11] Section 8.1). The same argument for the forward difference, with its error $h|f''|/2$, gives a best step near $\varepsilon^{1/2}w$ and a relative error near $\varepsilon^{1/2} = 1.5 \times 10^{-8}$, about 8 digits. That difference, not the cost, is why the central difference is the default.

The relative figure $\varepsilon^{2/3}$ rests on the assumption $|f| \approx w|f'|$. The rounding term of Eq. (3.1) grows with $|f|$, not with $|f'|$, so a function whose values are large compared with $w|f'|$ loses accuracy in proportion: about $\log_{10}(|f|/(w|f'|))$ digits. For the nitrogen example, with $w = T_0$, $|H|/(T_0|c_p|) = 0.66$, close to 1. Had the enthalpy been referred to 0 K, or included an enthalpy of formation, $|H|$ would be larger and the derivative

proportionally less accurate, although c_p would be the same: a formation enthalpy of a few hundred kilojoules per mole, against $T_0 c_p$ of a few tens, costs about one digit. Near a point where f' vanishes, the relative error has no meaning at all, while the absolute error stays near $\varepsilon^{2/3}|f|/w$.

On the nitrogen example the measured curve reaches its lowest point, 2.2×10^{-12}, at 0.010 K. That is below the model, because the rounding errors happen to cancel there; a neighbouring step does several times worse. The model's level, not the luckiest point, is what to expect.

3.4 The default step

The library does not know f, so it can compute neither h^* nor the scale w. It takes the formula of Section 3.3 without its constant, and guesses the scale:

$$h = \varepsilon^{1/3} s, \qquad s = \max(|x|, 1). \tag{3.3}$$

The scale s is the library's guess for w. For large $|x|$ it makes the step relative, a fixed fraction of x: functions of a physical quantity often vary on a scale comparable to its size, as the enthalpy varies with $T/1000$ K. Near zero $|x|$ says nothing about the scale, and the step becomes absolute, $\varepsilon^{1/3}$ in the units of x. This choice of scale is standard in numerical optimisation (Dennis and Schnabel,[12] Section 5.4). For the nitrogen example, $s = 1000$ K and the default step is 0.0061 K; the relative error is 2.4×10^{-11}, on the model line near the bottom of the V (Fig. 3.1). Every stencil of the library has a default step of the same form, $\varepsilon^{1/(P+N)} s$ (Chapters 4 and 5).

Dropping the constant $3^{1/3} \approx 1.4$ makes the default step smaller than the model's best step, by that factor when the guess is right, $w = s$. It costs little, and the guess itself is the weaker link (Sidebar 3.2).

Sidebar 3.2. The error model behind the default step, and why its constant is dropped

For a rule of accuracy order P for the N-th derivative, the error model is $E(h) = Ch^P|f^{(P+N)}| + S\varepsilon|f|/h^N$, with the error constant C and the sum of absolute weights S of the rule (Chapter 4); for the central difference $C = 1/6$, $S = 1$ and $N = 1$, which is Eq. (3.1). It is smallest at

$$h^* = \left(\frac{NS\varepsilon|f|}{PC|f^{(P+N)}|} \right)^{1/(P+N)},$$

and if f varies on the scale w, this is $k\varepsilon^{1/(P+N)} w$ with the constant $k = (NS/(PC))^{1/(P+N)}$. The default step leaves k out and puts the guess s for w, so when the guess is right it equals h^*/k. With $h = h^*/k$ the model error grows by the factor

$$\frac{E(h^*/k)}{E(h^*)} = \frac{Nk^{-P} + Pk^N}{P + N}.$$

For the central difference, $k = 3^{1/3} \approx 1.4$, and the factor is $7 \cdot 3^{1/3}/9 \approx 1.12$: the default step loses about a tenth of the best accuracy. For the other first-derivative rules k lies between that and 2 (`Forward<1>`), and the cost reaches 1.63 for `Central<10>`. For higher derivatives k is larger (2.6 for `Central<2>` and f'', 2.4 for $f^{(4)}$) and so is the cost (Chapter 5, Section B.4).

Why leave k out at all? Because it only matters when the guess $w = s$ holds, and the guess is the weaker link: a step off by a factor 2 either way costs up to a factor 1.7 for the central difference, more than leaving out k costs, and a

function that varies faster than s wants a smaller step anyway. On the nine test functions of the library's design record (`docs/deriv/PHASE4_API.md`), which vary faster than $\max(|x|, 1)$, half the default step did better for every order, f' included. So the library keeps one convention, $\varepsilon^{1/(P+N)}s$, for every rule and order, and leaves the scale to the caller (Section 3.5). The derivation in full is in Appendix B.

A step must also be large enough to move x. The doubles near 1000 K are 1.1×10^{-13} K apart, so $T_0 + 10^{-14}$ K rounds back to T_0, and the two points of the central difference would coincide. The library checks for coinciding points and returns `error::non_finite` rather than divide by zero:

```
// T0 + 1e-14 K rounds back to T0: there is no difference
auto const tiny = derivative(H, T0, 1e-14);
if (!tiny && tiny.error() == error::non_finite) {
    std::println("h = 1e-14 K: non_finite");
}
```

The default step is far from this limit, since $\varepsilon^{1/3}s$ is many spacings of x; a step you pass yourself may not be. The resolution of x and the rounding of $x + h$ are the subject of the sidebar below (Sidebar 3.3).

Sidebar 3.3. Machine epsilon, ulps and the rounding of $x + h$

The spacing of the numbers at x is its *unit in the last place*, $\mathrm{ulp}(x) = \varepsilon \cdot 2^e$ for $2^e \le |x| < 2^{e+1}$: $\mathrm{ulp}(1) = \varepsilon$, just below 1 the spacing is half of that, and $\mathrm{ulp}(1000) = 1.1 \times 10^{-13}$. Rounding to nearest errs by at most $\mathrm{ulp}(x)/2$, the *unit roundoff* $\varepsilon/2$ relative; this book and the library's rounding model use ε, a factor 2 of safety (Section A.2).

Since $x + h$ is rounded, the point the function sees is $\mathrm{fl}(x + h)$, not $x + h$. If h is below half a spacing of x, $\mathrm{fl}(x + h) = x$: the step has no effect. When this happens on both sides, both points round back to x, no difference can be formed, and the library reports `error::non_finite`. Near a power of two, however, the spacing differs on the two sides: below 2^e the numbers are twice as dense as above it, so a step can move x downwards and not upwards. The central difference then divides by the half-spacing that remains and returns a finite value that is mostly rounding, without failing. The rules with more than two nodes check both $x + h$ and $x - h$ and fail instead (Section 8.2). Hence the documented requirement: a step well above $\mathrm{ulp}(x)$.

3.5 When the default step is wrong: the scale of f

The default step assumes that the guess is right, $w = s$: that f varies on the scale $\max(|x|, 1)$, in the units of x. Two common situations break that assumption, and the remedy is the same for both.

A narrow feature at a large $|x|$. Take a function that changes from -1 to 1 over a width of about $w = 0.5$ around $x_0 = 1000$, such as $\tanh((x - x_0)/w)$, and differentiate it at $x = x_0 + 0.2$, as for a smoothed switch or a sharp transition in a variable far larger than its width. Here $s = 1000.2$, so the default step is 0.0061. That is small against the width, but not small enough: the truncation error grows as $(h/w)^2$, not as $(h/s)^2$, and the relative error is 2.8×10^{-5}. Figure 3.2 shows why: the default step sits far right of the V's bottom. The remedy is to put the true scale w in place of the guess

s:

$$h = \varepsilon^{1/3} w,$$

which gives the step 3.0×10^{-6} and the error 5.7×10^{-12}, at the bottom of the V. Ridders' method with an initial step on the same scale, $w/4$, is just as good: it returns an error of 4.8×10^{-15} with an estimate that covers it (Chapter 6). In code, $\varepsilon^{1/3}$ is `root_epsilon<3, double>()`:

`nxx::math::root_epsilon<N, T>()` returns $\varepsilon^{1/N}$ for the type `T`, correct to within an ulp (Section 8.7).

```
// a step from 0 to 1 over a width of about w, at x0 = 1000
double const x0 = 1000.0, w = 0.5;
auto const f = [=](double x) {
    return std::tanh((x - x0) / w);
};
double const x = x0 + 0.2;

auto const d = derivative(f, x); // step eps^(1/3) * |x|
using nxx::math::root_epsilon;
double const cbrt_eps = root_epsilon<3, double>();
auto const d_w = derivative(f, x, cbrt_eps * w);
auto const r = derivative<Ridders>(f, x, w / 4);
```

A small $|x|$ in your units. A mole fraction of 10^{-6}, a strain of 10^{-4} or a time constant in seconds of a fast process all have $s = 1$, so the default step is $\varepsilon^{1/3}$, as large as the variable itself or larger. Take $y \ln y$, which appears in every entropy of mixing, at $y = 10^{-6}$. The default step is 6.1×10^{-6}, which exceeds y; the central difference samples $y - h < 0$, where the logarithm is not defined, and the call fails with `error::non_finite`. With the step $\varepsilon^{1/3} y = 6.1 \times 10^{-12}$ the relative error is 7.9×10^{-12}, as accurate as at any ordinary point (Fig. 3.3). The 1 in $\max(|x|, 1)$ is in your units.

```
// y ln y is defined for y > 0 only; the default step is
// eps^(1/3) * 1, far larger than y itself
double const y = 1e-6;
auto const g = [](double t) { return t * std::log(t); };
auto const e = derivative(g, y);          // samples y < 0
auto const e_y = derivative(g, y, cbrt_eps * y);
```

Here the failure is loud, because the logarithm returns NaN below zero. A function that is defined across the bound but meaningless there, such

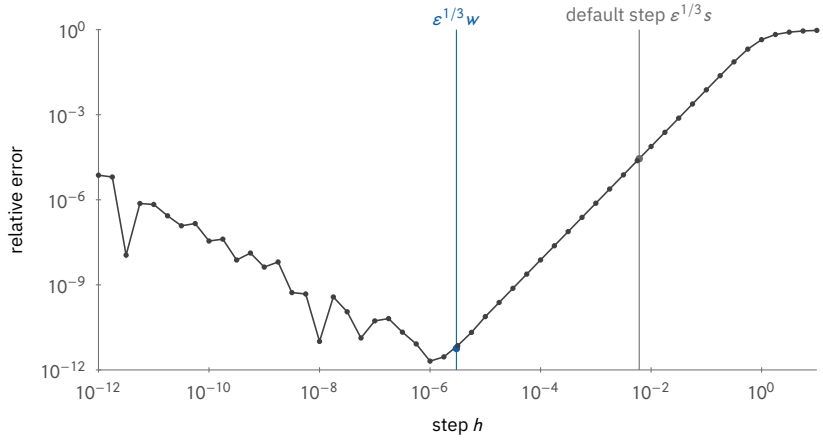


Figure 3.2: The V for a narrow feature at a large $|x|$, $\tanh((x - x_0)/w)$ with $x_0 = 1000$ and $w = 0.5$, at $x = x_0 + 0.2$. The default step $\varepsilon^{1/3} s$ sits far to the right of the minimum, and the step $\varepsilon^{1/3} w$ built from the feature's width lands at it. The default step guesses that f varies on the scale $s = \max(|x|, 1)$; put the scale w on which it does vary in place of s.

as a fit sampled outside its range, would return a wrong value without complaint; the one-sided rules of Chapter 4 are the tool for such an edge.

Units matter at large $|x|$ too: a property near room temperature varies on the scale of the absolute temperature, but at 20 °C the guess is $s = 20$ in degrees Celsius against 293 in kelvin, a default step about 15 times too small, which on the rounding branch costs about one digit.

Finding the scale. The scale w can usually be read off the model. For a transition it is the width of the transition; for a singularity or a bound at a, such as in $1/(x-a)$ or $\ln(x-a)$, the distance $|x-a|$; for an Arrhenius factor $\exp(-E_a/(RT))$, RT^2/E_a, much smaller than T for a large activation energy; for a smooth interpolant of a table, such as a cubic spline, the node spacing, although a spline is better differentiated by its own derivative (Section 7.9). A rough value is enough: at the model's best step, a step off by a factor of two either way costs `Central<2>` at most a factor 1.7 in error. If the scale is hard to judge, Ridders' method with an initial step a few times smaller than it chooses its own step and reports an error estimate (Chapter 6).

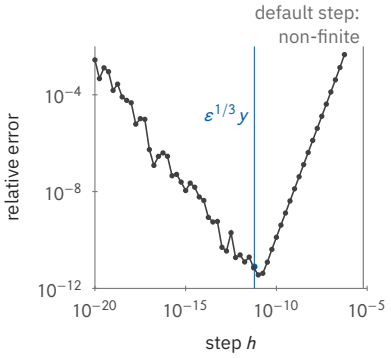


Figure 3.3: The V of `Central<2>` for $y \ln y$ at $y = 10^{-6}$. Steps above y cross zero, so the curve stops and the default step fails, while $\varepsilon^{1/3}y$ gives the expected accuracy. The 1 in $\max(|x|, 1)$ is in your units.

3.6 When f is less accurate than ε

The error model assumes that each value of f is accurate to about ε. Many engineering functions are not. The output of an iterative solver stopped at a relative tolerance of 10^{-8}, a flash calculation, or a simulation that iterates to convergence is accurate only to that tolerance, and its error changes irregularly from one argument to the next: it is *noise*. Call its relative size the *noise level* η. It takes the place of ε in the rounding term of Eq. (3.1),

$$E(h) = \frac{h^2}{6}|f'''| + \frac{\eta|f|}{h},$$

and the argument of Section 3.3 carries over with η for ε: the best step is about $\eta^{1/3}w$ and the best relative accuracy about $\eta^{2/3}$.[13] A solver converged to 10^{-8} gives about five correct digits, not 11, and only at a step $(\eta/\varepsilon)^{1/3}$ times the default.

Two cautions on η. A solver's tolerance is the noise level only when its convergence test is on the output itself, relative to it; a flash that converges a residual (an energy balance, equal fugacities) in its own units, or a simulator that converges tear streams, has a tolerance that is not η: measure η, as shown below. And if f is computed by cancellation, as a difference of large terms, its rounding error is ε times the largest intermediate, $k\varepsilon|f|$ with $k \gg 1$; treat $k\varepsilon$ as the noise level.

A smooth but inaccurate f is not noisy: the nitrogen Shomate fit describes nitrogen to about four digits, yet its derivative, the fit's, is accurate to $\varepsilon^{2/3}$.

Figure 3.4 measures this on e^x at $x = 1$, whose values are multiplied by $1 + \eta u$ with u drawn uniformly from $[-1, 1]$, the same u for the same x, as a converged solver would give. For $\eta = 10^{-8}$ the default step returns an error of 9.4×10^{-4}; the step $\eta^{1/3} = 0.0022$ gives 1.2×10^{-6}, close to the model's $\eta^{2/3} = 4.6 \times 10^{-6}$ (`code/03-rounding-noise.cpp`). So pass the step when f is noisy, $h = \eta^{1/3}s$, with η the solver's tolerance or your best estimate of the accuracy of f: in code, `derivative(f, x, std::cbrt(eta) * std::max(std::abs(x), 1.0))`.

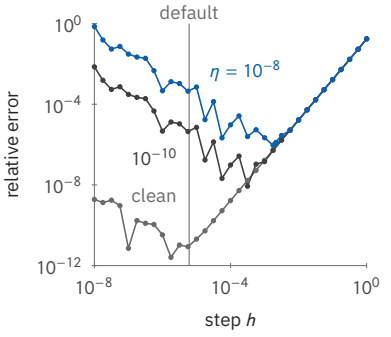


Figure 3.4: The V of `Central<2>` on e^x at 1 with the values perturbed by a relative noise $\eta = 10^{-10}$ and 10^{-8}, against the clean curve. The rounding branch lifts to $\eta|f|/h$ and the minimum moves right and up; the dot marks the step $\eta^{1/3}$ for $\eta = 10^{-8}$.

If the noise level is not known, it can be measured. Evaluate f at a handful of points spaced far more closely than the scale on which f varies, and form their successive differences, as in a difference table. The k-th differences of the smooth part of f shrink like h^k, so at such a spacing the higher differences show the noise alone, and their spread estimates $\eta|f|$ up to a known factor: the k-th differences of independent noise are $\sqrt{(2k)!/k!^2}$ times larger than the noise, and the recipe divides that out. It is a simplified form of Moré and Wild's procedure,[14] kept in the book's example header:

```
// The noise of fn near x from a difference table of its
// values at x, x + h, ..., x + m h (More and Wild).
struct NoiseEstimate
{
    double sigma; // the absolute noise; NaN if none settled
    int k;        // the difference order used; 0 if none
};
template<typename F>
NoiseEstimate estimate_noise(F fn, double x, double h,
                             std::size_t m = 7)
{
    std::vector<double> t(m + 1);
    for (std::size_t i = 0; i <= m; ++i)
        t[i] = fn(x + double(i) * h);
    std::vector<double> rms(m + 1, 0.0); // sigma_k
    double gamma = 1.0; // k!^2 / (2k)!
    for (std::size_t k = 1; k <= m; ++k) {
        for (std::size_t i = 0; i + k <= m; ++i)
            t[i] = t[i + 1] - t[i];
        gamma *= double(k) / double(2 * (2 * k - 1));
        double sum = 0.0;
        for (std::size_t i = 0; i + k <= m; ++i)
            sum += t[i] * t[i];
        rms[k] = std::sqrt(gamma * sum / double(m - k + 1));
    }
    for (std::size_t k = 1; k + 2 <= m; ++k) {
        auto const [lo, hi] = std::minmax(
            { rms[k], rms[k + 1], rms[k + 2] });
        if (hi <= 4 * lo) return { rms[k], int(k) };
    }
    return { std::numeric_limits<double>::quiet_NaN(), 0 };
}
```

Applied to the friction factor of Section 7.4.3, computed by bisection to a relative tolerance of 10^{-10}, at eight points spaced by the default step,

```
// f(Re) from a solver converged to the tolerance 1e-10
auto const f = [](double re) {
    return book::friction(re, 1e-10);
};
double const Re = 1e5;
double const h = nxx::deriv::default_step(Re);
auto const e = book::estimate_noise(f, Re, h); // 8 values
double eta = 1e-10; // the solver's tolerance, as fallback
if (e.k > 0) {      // the estimate settled
    eta = e.sigma / std::abs(f(Re));
```

```
}
double const h_eta = std::cbrt(eta) * Re; // the step
```

it reports 4.3×10^{-11} relative to f (order 2, 8 evaluations); the actual root-mean-square deviation from the exact root is 4.9×10^{-11}. On the noisy e^x above it reports 6.2×10^{-9}, near $\eta/\sqrt{3}$ for $\eta = 1.0 \times 10^{-8}$; on the clean one, 9.2×10^{-17}. If no three orders agree, it reports $k = 0$: the spacing is too large for the smooth part to drop below the noise within eight points; retry with a tenth of the spacing, or fall back to the solver's tolerance, as the listing does. A spacing on the scale of f itself can make the orders agree by accident at a wrong level (the full procedure tests signs and adjusts the spacing): use the default step, check k, and distrust a level near ε for a solver. The header, `docs/deriv-book/code/noise.hpp`, is under the repository's MIT licence.

Ridders' estimate assumes that f is accurate to about ε, so for a noisy f it can be optimistic. Over 300 points in $[0.2, 3]$ the estimate covered the actual error in 100.0 % of the cases for the clean e^x, but only in 90.7 % for $\eta = 10^{-8}$, where the worst error was 42 times its estimate. Section 7.4.3 differentiates the friction factor with this rule.

3.7 Float, double and more digits

Everything above holds for any number type, with its own ε. A `float`, $\varepsilon = 1.2 \times 10^{-7}$, reaches only about $\varepsilon^{2/3} = 2.4 \times 10^{-5}$; on e^x at 1 its default step gives 6.5×10^{-6} (Fig. 3.5). A 50-digit type, $\varepsilon = 5.3 \times 10^{-51}$, reaches far beyond `double` (Section 7.6).

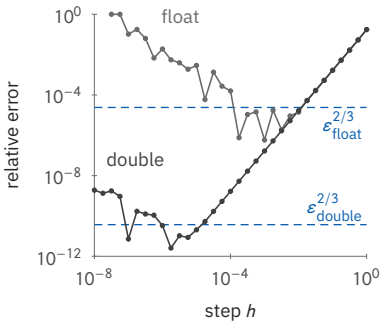


Figure 3.5: The V of `Central<2>` for `float` and `double` on e^x at 1. Each floor lies at or below $\varepsilon^{2/3}$ of its type (dashed), with the rounding scatter up to a decade or more below it.

3.8 A checklist for the step

Before you accept the default step, answer six questions.

1. *On what scale does f vary, in the units of x?* If it is not about $\max(|x|, 1)$, pass $h = \varepsilon^{1/3} w$ for `Central<2>`, with w that scale (Section 3.5).
2. *Is there a bound within a step of x*, such as zero for a mole fraction or the end of a fit's range? Use a smaller step, or a one-sided rule (Chapter 4).
3. *How accurate is f?* If it is a solver output or otherwise noisy at a level η above ε, pass $h = \eta^{1/3} s$ and expect about $\eta^{2/3}$; if you do not know η, measure it (Section 3.6).
4. *Is $|f|$ much larger than $s|f'|$*, as for an enthalpy with a formation term or an absolute pressure with a small slope? Expect to lose about $\log_{10}(|f|/(s|f'|))$ digits; subtract a constant reference from f before differentiating if you can (Section 3.3).
5. *Which method and order?* A step chosen for one rule is wrong for another (Chapter 5).
6. *Unsure?* Use Ridders' method, which chooses its step and estimates its error (Chapter 6), or repeat a stencil at $2h$ and $h/2$ (Section 7.5).

Chapter 4

Higher-order and one-sided stencils

Chapter 2 built the forward and the central difference from Taylor's theorem, and Chapter 3 showed that the central difference reaches about 11 digits in double and no more, because below its best step the rounding of f takes over. This chapter asks what more points buy. The answer is a higher accuracy order, which lets the step grow and the error fall, at a price in rounding, in smoothness and in evaluations. It also answers a practical question that the central difference cannot: how to differentiate at the edge of a domain, where f is defined on one side of x only. The nitrogen example continues: $c_p = \mathrm{d}H/\mathrm{d}T$ of nitrogen at $T_0 = 1000\,\mathrm{K}$, and at the lower end of the fit's range.

4.1 Stencils: nodes, weights and the moment conditions

Every rule of this book has the same shape. It samples f at a few points $x_k = x + n_k h$ and combines the samples with fixed weights:

$$f^{(N)}(x) \approx \frac{1}{h^N} \sum_k w_k \, f(x + n_k h). \tag{4.1}$$

A *node* is an integer offset n_k; the *sample* is $f_k = f(x + n_k h)$; the *stencil* is the pattern of nodes; a *rule* is a stencil with its weights.

The integers n_k are the nodes, the rational numbers w_k the weights, and N the derivative order, which is 1 in this chapter. The central difference has the nodes $-1, 1$ with the weights $-\frac{1}{2}, \frac{1}{2}$; the forward difference the nodes $0, 1$ with the weights $-1, 1$. Where do the weights come from, and how are more of them found?

Taylor's theorem answers both. Expanding each sample about x, $f(x + n_k h) = \sum_j f^{(j)}(x)\, n_k^j h^j / j!$, and collecting the powers of h gives

$$\sum_k w_k \, f(x + n_k h) = \sum_{j \ge 0} \frac{M_j}{j!} \, h^j f^{(j)}(x), \qquad M_j = \sum_k w_k \, n_k^j. \tag{4.2}$$

The numbers M_j are the moments of the rule, and they decide everything. The weighted sum is $h^N f^{(N)}$ plus an error of order h^{N+P} exactly when $M_N = N!$ and every other moment up to M_{N+P-1} vanishes; these are the moment conditions. The first moment that does not vanish, M_{N+P}, gives the leading error term of the rule, $\pm C\, h^P f^{(N+P)}$: its sign is that of M_{N+P}, which the tables of Chapter 9 give, and $C = |M_{N+P}|/(N+P)!$ is its magnitude; P is the accuracy order.

Take the four-point central rule for f', the library's `Central<4>`. Its nodes are $-2, -1, 1, 2$. A rule that is to be exact for the odd function $f(x + t) = t$ and give zero for even functions has antisymmetric weights, $w_{-1} = -a$, $w_1 = a$, $w_{-2} = -b$, $w_2 = b$, so every even moment vanishes by itself. The two odd moments that remain give two conditions, $M_1 = 2a + 4b = 1$ and

$M_3 = 2a + 16b = 0$, whose solution is $a = \frac{2}{3}$ and $b = -\frac{1}{12}$:

$$f'(x) = \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h} + \frac{h^4}{30} f^{(5)}(\xi). \quad (4.3)$$

The error comes from $M_5 = 2a + 64b = -4$: the signed coefficient is $M_5/5! = -1/30$, so $C = 1/30$ and the rule's value is f' minus that term; it underestimates f' where $f^{(5)} > 0$.[1] Two more points have raised the order from 2 to 4. Table 9.1 in Chapter 9 lists the nodes, weights, S and C of every central rule for f', and the general machinery that produces them, one formula over the nodes, is in Sidebar 4.1. One consequence of the moment conditions is worth keeping in mind: a rule of order P is exact for every polynomial of degree up to $N + P - 1$, because all the derivatives in its error terms vanish.

[1] The single intermediate point ξ in the span of the stencil comes from the interpolation error formula (Section B.1); the next term is h^6, since all even moments vanish.

Table 4.1: Nodes and evaluations of `fn` per call of a stencil, for the derivative order N. The central rules omit node 0 for odd N. Ridders' nodes per level are in Table 8.2.

Method	Nodes	Evaluations
`Central<P>`	$P + N - 1$	$P + N - 1$
`Forward<P>`	$P + N$	$P + N$
`Backward<P>`	$P + N$	$P + N$

Sidebar 4.1. The moment conditions and the Lagrange generator

A rule for the N-th derivative of accuracy order P on the nodes $n_1, \ldots, n_m$ satisfies

$$M_j = \sum_k w_k n_k^j = 0 \quad \text{for } j \neq N,\ 0 \le j \le N + P - 1, \qquad M_N = N!,$$

a linear system for the weights. The library does not solve it as a system. It uses the Lagrange formula (Burden et al.,[7] Section 4.1): the polynomial p that interpolates the samples at the nodes, in the scaled variable t with $x + th$, is $p(t) = \sum_k f_k L_k(t)$ with $L_k(t) = \prod_{j \neq k}(t - n_j)/(n_k - n_j)$, and its N-th derivative at $t = 0$ gives the weights directly:

$$w_k = L_k^{(N)}(0) = N!\,[t^N] \prod_{j \neq k} \frac{t - n_j}{n_k - n_j},$$

where $[t^N]$ takes the coefficient of t^N. Why this solves the moment conditions: p reproduces every polynomial of degree below the number of nodes, so for $f = t^j$ the weighted sum equals the N-th derivative of t^j at 0, which is $N!$ for $j = N$ and 0 otherwise (Section B.2). The library's own worked example is the node -1 of `Central<4>`: $\prod_{j \neq k}(t - n_j) = (t+2)(t-1)(t-2) = t^3 - t^2 - 4t + 4$, whose t^1 coefficient is -4, over $\prod(n_k - n_j) = (-1+2)(-1-1)(-1-2) = 6$, so $w = -2/3$, as in Eq. (4.3).

The library stores every rule as integer numerators c_k over one common denominator d, so that $w_k = c_k/d$; for `Central<4>` the numerators are $1, -8, 8, -1$ over 12, and with the integers the moment conditions read $M_N = N!\,d$. This is the form Chapter 9 tabulates, and it keeps the weights exact in every real type. One `consteval` function evaluates the Lagrange formula for every family and order; each family only supplies its nodes. It works in exact rational arithmetic (`long long`, reduced with the greatest common divisor), then puts all weights over their least common multiple, so a table holds integers only. The largest intermediate it reaches is 12! (the end node's product of node differences for `Central<10>` and the fourth derivative; $12! \cdot 4!$ is a bound that is not reached), far inside `long long`. Every numerator and denominator must fit in 24 bits, which keeps the weights exact in float as well as in double (the largest numerator is 6,222,216); the generator checks this before narrowing to `int`, and a type too narrow to hold a rule's integers, such as half precision under `Central<10>`, is rejected by a constraint of `derivative` rather than used with rounded weights. The tests check every table against the moment conditions with a moment function of their own, and against the published rows of Fornberg[15] and Chapra (Chapter 23);[16] they also use

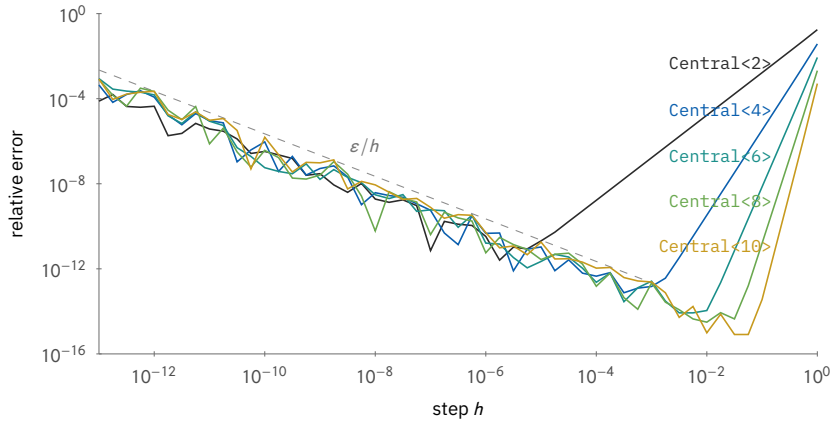


Figure 4.1: Relative error against h for `Central<2>` to `Central<10>`, e^x at 1: steeper truncation slopes, rounding branches that nearly coincide, and minima that slide up the common rounding line as the best step grows. A higher order buys accuracy by letting the step grow; the minimum falls from about $\varepsilon^{2/3}$ to about $\varepsilon^{10/11}$.

the exactness on polynomials, with a point and a step that are short binary fractions so that no rounding occurs, and check that `Central<4>` on x^4 returns exactly 4 at compile time and at run time.

4.2 The order ladder

Figure 4.1 puts the five central rules on one plot. From here to the end of the chapter the function is e^x at $x = 1$ rather than the nitrogen example, because every derivative of e^x equals the function, so the errors of different orders are comparable; the nitrogen example returns in Section 4.3. Three things to see. On the right, where truncation dominates, each curve is a line whose slope is its order P: the error of `Central<4>` falls sixteen times per halving of the step, that of `Central<10>` a thousand times. On the left, where rounding dominates, the curves nearly coincide: the rounding error is $S\varepsilon|f|/h$ with the amplification $S = \sum_k |w_k|$, which grows only from 1.00 to 2.28 across the five rules (Table 9.1). And the minimum of each V slides up that common rounding line to the right: a higher order does not lower the rounding branch, it lets the step grow, and a larger step means a smaller rounding error. The best error falls from about $\varepsilon^{2/3}$ for `Central<2>` to about $\varepsilon^{10/11}$ for `Central<10>`, by the model 3.7×10^{-11} and 5.9×10^{-15}; measured at the default steps the medians over eleven points are 5.8×10^{-12} and 1.6×10^{-15} (Fig. 4.2).

The returns diminish. Figure 4.2 shows the error at the default step against the order: going from `Central<2>` to `Central<4>` gains two decades, from `Central<4>` to `Central<10>` less than two more, and the whiskers show that at a single point the error lies up to about 3.5 times above the median and up to 53 times below it, by luck in the rounding. In double, `Central<4>` is close to as good as it gets for f'; the higher orders earn their keep with wider types, where ε is smaller and $\varepsilon^{P/(P+1)}$ has further to fall (Section 7.6).

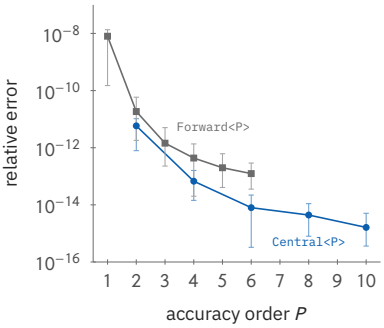


Figure 4.2: Relative error at the default step against P for `Central<P>` and `Forward<P>`, e^x: the median over eleven points in $[0.5, 1]$, the whiskers the smallest and largest value. Diminishing returns beyond 4 or 6 for double.

4.3 The price

What the ladder costs is read off the error model, now written for any rule of order P for f':

$$E(h) = C\,h^P|f^{(P+1)}| + S\,\varepsilon\frac{|f|}{h}, \tag{4.4}$$

with the error constant C and the amplification S defined from the weights as above; the concrete model of Chapter 3 is the case $C = \frac{1}{6}$, $S = 1$. The best step is proportional to $\varepsilon^{1/(P+1)}$ and the best error to $\varepsilon^{P/(P+1)}$, as in Side-

bar 3.2 and Section B.4; the default step is $\varepsilon^{1/(P+1)}s$ with $s = \max(|x|, 1)$, without the constant that C and S would add. On the nitrogen example this is visible in the steps themselves: `Central<2>` differentiates H with a step of 0.0061 K, `Central<4>` with 0.74 K and `Central<6>` with 5.8 K, and the relative errors fall from 2.4×10^{-11} to 2.1×10^{-13} and 3.3×10^{-14}:

```
// cp = dH/dT at T0 with the default step of each rule: more
// points, a higher order, and a larger default step.
auto const d2 = derivative<Central<2>>(H, T0);
auto const d4 = derivative<Central<4>>(H, T0);
auto const d6 = derivative<Central<6>>(H, T0);
```

Three prices come with the larger stencil. The first is rounding of the points. Equation (4.4) accounts for the rounding of the samples f_k, but the points $x + n_k h$ are rounded too, each by up to $\varepsilon|x|/2$, which moves its sample by about that times f'. Over the stencil this adds a third term, $(S/2)\,\varepsilon|xf'|/h$, of the same order as the second. The two-node rules escape it: they divide by the distance between the points actually evaluated rather than by $2h$, which removes the point rounding exactly (Sidebar 3.1). A rule with more nodes cannot. Dividing by the distance between its outer nodes would not remove their rounding but quadruple it, since their weight in `Central<4>` is $1/12$ and the distance carries the rounding with weight $4/12$, and it would leave the inner nodes' rounding untouched; so the rule divides by a fixed multiple of h and keeps the third term, which the tests allow for (Section 10.6).

The second price is smoothness. The error term in Eq. (4.3) holds only if f has $P + N$ continuous derivatives across the whole stencil, which reaches Kh from x on the sides it samples, with K the largest node, and both K and the default step grow with the order: on the nitrogen example `Central<6>` samples up to 5.8 K times three on either side of T_0. A kink or a range break anywhere in that span gives a wrong value without any failure, and a higher order makes it worse, not better; Section 7.8 measures this on the nitrogen example's own range break. The third price is evaluations: $P + N - 1$ per call (Table 4.1), which inside an iteration is the whole cost of the derivative.

Sidebar 4.2. `Central<4>` is Richardson extrapolation in disguise

Let $D(h) = (f(x+h) - f(x-h))/2h$ be the central difference. Its error expansion has even powers only, $D(h) = f'(x) + a_2h^2 + a_4h^4 + \cdots$ (Sidebar 2.1), so halving the step divides the leading term by 4, and one combination removes it:

$$\frac{4D(h/2) - D(h)}{3} = f'(x) - \frac{a_4}{4}h^4 + \cdots .$$

Writing out the four samples with $h' = h/2$ gives $(8f(x+h') - 8f(x-h') - f(x+2h') + f(x-2h'))/(12h')$, which is Eq. (4.3) exactly. Richardson extrapolation at one fixed level is a stencil: the same samples, the same weights, the same error, and no information about how large that error is. What Ridders' method (Chapter 6) adds is not a better formula but a decision: it keeps halving and extrapolating until the entries stop changing by more than rounding, and reads an error estimate off the changes. The price is that the number of samples is decided at run time.

4.4 One-sided rules

A central rule samples both sides of x. Where f exists on one side only, at the first temperature of a correlation, at a phase boundary, at $t = 0$ of a simulation, the samples on the other side are either unavailable or meaningless. The one-sided rules put all their nodes on one side: `Forward<P>` uses the nodes $0, 1, \ldots, P$ for f' and never evaluates below x; `Backward<P>` is its mirror image, the nodes $-P, \ldots, 0$, with the weights reversed and multiplied by $(-1)^N$, and never evaluates above x (Fig. 4.3). Both include x itself, so they also serve at the upper end of a range, where `Backward<P>` stays inside it, and for a function that is cheap to evaluate on one side only.

Each point buys one order. `Forward<P>` has $P+1$ weights and no symmetry to help, so the $P+1$ moment conditions $M_0, \ldots, M_P$ fix them and M_{P+1} is the first that does not vanish: order P from $P+1$ points. A central rule gets its even moments for nothing from antisymmetry, which is why it reaches the same order with one point fewer, and why its orders are even. The one-sided rules pay in their constants. The amplification S of `Forward<1>` to `Forward<6>` runs from 2.00 to 27.7 against 1.00 to 2.28 for the central rules, and the error constant of `Forward<2>` is 0.333 against 0.167 (Table 9.5). The weights alternate in sign and grow, so the samples cancel more and the rounding is amplified more. Figure 4.4 shows the effect: the same truncation slopes as the central rules of the same order, with the rounding branch shifted up. At the default step on e^x, `Forward<2>` reaches 1.8×10^{-11} where `Central<2>` reaches 5.8×10^{-12}, and `Forward<6>` 1.2×10^{-13} where `Central<6>` reaches 8.0×10^{-15} (Fig. 4.2). The one-sided rules stop at order 6, a design decision discussed with the central cap in Section 4.5.

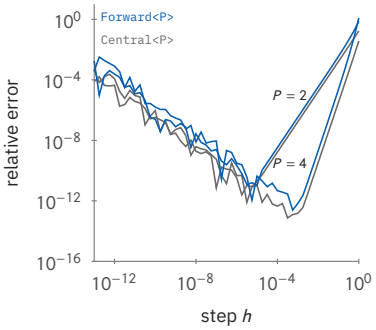


Figure 4.4: `Forward<2>` against `Central<2>` and `Forward<4>` against `Central<4>`: the same slopes, higher floors (a larger S).

The nitrogen example shows why the edge matters. Its Shomate fit is valid from 500 K to 2000 K, but the formula, a polynomial plus a $1/T$ term, evaluates without complaint at 499 K and returns a plausible number that describes nothing: evaluable is not valid. A central rule at 500 K samples below the range and returns a derivative of the extrapolated formula, with no sign that anything is wrong. The remedy is to make the domain explicit. The wrapper below returns NaN outside the fit's range, so that any rule sampling there fails with `non_finite` instead of succeeding; the one-sided rules never sample there and succeed, `Forward<2>` with its three points and `Forward<4>` with five:

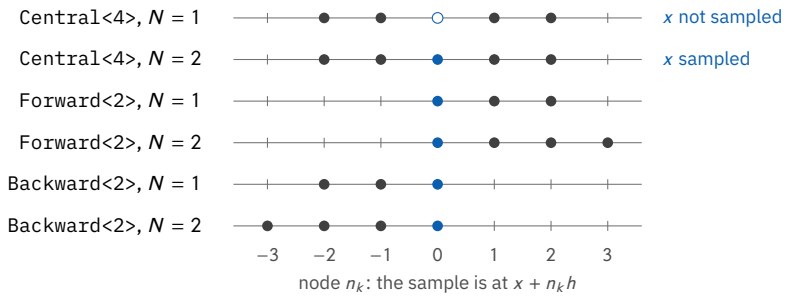


Figure 4.3: The nodes of `Central<4>`, `Forward<2>` and `Backward<2>` for $N = 1$ and $N = 2$ on a line: node 0 filled where the rule uses it. A central rule for an odd derivative does not sample x itself. A diagram without data.

```
// The fit is valid on [T_min, T_max] only. Outside it the
// wrapper returns NaN, so a rule that samples there fails
// instead of returning a plausible number.
auto const H = [](double T) {
    using book::nitrogen;
    if (T < nitrogen.T_min || T > nitrogen.T_max) {
        return std::numeric_limits<double>::quiet_NaN();
    }
    return book::shomate_H(nitrogen, T);
};
double const T_edge = book::nitrogen.T_min; // 500 K
auto const central = derivative<Central<2>>(H, T_edge);
auto const fwd2 = derivative<Forward<2>>(H, T_edge);
auto const fwd4 = derivative<Forward<4>>(H, T_edge);
```

Measured, `Forward<2>` is 4.4×10^{-11} off the closed form and `Forward<4>` 3.0×10^{-13}. Ridders' method, which restarts below a step at which f is not finite (Chapter 6), does not help exactly at an edge: with the same wrapper at 500 K every level samples below the range, each restart halves the step in vain, and the result is non-finite after 64 evaluations, the level cap; near such an edge rather than at it, as for $\log x$ close to 0, it recovers (Section 6.4). Its one-sided form `ForwardRidders` samples at 500 K and above only, and returns the derivative there with an error estimate (Section 6.7). Below 500 K the published enthalpy of nitrogen is a different fit, and the two do not meet, which makes the junction the range break of Section 7.8.

4.5 Choosing P

As a rule: `Central<2>` or `Central<4>` for double inside an iteration, where two or four evaluations buy a relative error of about 5.8×10^{-12} or 6.7×10^{-14} with the default step; `Central<6>` and above only for wide types, where ε is small enough for the order to pay (Section 7.6 uses `Central<6>` with fifty digits); `Forward<2>` or `Forward<4>`, or their mirrors, exactly at an edge and nowhere else, since their constants are larger. For a value to report with an error estimate, use Ridders, or at an edge its one-sided forms. The caps are design decisions recorded in the design record of the repository (`docs/deriv/PHASE2_API.md`), not measured limits. The central rules stop at order 10: by the model's exponent alone, a rule of order 12 would cut the error of f' by a factor of about 1.7 in double and 5.0 in a 50-digit type (3.6 and 62 for the fourth derivative), and the library stops there. The one-sided rules stop at order 6, where the amplification, already 27.7 for f', grows about as fast as the exponent pays (Section 10.4). The decision table by situation is Table 7.1, and every rule's nodes, weights and constants are in Chapter 9.

Chapter 5

Second to fourth derivatives

The slope of a heat capacity, $\mathrm{d}c_p/\mathrm{d}T$, is the second derivative of the enthalpy; the curvature a solver needs for a minimum is a second derivative too, and a stiffness or a Taylor coefficient may need the third or the fourth. The library computes derivatives of order 1 to 4 with every method, through one more template argument. This chapter shows the rules it uses, why each order costs digits and how many, what the default step does per order, what the float warning means, and how Ridders' method extends to higher orders. The nitrogen example's second derivative, $H''(T_0) = \mathrm{d}c_p/\mathrm{d}T$ at 1000 K, runs through it.

5.1 The rules for N = 2 to 4

The derivative order is the second template argument of `derivative`, an integer or an enumerator of `order`: `derivative<Central<2>, 2>(fn, x)` and `derivative<Central<2>, order::second>(fn, x)` are the same call, and `derivative<M>` is `derivative<M, 1>`. The rules behind it come from the same moment conditions as in Section 4.1, with $M_N = N!$ in place of $M_1 = 1$. For the second derivative on the nodes $-1, 0, 1$, a symmetric rule has the weights a, b, a; the conditions $M_0 = 2a + b = 0$ and $M_2 = 2a = 2! = 2$ give $a = 1$ and $b = -2$, and $M_3 = 0$ by symmetry, so

$$f''(x) = \frac{f(x-h) - 2f(x) + f(x+h)}{h^2} - \frac{h^2}{12} f^{(4)}(\xi), \tag{5.1}$$

with the error from $M_4 = 2$ over 4!. This is the three-point rule, the library's `Central<2>` for $N = 2$: the same tag, the same meaning. `Central<P>` is the symmetric rule whose error is $O(h^P)$ for every derivative order, and for the N-th derivative it uses the $P + N - 1$ integers around 0, with node 0 for even N and without it for odd N, since an odd derivative gives the centre no weight by antisymmetry (Fig. 4.3). So `Central<2>` evaluates f three times for f'', four times for f''' on the nodes $\pm1, \pm2$, five times for f'''':

A rule that samples x itself fails with `non_finite` where $f(x)$ is not finite: $\sin x/x$ at 0 works for f' and f''' with the central rules and not for f''.

$$f'''(x) \approx \frac{f(x+2h) - 2f(x+h) + 2f(x-h) - f(x-2h)}{2h^3},$$
$$f''''(x) \approx \frac{f(x+2h) - 4f(x+h) + 6f(x) - 4f(x-h) + f(x-2h)}{h^4},$$

the centred rules of $O(h^2)$ in Chapra's tables (Chapter 23),[16] whose $O(h^4)$ rules are `Central<4>`. The one-sided families work for every order too, `Forward<P>` on the nodes $0, \ldots, P+N-1$, and `Backward<P>` on their mirror image; their amplification grows quickly with N (Table 9.8), but exactly at an edge they are what works. The library's three-point rule for f'' is evaluated in a slightly different form from Eq. (5.1), from the offsets of the points as they were actually rounded (Sidebar 5.2); the two forms agree in exact arithmetic whenever the points are exact. The complete tables for $N = 2$ to 4 are in Chapter 9.

5.2 Why accuracy falls with the order

Figure 5.1 is the V of Chapter 3 for each derivative order, again on e^x at $x = 1$ so that the orders compare. The truncation branches on the right nearly coincide, because every derivative of e^x is e^x and the error constants of the four rules, $\frac{1}{6}$, $\frac{1}{12}$, $\frac{1}{4}$ and $\frac{1}{6}$, differ little. The rounding branches on the left do not. A rounding error of $\varepsilon|f|$ in each sample is divided by h^N, so the branch has slope $-N$: halving the step doubles the rounding error of f' and multiplies that of f'''' by sixteen. The error model of Section 4.3 becomes

$$E(h) = C\,h^P|f^{(P+N)}| + S\,\varepsilon\frac{|f|}{h^N}, \tag{5.2}$$

and its minimum, worked out in Section B.4, is of order $\varepsilon^{P/(P+N)}$. For `Central<2>` in double that is $\varepsilon^{2/3}$, $\varepsilon^{1/2}$, $\varepsilon^{2/5}$ and $\varepsilon^{1/3}$: 3.7×10^{-11}, 1.5×10^{-8}, 5.5×10^{-7} and 6.1×10^{-6}, about 10, 8, 6 and 5 digits. The minima of Fig. 5.1 rise accordingly, and the measured medians at the default step, in Fig. 5.2, follow the model within its usual factor: 1.2×10^{-5} for the fourth derivative against 5.8×10^{-12} for the first. These are the 5 and 11 digits that Chapter 1 promised. What is lost to each order, and why float cannot afford a derivative above the first, is quantified in Sidebar 5.1.

One caution about the word *relative*. The model's rounding term is an absolute error, $S\varepsilon|f|/h^N$, and dividing it by $|f^{(N)}|$ at the default step gives the relative error $S\varepsilon^{P/(P+N)}$ times the factor $|f|/(s^N|f^{(N)}|)$. For e^x at 1 that factor is 1; the figures of Fig. 5.2 presuppose that f varies on the scale s, so that $|f^{(N)}|$ is about $|f|/s^N$. For the nitrogen example's second derivative at T_0 the factor $|H|/(s^2|H''|)$ is 3.7, so the model's relative error is $S = 4$ times that times $\varepsilon^{1/2}$, which is 2.2×10^{-7}, and the measured 6.5×10^{-8} of Section 5.6 lies below it. Near a point where $f^{(N)}$ vanishes the relative error grows without bound while the absolute error stays what the model says; there, judge a result by its absolute error (Section 7.5).

Sidebar 5.1. Why the fourth derivative is so sensitive

By Eq. (5.2) the rounding error of a rule for $f^{(N)}$ is $S\varepsilon|f|/h^N$, so each halving of the step multiplies it by 2^N, and the balance with truncation is reached at a larger step and a larger error: at the best step the error is of order $\varepsilon^{P/(P+N)}$, and the exponent falls from $\frac{2}{3}$ for f' to $\frac{1}{3}$ for f'''' with `Central<2>`. In double that is the difference between about 10 digits and about 5; `Central<4>` does better, $\varepsilon^{4/5}$ to $\varepsilon^{1/2}$, at two more evaluations (Table 9.9).

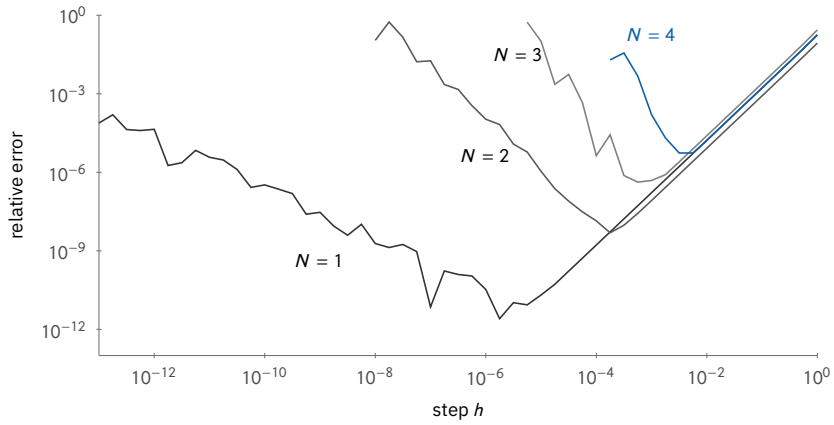


Figure 5.1: The V of `Central<2>` for $N = 1$ to 4, e^x at 1: the truncation branches nearly coincide, the rounding branch steepens as $1/h^N$ and the minimum rises. Why the fourth derivative gives about 5 digits where the first gives 11.

In float, ε is 1.2×10^{-7} and the same exponents leave much less: $\varepsilon^{2/3}$ is 2.4×10^{-5} for f', and $\varepsilon^{1/3}$ is 0.0049 for f'''', two digits by the model. Measured on e^x, the median error of `Central<2>` for the fourth derivative in float is 0.014, and Ridders reaches 0.0019 with a relative estimate of 0.28: the estimate reports the noise honestly. The second and third derivatives keep more, 1.4×10^{-4} and 4.0×10^{-4} with `Central<2>`, but far less than the 5.0×10^{-6} of f', which is why every derivative above the first carries the warning of Section 5.4.

The step sensitivity grows the same way. The rounding term is proportional to h^{-N} and the truncation term to h^P: a step a factor 2 below the right one multiplies the rounding term by 2^N, and a step a factor 2 above it multiplies the truncation term by 2^P. The default step assumes that f varies on the scale s, and at the model's balance misjudging that scale by a factor 2 grows the total error by at most 1.7 for f' and 5.5 for f'''' with `Central<2>`, and 8.0 for `Central<4>` and f''''; the factor 16 that the library's documentation quotes for f'''' is the growth of the rounding term alone, 2^N. A step on f's own scale (Section 3.5) removes the sensitivity; Ridders removes the need to know the scale at all.

5.3 The default step per order and its sensitivity

The default step balances the two terms of Eq. (5.2) under the assumption that f varies on the scale $s = \max(|x|, 1)$, with the constant dropped as in Sidebar 3.2:

$$h = \varepsilon^{1/(P+N)}\, s. \tag{5.3}$$

The exponent is $1/(P+N)$, so the step grows with the order: for `Central<2>` in double it is $\varepsilon^{1/3}$, $\varepsilon^{1/4}$, $\varepsilon^{1/5}$ and $\varepsilon^{1/6}$ times s, that is 6.1×10^{-6}, 1.2×10^{-4}, 7.4×10^{-4} and 0.0025 at $s = 1$; at $T_0 = 1000\,\mathrm{K}$ the step for H'' is 0.12 K, against 0.0061 K for H'. Table 9.9 lists every rule's exponent, step and model error. The constant that the model would add, $(NS/(PC))^{1/(P+N)}$, is 2.6 for f'' and 2.4 for f'''' with `Central<2>`, and dropping it costs more by the model as N grows; yet on test functions that vary faster than s, half the default step was better for every order (Sidebar 3.2), so the library keeps one convention and documents the sensitivity instead (Section B.4).

That sensitivity is the practical point. A step off by a factor 2 from the right one grows the total error, by the model, by up to 1.7 for f' and 5.5 for the fourth derivative with `Central<2>` (Sidebar 5.1), and the rounding term alone by 2^N. When f varies on a scale w much smaller than s, pass $h =$

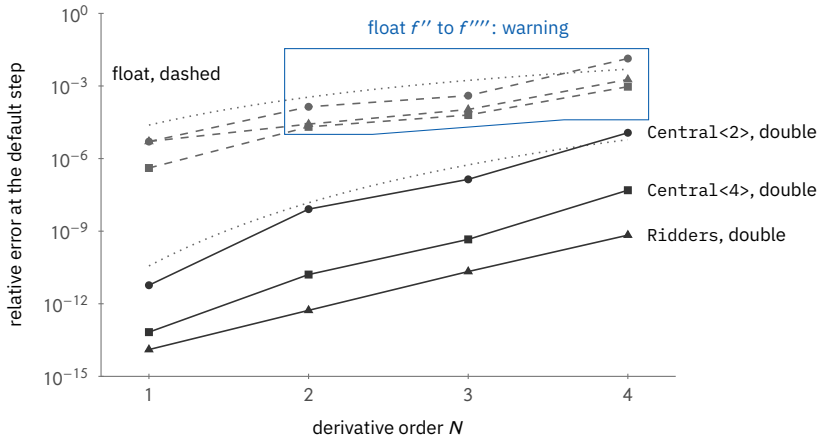


Figure 5.2: Relative error at the default step against N for `Central<2>`, `Central<4>` and Ridders, double and float, e^x (medians over eleven points); the model $\varepsilon^{P/(P+N)}$ of `Central<2>` as a dotted guide line for each type; the float points for f'' to f'''' boxed as the cases that warn.

$\varepsilon^{1/(P+N)}w$ as Section 3.5 does for f'; and since the default step depends on both the method and the order, generic code that varies N should call the default-step overload rather than pass one h for every order.

5.4 The float warning

In float, every derivative above the first keeps few correct digits. Measured as medians on e^x, `Central<2>` is 5.0×10^{-6} off for f' but 1.4×10^{-4}, 4.0×10^{-4} and 0.014 for f'' to f'''', against 1.2×10^{-5} for f'''' in double (Fig. 5.2, Sidebar 5.1); Ridders does better, 2.7×10^{-5} for f'' to 0.0019 for f'''', and its one-sided form of Section 6.7 worse, 0.0022 for f'' to 5.7, more than the value itself, for f''''. The library says so at compile time: a call for a derivative above the first, by any method, in a type whose epsilon is above 10^{-7}, which is float, decimal32 or bfloat16 but not double, selects a deprecated overload of `derivative`, and the compiler reports at the caller's line: "derivatives above the first in a type with 7 or fewer significant digits keep few correct digits: use double, or silence this warning at the call or with NUMERIXX_NO_PRECISION_WARNING". The cure is to use double. Where a float result is wanted all the same, the warning can be silenced for one call or for the whole build, and the result is the same with or without it (Section 8.6, which quotes the call and its numbers); Ridders' estimate then says how little it is worth, 5.3×10^{-4} relative for f'' and 0.28 for f''''.

5.5 Ridders for higher orders

Ridders' method extends to every order with the same tableau: the base of level i is the `Central<2>` rule for N at the step $h_0/2^i$, whose error expansion has even powers of h only, so the extrapolation formula of Chapter 6 applies unchanged. Three things differ. The default initial step is $s/4$ for $N \geq 2$ rather than $s/32$, because the base's rounding error grows as $1/h^N$ and a larger start pays. A level costs 2 evaluations for f' and f'' and 4 for f''' and f'''', plus $f(x)$ once per call for the even orders, which is kept across restarts. And the gain over a stencil grows with the order: for the fourth derivative in double, Ridders reaches a median of 6.8×10^{-10} where `Central<2>` reaches 1.2×10^{-5}, four more digits for a median of 29 evaluations over the random study of Chapter 6, against five. Figure 5.2 shows the three methods side by side.

5.6 The nitrogen example: $\mathrm{d}c_p/\mathrm{d}T$

The slope of the heat capacity at T_0 is $H''(T_0) = 0.005\,744\,\mathrm{J/(mol\,K^2)}$ by the closed form. The three-point rule with its default step of 0.12 K and Ridders from $h_0 = T_0/4$ both compute it in one call each:

```
// d cp / dT = H''(T0): the three-point rule with its
// default step (eps^(1/4) T0, about 0.12 K), and Ridders
// with an error estimate.
auto const d = derivative<Central<2>, 2>(H, T0);
auto const r = derivative<Ridders, order::second>(H, T0);
```

The stencil's relative error is 6.5×10^{-8}, below the model's 2.2×10^{-7} of Section 5.2; Ridders returns a value 3.3×10^{-11} off with an estimate

of 1.3×10^{-12} J/(mol K^2) against an actual error of 1.9×10^{-13} J/(mol K^2), after 17 evaluations. The integer and the enumerator spell the same order, and the two calls return equal results, value and estimate alike:

```
// order::second and the integer 2 are the same template
// argument; the enumeration only reads better in a call.
auto const a = derivative<Central<2>, order::second>(H, T0);
auto const b = derivative<Central<2>, 2>(H, T0);
```

Sidebar 5.2. The three-point f'' from the actual offsets

The two-node rules of Sidebar 3.1 divide by the distance between the points as evaluated, which removes the rounding of the points. The library's three-point rule for f'' does the same with its two offsets. With $x_+ = \mathrm{fl}(x + h)$ and $x_- = \mathrm{fl}(x - h)$ it forms $h_+ = x_+ - x$ and $h_- = x - x_-$ and evaluates

$$f''(x) \approx \frac{\dfrac{f(x_+) - f(x)}{h_+} - \dfrac{f(x) - f(x_-)}{h_-}}{(h_+ + h_-)/2},$$

the second divided difference of the three points where they actually are. The offsets h_+ and h_- are exact for every $0 < h \le |x|$, in any radix, by Sterbenz's lemma (Theorem 4.3.1):[10] for $0 < h \le |x|/2$, x_+ and x lie within a factor 2 of each other and have the same sign, and so do x and x_-, so the subtractions round nothing; for $|x|/2 < h \le |x|$ the subtraction $x - h$ is itself exact by the same lemma, so $x_- = x - h$ and the offset h_- is h, a representable number, with $x_- = 0$ at $h = |x|$, while x_+ stays within a factor 2 of x as before. With exact offsets the formula is exact for every quadratic whatever the rounding of the points, and the point-rounding term that a nominal-step formula carries, about $\varepsilon|xf'|/h^2$, drops out. This mattered most for Ridders in float, whose second-derivative levels use the same form: in the prototype of the design record (`docs/deriv/PHASE4_API.md`) the nominal step gave a worst case of 8.7 times the estimate, the actual offsets 1.4. The form also never computes h^2, which underflows in double for h below about 10^{-154}; every rule with more nodes divides by dh and then by h again, $N - 1$ times, in the order that avoids a spurious underflow or overflow (by h first when $dh > 1 > h$), for the same reason. Only `Central<2>` for f'' uses the offsets; the rules with more nodes divide by the nominal step, as Section 4.3 explains, and Ridders' levels for f''' and f'''' account for the point rounding in their floor instead (Sidebar 6.2).

Chapter 6

Ridders' extrapolation

Every stencil of the last three chapters returns a number and nothing else. Its step is a guess, the default or the user's, and whether the guess was good shows only when the answer is known from elsewhere. Ridders' method,[17] in the form the library implements it, removes the guess and adds the one thing a stencil cannot give: an estimate of its own error. It evaluates the central difference at a step, halves the step, and again, extrapolating the sequence to the limit of zero step until rounding stops the improvement; the size of the last improvement is the estimate. The library offers it as one more method tag, `derivative<Ridders>(fn, x)`, for every derivative order. This chapter explains the extrapolation, shows the tableau on the nitrogen example, says how to read `abs_error` and when not to trust it, what it all costs, and what its one-sided forms do at the edge of a domain.

6.1 Richardson's idea

The central difference $D(h) = (f(x+h)-f(x-h))/2h$ has an error expansion in even powers of the step (Chapter 2, Sidebar 2.1):

$$D(h) = f'(x) + a_2h^2 + a_4h^4 + a_6h^6 + \cdots,$$

with coefficients a_{2m} that depend on f but not on h. Halving the step divides the leading error term by 4, so $D(h)$ and $D(h/2)$ are two measurements with a known relation between their errors, and one combination cancels the term:

$$\frac{4D(h/2) - D(h)}{3} = f'(x) - \frac{a_4}{4}h^4 + \cdots. \tag{6.1}$$

This is Richardson extrapolation,[18,19] the deferred approach to the limit: from two estimates of order h^2, one of order h^4, without knowing a_2. Applied once at a fixed step it is nothing new; Sidebar 4.2 showed that Eq. (6.1) is the four-point rule `Central<4>` written differently. What makes it a method is repetition: the new estimate has the same form, an expansion in even powers with the leading term now h^4, so a second combination with the factor $4^2 = 16$ removes that term, a third with 4^3 the next, and so on, each from values the previous stage already has. Burden et al.[7] (Section 4.2) present the tableau this builds and Sidi[20] the general theory. Ridders' paper supplies the idea; the rules, the ratio 2, a rounding floor, a two-level stall, a final check and restarts, are the library's own, chosen on a prototype (Sidebar 6.2), and every statistic in this chapter is of them.

6.2 The tableau in one picture

Write $h_i = h_0/2^i$ for the step of level i and $D_i = D(h_i)$ for the base difference there. The extrapolated values form a triangular tableau (Fig. 6.1), one row

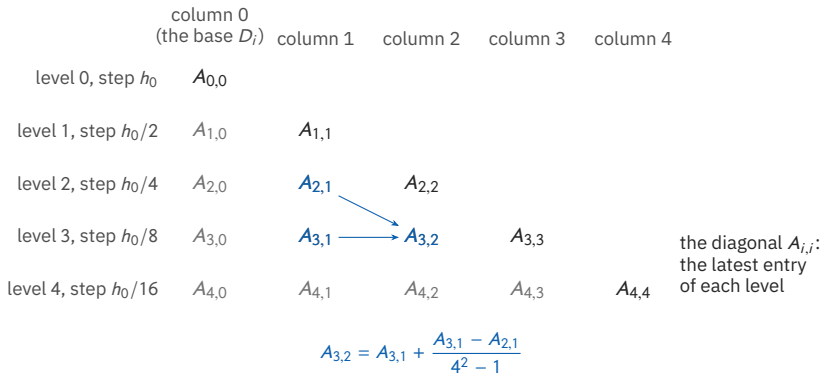


Figure 6.1: The shape of the tableau: column 0 holds the base differences D_i at the halved steps; every other entry is built from its left neighbour and that neighbour's parent one level up. The diagonal entries are the latest of each level. A diagram without data.

per level and one column per cancelled term:

$$A_{i,0} = D_i, \qquad A_{i,j} = A_{i,j-1} + \frac{A_{i,j-1} - A_{i-1,j-1}}{4^j - 1}, \quad 1 \le j \le i. \tag{6.2}$$

Column j is free of the terms up to h^{2j}, and the entry $A_{i,j}$ is built from its left neighbour and that neighbour's parent one level up. The form of Eq. (6.2), a correction added to the left neighbour rather than the weighted combination of Eq. (6.1), is the one Burden et al. write and the library computes, for reasons given in Sidebar 6.1; the two are algebraically the same.

Table 6.1: Ridders' tableau for c_p at T_0 from $h_0 = T_0/32$: the 7 levels the method used, the entries $A_{i,j}$ above and their relative errors with the rounding floor r_i below; the entry Ridders returned (level 4, column 4) in the accent colour.

		entry $A_{i,j}$ in J/(mol K)						
level i	h_i (K)	$j = 0$	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
0	31.25	32.690784416						
1	15.62	32.691493510	32.691729874					
2	7.812	32.691670872	32.691729992	32.691730000				
3	3.906	32.691715218	32.691730000	32.691730000	32.691730000			
4	1.953	32.691726304	32.691730000	32.691730000	32.691730000	**32.691730000**		
5	0.9766	32.691729076	32.691730000	32.691730000	32.691730000	32.691730000	32.691730000	
6	0.4883	32.691729769	32.691730000	32.691730000	32.691730000	32.691730000	32.691730000	32.691730000

		relative error $\lvert A_{i,j} - c_p \rvert / c_p$							floor
level i	h_i (K)	$j = 0$	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$	r_i/c_p
0	31.25	2.9×10^{-5}							4.7×10^{-15}
1	15.62	7.2×10^{-6}	3.9×10^{-9}						9.3×10^{-15}
2	7.812	1.8×10^{-6}	2.4×10^{-10}	2.4×10^{-13}					1.9×10^{-14}
3	3.906	4.5×10^{-7}	1.5×10^{-11}	2.5×10^{-14}	2.2×10^{-14}				3.7×10^{-14}
4	1.953	1.1×10^{-7}	9.9×10^{-13}	5.8×10^{-14}	6.0×10^{-14}	**6.0×10^{-14}**			7.5×10^{-14}
5	0.9766	2.8×10^{-8}	1.1×10^{-13}	5.1×10^{-14}	5.1×10^{-14}	5.1×10^{-14}	5.1×10^{-14}		1.5×10^{-13}
6	0.4883	7.1×10^{-9}	5.4×10^{-15}	1.5×10^{-15}	2.4×10^{-15}	2.6×10^{-15}	2.6×10^{-15}	2.6×10^{-15}	3.0×10^{-13}

Table 6.1 is the tableau for the nitrogen example, $c_p = \mathrm{d}H/\mathrm{d}T$ at T_0, from the default initial step $h_0 = T_0/32$, about 31 K. Read it by columns. Column 0 is the plain central difference, which at a step of thirty kelvin is 2.9×10^{-5} off and gains a factor 4 per level, as Chapter 2 said it would. Column 1 is already `Central<4>` in disguise and is far more accurate from its first entry on; column 2 is at the rounding floor by level 3, and from there on the entries of columns 2 and beyond agree with each other and with the closed form to the last digit shown. The lower table shows what the upper cannot: the relative errors, and in the last column the rounding floor r_i of each level, the error that the rounding of the two samples alone imposes on D_i, which

doubles per level as the step halves. The errors of the extrapolated entries stop falling once they reach the floor, and from then on they drift up with it.

Figure 6.2 continues the same tableau to sixteen levels, beyond where Ridders stopped, so that the whole picture is visible. The base difference alone needs 13 halvings to reach its best, 5.8×10^{-13}, exactly the V of Chapter 3 traced level by level; column 2 is at the floor by level 3. Ridders' method, measured on this call, used 7 levels, 14 evaluations in all, and returned the entry $A_{4,4}$, whose relative error is 6.0×10^{-14}, for the cost of 7 central differences, where `Central<2>` with its default step gave 2.4×10^{-11} for one. The example that makes this call counts the evaluations with a wrapper around `fn`:

```
// Ridders from the default initial step T0 / 32, with fn
// wrapped to count how often it is called.
int calls = 0;
auto const H = [&calls](double T) {
    ++calls;
    return book::shomate_H(book::nitrogen, T);
};
auto const r = derivative<Ridders>(H, T0);
if (r) {
    std::println("cp = {:.12f} +- {:.1e}, {} evaluations",
        r->value, *r->abs_error, calls);
}
```

6.3 Reading `abs_error`

Where does the number after ± come from, and what does it mean? Every extrapolated entry of the tableau carries an estimate of its own error,

$$e_{i,j} = \max\left(|A_{i,j} - A_{i-1,j-1}|,\ r_i\right), \qquad j \geq 1, \tag{6.3}$$

the change of the entry from its parent, but never less than the rounding floor r_i of its level. The change is the textbook estimate of successive approximations: the parent's error is 4^j times that of the entry's left neighbour, and the entry's own error is smaller still, so the difference between entry and parent is nearly the parent's whole error, which overestimates the entry's (Section B.3). The floor is what keeps the estimate honest once truncation is gone: the entries of a converged column differ by rounding

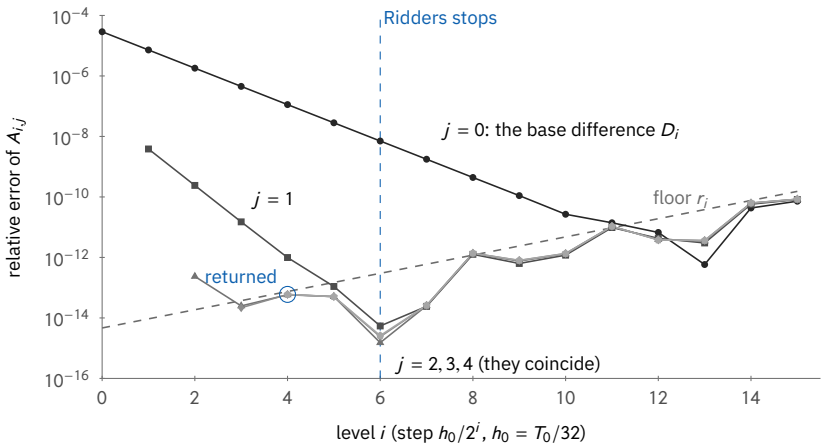


Figure 6.2: Relative error of $A_{i,j}$ against the level i over 16 levels, one curve per column j, the rounding floor r_i dashed: column 0 descends slowly while each higher column reaches the floor within a few levels; the floor doubles per level, so the entries then drift up. Ridders' stop and its returned entry are marked. The extrapolation reaches in a few levels what the base difference would need a dozen for; after that, more levels only add rounding.

alone, the smallest of several such differences is small by luck rather than by accuracy, and without the floor the method would pick it and report an error it cannot vouch for. The floor of level i is the rounding error of the base difference, $(\varepsilon|f(x - h_i)| + \varepsilon|f(x + h_i)|)$ divided by the distance between the two points, under the assumption that each value of `fn` is accurate to about ε.

The method keeps the entry with the smallest estimate, and stops when two further levels produce no smaller one, because each level doubles the floor, so more levels only add rounding (on the nitrogen example, the last two levels of Table 6.1). The returned `abs_error` is that smallest estimate, raised if necessary to the distance between the returned entry and the latest diagonal entry $A_{i,i}$, which catches an entry that only looked good. On the nitrogen example the estimate is 2.7×10^{-12} J/(mol K) against an actual error of 2.0×10^{-12} J/(mol K); note that the entries of the last level are more accurate than the one returned, but their estimates, pinned at a larger floor, cannot say so, and the method reports what it can vouch for rather than what happens to be best. The exact stopping rules, the restart rules and the constants are in Sidebar 6.2, with the evidence that they hold.

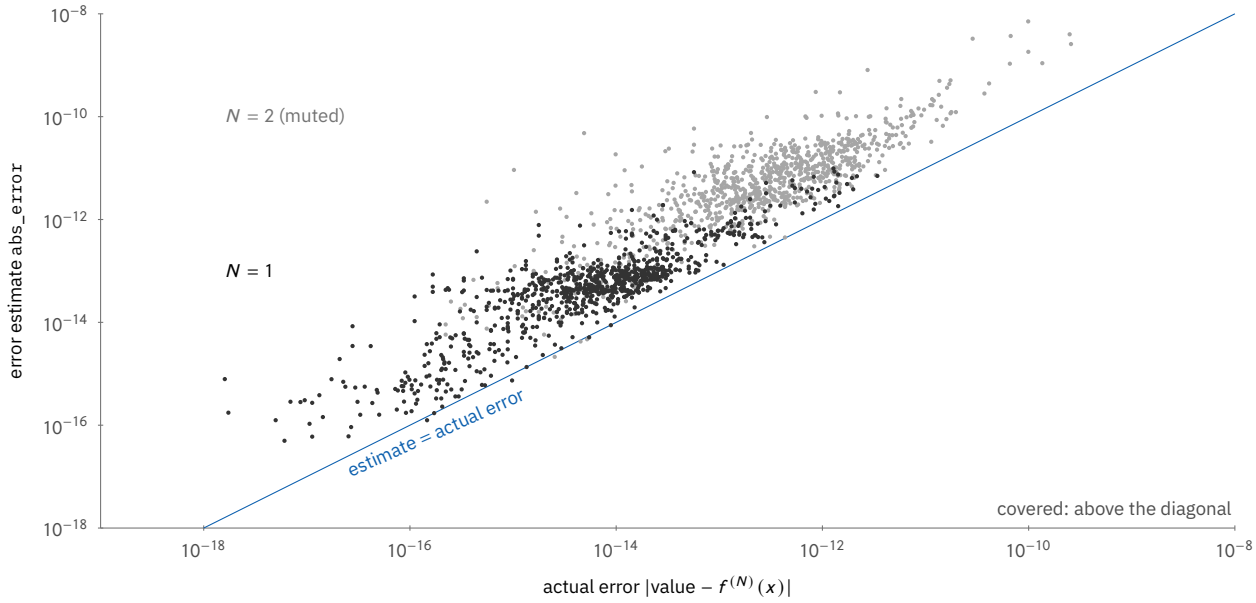


Figure 6.3: Ridders' error estimate against the actual error over the random study ($N = 1$ dark, $N = 2$ muted; 9 smooth functions on $[0.2, 3]$, 300 points each, no noise; no singularity within h_0 for f', while for f'' the initial step $s/4$ reaches past zero below $x = 0.25$, the restarts of Sidebar 6.2), the diagonal drawn: points above it are covered, 99.7 % of the 2700 cases for $N = 1$ and 99.9 % for $N = 2$. Every third case is drawn (900 per order; all enter the statistics). An actual error of exactly 0 (13 and 8 cases) is clipped to $\varepsilon|f^{(N)}|$.

How good is the estimate? Figure 6.3 plots it against the actual error over the book's own study: 9 smooth functions with closed-form derivatives, 300 random points each in $[0.2, 3]$, the default initial step. A point above the diagonal is a case where the estimate covered the error. Measured: 99.7 % of the first derivatives and 99.9 % of the second are covered; the worst uncovered case has an error of 1.8 and 1.4 times its estimate; and the median ratio of error to estimate is 0.14 and 0.059, so the typical estimate is an overestimate by a factor of ten or so. Two things the figure does not say must be said: the estimate is not a bound, and the study's scope is smooth functions, no noise, and for f' no singularity within the initial step

(for the higher orders the larger initial step does reach a singularity at small x, which the method survives by restarting, or, where the samples stay finite, by taking more levels, Sidebar 6.2). Outside that scope is the next section.

6.4 When not to trust it

A non-finite value: the edge of a domain. Near the edge of a domain Ridders' method still works, at a higher cost; exactly at an edge it does not, and its one-sided forms (Section 6.7) or the one-sided stencils of Section 4.4 are the tool. The initial step is $s/32$ for f' with $s = \max(|x|, 1)$, which near an edge may reach across it. The method then sees a NaN or an infinity, discards the tableau and restarts from half the failing step, since every earlier level's interval contained the bad point too. Take $\log x$ at $x = 0.01$: the step $1/32$ samples below zero, and so does $1/64$, so two restarts precede the tableau that converges:

```
// log x at 0.01: the default initial step 1/32 crosses 0,
// where log is NaN. Ridders restarts below the failing
// step; a stencil with its default step (about 6e-6) never
// reaches 0.
int calls = 0;
auto const f = [&calls](double x) {
    ++calls;
    return std::log(x);
};
auto const r = derivative<Ridders>(f, 0.01);
auto const d = derivative<Central<2>>(f, 0.01);
```

Ridders returns 100.000 000 000, accurate to 5.0×10^{-14} relative and covered by its estimate, after 24 evaluations against 16 for $\log x$ away from its edge: four of the extra evaluations are the two failed levels, and the rest a tableau that starts at $1/128$, nearly the size of x, and needs more levels to converge. Figure 6.4 measures how far this goes: each further halving of x adds one failed level, two evaluations; the level budget of 32 is used up from $x = 2^{-27}$; ten correct digits hold down to $x = 9.3 \times 10^{-10}$; a value is still returned down to $x = 5.8 \times 10^{-11}$, with the estimate risen to 0.10 of the value; and below that the result is `non_finite`. As a rule, in its two readings: a value is returned as long as the nearest bad point of f is about $2^{-34}s$ away for f', $2^{-31}s$ for f'' and $2^{-30}s$ for f''' and f'''', whose initial step is $s/4$ and whose rules reach $2h$ (derived from the initial step and the level cap; measured in Section 10.4); for ten digits keep it at least $2^{-30}s$ away (measured for f'). The stencil in the same example does not fail, since its default step of $\varepsilon^{1/3}$ never reaches zero, but its relative error is 1.2×10^{-7}: the truncation $h^2/3x^2$ with a step scaled by $s = 1$ rather than by x, the small-$|x|$ case of Section 3.5, which Ridders handles by adapting the step.

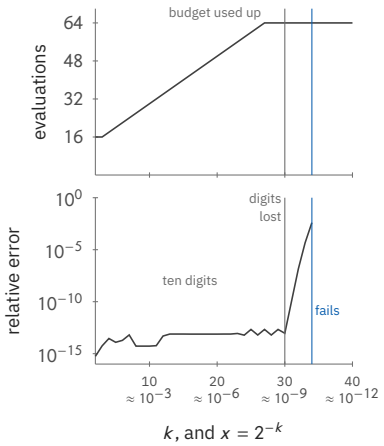


Figure 6.4: Evaluations and relative error of Ridders for $\log x$ at $x = 2^{-k}$ against k: each halving of x costs about two evaluations until the budget is used up at $k = 27$; ten correct digits hold to $k = 30$, digits are lost from there, a value is returned to $k = 34$ and the result is non-finite beyond.

A pole between the samples, or aliasing. The samples can be finite and still lie: a pole of f between two sample points gives finite differences that converge to nothing, and a function that oscillates faster than the initial step can vanish at every sample. The method has a test for a result that has at most two correct bits by its own estimate while that estimate is far above rounding, and restarts at half the smallest step it tried when it fires

(Sidebar 6.3). That restart descends as the one after a non-finite value does, so a pole close to x is resolved: at the tests' poles, 10^{-3} ahead of 0 and 10^{-4} behind it, far inside the limits above, 8 of the 8 results for f' to f'''' are covered, from 1.6×10^{-15} to 3.8×10^{-9} off. Much closer, a pole gives no `non_finite` but a wrong value, flagged by its estimate in every case measured (Section 10.4): test `abs_error`.

A function less accurate than ε. The floor assumes that `fn` is accurate to about ε. A function that carries the noise η of an iterative solver (Section 3.6) breaks that assumption, and the estimate becomes optimistic: measured over the study's 300 random points of e^x with the values perturbed by $\eta = 10^{-8}$, the coverage falls to 90.7 % with a worst ratio of 42 (Fig. 3.4). Pass an initial step on the scale of the noise, and do not read the estimate as a bound.

A jump or a kink inside the span. A range break within h_0 of x does not produce a non-finite value, and Ridders returns a value for the wrong function. Where f itself jumps, it also returns a large estimate, since the base differences then converge like $O(h)$ rather than $O(h^2)$ and the entries keep changing: on the nitrogen example's own range break the stencils return errors of order one in silence, while Ridders' relative estimate is 0.0062, ten orders of magnitude above the 2.7×10^{-13} it reports 50 K from the break (Section 7.8, Fig. 7.2). The estimate flags a jump; the stencil does not. A kink, a jump in f' with f continuous, at x itself shows no symptom at all: the central difference of $c|t|$ is exactly zero at every step, so every central rule returns the mean of the two one-sided slopes plus the smooth part's own error, Ridders' tableau converges to that mean with its estimate at the floor, and nothing in the samples can show it; know the branch boundaries and differentiate the branch you mean (Section 7.8).

6.5 Cost

A level costs 2 evaluations of `fn` for f' and f'' and 4 for f''' and f'''', plus $f(x)$ once per call for the even orders. Measured over the random study (Fig. 6.5), a first derivative takes a median of 14 evaluations, between 8 and 18, and a second 17, between 7 and 21. The tail of the second-derivative histogram, the 332 cases of 19 evaluations or more, is the slower convergence of functions whose higher derivatives are large relative to f'': e^{-x^2} (117), $1/x$ (69), the Runge function (67), $\log x$ (46) and $\sqrt{x}$ (33); the 11 restarts of $\log x$ and $\sqrt{x}$ below $x = 0.25$, where the initial step $s/4$ reaches below zero, all take the most evaluations and are a small part of the last bar. The third and fourth derivatives take about twice as many (Table 10.3). The caps are 32 levels in all, restarts included, which is at most 64, 65, 128 and 129 evaluations for the four orders; Section 10.2 collects the statistics for all four. Against a stencil's two to six evaluations, Ridders takes 2.3 to 7 times as many for f', and for them returns about three more digits than `Central<2>` (medians 5.8×10^{-12} and 1.3×10^{-14} on e^x: 11 against 14 digits), as many as `Central<4>` (6.7×10^{-14}), none over `Central<6>` (8.0×10^{-15}, Table 10.1), and an estimate.

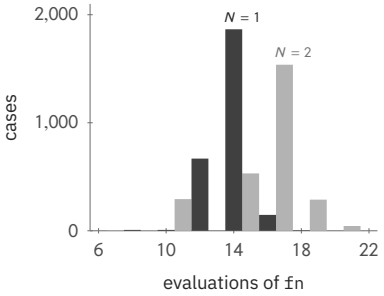


Figure 6.5: Evaluations of `fn` per call over the random study: $N = 1$ dark, $N = 2$ muted. The tail of $N = 2$ is the slower convergence of several functions, not the restarts, which are a small part of its last bar (Section 6.5).

6.6 Stencil or Ridders

Why does only Ridders estimate its error? A stencil could bound its truncation error only with extra evaluations, at a second step or a higher order,

and its rounding part alone is smallest exactly where truncation is largest; an estimate that costs nothing would be worth nothing. So the library leaves a stencil's `abs_error` empty, and a code that needs an estimate asks for Ridders. The choice follows from that. Inside an iteration, where the derivative is recomputed many times and a Newton step tolerates an approximate slope, a stencil with the default step is the right tool: two or four evaluations, a relative error of about 5.8×10^{-12} or 6.7×10^{-14}, no decisions. For a value that is reported, compared or published, Ridders: several times the evaluations, about three more digits than `Central<2>` (14 against 11) and as many as `Central<4>`, and a number that says how far to trust it (a stencil's result can be checked as in Section 7.5).

6.7 One-sided Ridders

Where f is undefined or different on one side of x, at a domain edge, at the end of a correlation's range, or for the right-hand or left-hand slope at a kink, Ridders' method has one-sided forms. `ForwardRidders` extrapolates the `Forward<1>` rule of order N, for f' the quotient $(f(x + h_i) - f(x))/h_i$ (over the actual distance and offsets, Sidebars 3.1 and 5.2), and evaluates `fn` at x and above only, restarts included; `BackwardRidders` applies it to $f(-t)$ at $-x$ and negates the value for odd N, so it samples at x and below, with the same accuracy, cost and limits.

The forward rule's error has every power of the step, $b_1 h + b_2 h^2 + \cdots$, so column j removes the term in h^j and divides by $2^j - 1$ instead of $4^j - 1$: one order per level instead of two, the general case of Burden et al.[7] (Section 4.2; Section B.3). The smaller divisors amplify rounding more, so the floor of an entry is carried through the tableau, $r_{i,j} = (2^j r_{i,j-1} + r_{i-1,j-1})/(2^j - 1)$ from $r_{i,0} = r_i$, rather than taken from its level; the rest, restarts and cap included, is Ridders' method as described. The default initial step is $s/32$ for every order, chosen on a prototype sweep (the design record). A call evaluates $f(x)$ once and N points per level, at most 33 to 129 times, typically 10 for f' and 29 for f''''.

The side has a price. On the study of Fig. 6.3 (the same smooth functions and points, finite above every x, no noise, double) the median relative error runs from 1.5×10^{-13} for f' to 3.5×10^{-6} for f'''', 14 to 3007 times Ridders' on the same cases; the estimate covers 99.9 % to 99.96 % of them, the worst error 1.7 times its estimate. A pole just ahead of x, inside the span the rule samples, can defeat it for f'''', unflagged, well inside the reach below (Section 10.2). Where f is smooth on both sides, use Ridders, also near an edge, where the non-finite values guide its restarts. At the nitrogen fit's lower end, with the wrapper of Section 4.4:

```
// H is NaN below 500 K: Ridders samples there and fails.
double const T = book::nitrogen.T_min; // 500 K
auto const c = derivative<Ridders>(H, T); // non_finite
auto const r1 = derivative<ForwardRidders>(H, T);
auto const r2 = derivative<ForwardRidders, 2>(H, T);
```

Ridders returns non-finite after 64 evaluations. The one-sided form, from an initial step of about 16 K, returns c_p 2.1×10^{-13} off, estimate 2.9×10^{-12}, after 9 evaluations, and $\mathrm{d}c_p/\mathrm{d}T$ 4.5×10^{-9} off, 3.4×10^{-7}, after 17, all relative.

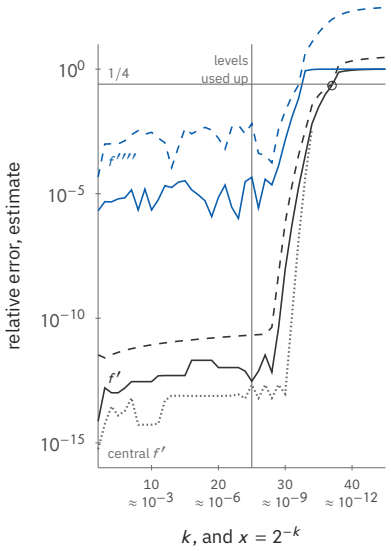


Figure 6.6: The one-sided form for $\log x$ at $x = 2^{-k}$ from its default initial step: relative error (solid), and estimate over the value returned (dashed), for f' (dark) and f'''' (accent); central Ridders' error for f' (dotted). Each keeps its accuracy for some halvings after all its evaluations are spent (the line, for f'); then the one-sided error rises towards 1, covered for a while; further out, to the end of the study, the dashed curve is above the line at 1/4: the result is flagged. At one distance, for f' (the circle), it is neither covered nor flagged. Central Ridders is more accurate and returns no value beyond $k = 34$.

Its reach is limited the other way: where f varies on a scale far below the initial step, as $\log x$ near 0 does, a one-sided base sees no non-finite value, only untrusted results, and restarts at half the smallest step it tried; beyond its reach the levels run out first (Fig. 6.6). On $\log x$ at $x = 2^{-k}$, f' has ten digits to $k = 29$; at the documented reach, $k = 30$, f' is 9.4×10^{-9} off, f'''' 0.0014. The estimate covers the growing error to $k = 36$ for f' (32 for f''''), and from $k = 38$ (33) to the end of the study, $k = 45$, the wrong result is flagged by an estimate of at least a quarter of its value. At $k = 37$ between them, f' is 0.22 off with an estimate of 0.19: neither covered nor flagged. As a rule, keep f's nearest feature at least about $2^{-30}s$ from x, or pass an initial step on its scale.

Sidebar 6.1. Burden's difference form of the tableau

Equation (6.1) generalises to the weighted combination $(4^j A_{i,j-1} - A_{i-1,j-1})/(4^j - 1)$ for column j. Burden et al.[7] (Section 4.2) write the equal correction form,

$$A_{i,j} = A_{i,j-1} + \frac{A_{i,j-1} - A_{i-1,j-1}}{4^j - 1},$$

and the library computes that form, for three reasons. When the two parents agree exactly, the correction is 0 and the entry equals them bit for bit, which is what the stop on exact agreement relies on; the product form could round. The correction is a small quantity divided by a number above 1, so it cannot overflow where the entries themselves are finite, whereas $4^j A_{i,j-1}$ can exceed the largest finite value in float from $j = 7$ on for a derivative near 2^{115}; the library still checks the one subtraction and the one addition before forming them, since two finite entries of opposite sign can differ by more than the largest value, and treats a failure as a failed level. And only the previous row is needed: the tableau is two arrays of 32 entries, filled from `T(0)` so that the real type need not be default-constructible, and swapped after each level. The factors 4^j are built by repeated multiplication in the real type and are exact in binary; $4^j - 1$ needs $2j$ bits and rounds once $2j$ exceeds the significand, from $j = 13$ in float and $j = 27$ in double (far beyond the cap in a 50-digit type), by at most half an epsilon, which does not matter.

Sidebar 6.2. How Ridders stops, and how often its estimate holds

The rules, as the code has them. Level i is the `Central<2>` rule for the order at $h_i = h_0/2^i$; before it is evaluated the method checks that h_i is positive, that $x \pm h_i$ differ from x (and for f''' and f'''' that $x \pm 2h_i$ differ from $x \pm h_i$), and that no point can overflow; at the resolution of x the tableau simply ends. Each extrapolated entry gets the estimate $e_{i,j}$ of Eq. (6.3) with the floor r_i of its level; for f' the floor is the rounding of the two samples over their distance, for f'' the same from the three samples and the two offsets, and for f''' and f'''' the rounding of the samples plus the rounding of the points, $\frac{1}{2}\varepsilon(|x| + 2h)\,|f'|\sum_k |c_k|/(dh^N)$ with the integer numerators c_k over the denominator d and f' taken from the samples at $x \pm h$, since those rules divide by the nominal step. The best entry is the one with the smallest estimate since the last restart. The tableau stops when an entry equals its parent exactly (nothing is left to learn; that entry is best, with the floor as its estimate), when two consecutive levels bring no smaller estimate, at the resolution of x, or when the level budget runs out. The result is the best entry with $\hat{e} = \max(e_{\text{best}}, |A_{\text{best}} - A_{i,i}|)$ for the latest diagonal entry; it needs at least two successful levels since the last restart.

Restarts. A level fails if `fn` is not finite at a point, if a difference, quotient or tableau entry would overflow, or if the points would; the tableau, its best entry and the stall counter are discarded and a new tableau starts from half the failing step. A result is untrusted if $\hat{e} \geq |A_{\text{best}}|/4$, at most two correct bits, and $\hat{e}/64 > r_{\text{best}}$, more than rounding explains; the method then restarts at half the smallest step it tried, as after a failed level, and keeps the untrusted result as a fallback, returned if the budget runs out before a trusted one appears. A tableau that ended at the resolution of x is returned as it is, since no restart could go lower; a derivative that is exactly 0 has an estimate near its floor and is not affected. The budget is 32 levels in all; for the even orders $f(x)$ is evaluated once and kept across restarts, and if it is not finite the result is `non_finite` after that one evaluation.

Where the constants come from. The ratio 2, the two-level stall, the factor 1 on the floor, the factors 4 and 64 of the untrusted test and the cap of 32 were chosen with a prototype on random cases, and the design record in the repository, `docs/deriv/PHASE3_API.md` to `docs/deriv/PHASE5_API.md`, gives the evidence for each: without the floor the estimate fell below the actual error in more than half of the prototype's cases; a one-level stall let a Runge-function case stop early; an untrusted test at $|A|$ (factor 1) instead of $|A|/4$ left poles between samples uncovered; factors between 16 and 256 over the floor gave identical results. The book's own evidence is the study of Fig. 6.3: 9 functions (e^x, $\sin x$, $\log x$, $\sqrt{x}$, $1/x$, the Runge function $1/(1+25x^2)$, x^5, e^{-x^2} and $\tanh x$) at 300 random points each in $[0.2, 3]$, with the default initial step, no noise and no singularity within h_0 for f'. Measured: coverage 99.7 % for f' (9 of 2700 cases uncovered) and 99.9 % for f'' (4); worst ratios of error to estimate 1.8 and 1.4; median ratios 0.14 and 0.059. The uncovered cases have a shape in common: for f', 8 of the 9 are e^{-x^2} at x between 1.6 and 2.9 (the other: x^5 (1)), and for f'' all 4 are e^{-x^2} at x between 2.1 and 2.9, where the derivatives grow like $(2x)^k$ and the first levels are not yet in the asymptotic regime, so the change from the parent undershoots the entry's error; a function whose derivatives grow geometrically on the scale of h_0 is where the estimate is tightest, and a smaller initial step usually restores the margin: a quarter of the default initial step covers 11 of the study's 14 uncovered cases (all four orders), a sixty-fourth 14. There were no restarts for f' and 11 for f'', the cases of $\log x$ and $\sqrt{x}$ below $x = 0.25$, where $h_0 = s/4$ reaches below zero and the first levels are NaN. $1/x$ at the same points does not restart: its pole lies between finite samples, the first levels are simply wrong, and the tableau takes more levels and converges from the later ones, accurately.

Sidebar 6.3. Aliasing: when the samples lie

Take $f(x) = \sin(64\pi x)$ at $x = 0$, whose derivative is 64π. The default initial step is $1/32$, so the first level samples $\sin(\pm 2\pi)$ and the second $\sin(\pm\pi)$: every sample is zero up to rounding, the two base differences nearly agree, and the first extrapolated entry is a confident 0 with a tiny estimate. What saves the call is the sequence: the third level samples $\sin(\pm\pi/2) = \pm 1$, and the diagonal entries jump to numbers of order 100 while the best entry stays near 0. The final check raises the estimate to the distance from the latest diagonal entry, the untrusted test fires, and the method restarts at half the smallest step it tried; the second tableau converges to 64π (measured: the double nearest to 64π, after 20 evaluations). The design record names the case that cannot be caught: samples that are exactly periodic *and* lie on a polynomial, such as $\cos(64\pi x)$ at 0 for f''', whose first samples are all exactly 1: two entries agree exactly and the method stops with the floor as its estimate. That fools every sampling method; the remedy is a step on the scale of the period.

Chapter 7

Using the library

This chapter is for the reader with a problem: a function, a point and a derivative to compute. It covers getting and calling the library, reading and checking the result, choosing a method, three engineering examples, other number types, compile-time use and the pitfalls. Chapter 8 is the exact reference; Chapter 9 holds the tables.

7.1 The call

Getting the library. Numerixx is header-only and has no dependencies. It ships no install rule and no `find_package` configuration, so a CMake project adds it as a subdirectory, with `add_subdirectory`, CPM or `FetchContent`; the target `numerixx::numerixx` adds the include path and requires C++23. With `FetchContent`, at a pinned commit or tag:

```
# Numerixx is header-only, with no install rule or
# find_package config: fetch it at a pinned commit and
# add it as a subdirectory. The target numerixx::numerixx
# adds the include path and requires C++23.
include(FetchContent)
# a tag or a full commit hash; CMake 3.25 or later
FetchContent_Declare(numerixx
    GIT_REPOSITORY https://github.com/troldal/Numerixx.git
    GIT_TAG        90d83378863cfe92efddc062a6e6fc4568d98483)
FetchContent_MakeAvailable(numerixx)

add_executable(app main.cpp)
target_link_libraries(app PRIVATE numerixx::numerixx)
```

The library's tests and this book's examples run, in the continuous integration, on GCC 16, on Clang 22 with libc++, on MSVC and clang-cl of Visual Studio 2026, and on Emscripten 6.0.10. These are the toolchains tested, not minimum versions.

One function. Everything is in `<numerixx/deriv.hpp>`, in the namespace `nxx::deriv`. There is one function, `derivative`, in two forms:

- `derivative<M, N>(fn, x)` uses the method's default step;
- `derivative<M, N>(fn, x, h)` uses the step `h` you pass (for the Ridders methods, the initial step).

The method `M` is a template argument: `Central<P>`, `Forward<P>`, `Backward<P>`, `Ridders`, `ForwardRidders` or `BackwardRidders`, and `Central<2>` when it is left out. The derivative order N is 1 by default; it is an integer from 1 to 4 or a value of the enumeration `order`, such as `order::second`. The number type T is deduced from `x`, and a step of another type converts to it, so `0.01` is a valid step for a 50-digit `x`; for a `float` `x` write `1e-3f`, since a `double` step is diagnosed under `-Wconversion`

unless `float` holds it exactly. `fn` is any callable that takes a T and returns something convertible to T. The call of Chapter 1, `derivative(H, T0)`, is the shortest form: `Central<2>`, the first derivative, the default step. Section 8.2 shows one call of each form, compiled.

A *callable* is anything `std::invoke` can call: a function, a lambda, a function object, or a pointer to a const member function of T itself. For a member function of your model, capture the object: `[&m](double t) { return m.H(t); }`.

A stencil calls `fn` once per node, so `Central<2>` costs two evaluations for f' and three for f''; Ridders' method costs two or four per level, with a median of 14 for f' and 32 for f''' in the random study of Chapter 6 (Table 10.3).

7.2 The result

The library itself throws nothing: a failure is a value. An exception thrown by `fn` or by the operations of T passes through unchanged, and `derivative` is not `noexcept`. The result holds either an estimate or the reason there is none, as a `std::expected<Estimate<T>, error>`. An `Estimate<T>` has two members, `value` and `abs_error`, a `std::optional<T>` filled only by Ridders' method and its one-sided forms. The one reason so far is `error::non_finite`. Test the result, then read it:

```
auto const ln = [](double x) { return std::log(x); };
auto const d = derivative(ln, 0.0); // samples log(-h): NaN
if (d) {
    std::println("slope {}", d->value);
} else if (d.error() == error::non_finite) {
    std::println("no estimate at x = 0: {}", d.error());
}

auto const g = derivative(ln, 2.0); // succeeds
if (g) {
    double const a = g->value;  // the member
    double const b = *g;        // the conversion to double
    // A stencil has no error estimate: abs_error is empty.
    double const e = g->abs_error.value_or(0.0);
    std::println("{} {} {}", a, b, e);
}
```

The central rule at $x = 0$ samples $\log(-h)$, which is NaN, so the first call fails. `error::non_finite` covers every case in which no finite estimate exists: `x` is not finite; `fn` returned an infinite or NaN value; a difference, term, sum or quotient would overflow; a point $x + kh$ would overflow; or h is below the resolution of x (Sidebar 3.3). `error` may gain values, so a `switch` on it needs a `default`. It is formattable: `std::println("{}", d.error())` writes the enumerator's name, `non_finite`. To find the cause, evaluate `fn` at $x \pm h$ yourself and compare h with the spacing of the numbers at x.

The member functions `transform` and `and_then` of `std::expected` apply a function only when there is a value, and pass a failure through; `and_then` chains a second call that can fail itself:

```
// cp in J/(kg K) from cp in J/(mol K): M = 28.0134 g/mol
auto const cp = derivative(H, T0).transform(
    [](double c) { return c / 28.0134e-3; });

// d ln cp / dT = H'' / H' in 1/K: two calls that can each
// fail, combined; the first failure is the result.
auto const slope = derivative(H, T0).and_then(
    [&](double c) {
        return derivative<Central<2>, 2>(H, T0).transform(
            [c](double c2) { return c2 / c; });
    });
```

Both lambdas take a `double`: an `Estimate<double>` converts to its value where a `double` is expected. The conversion is convenient and has its traps, shown in the next snippet:

```
auto const cosine = [](double x) { return std::cos(x); };
auto const r = derivative<Ridders>(cosine, 0.0);
if (r) {                // test first: r holds a result
    auto const e = *r;  // an Estimate<double>, not a double
    double const v = *r; // the value, by the conversion
    // if (*r) would test v != 0: false here, as v is 0.
    std::println("{} +- {:.1e}", v, *e.abs_error);
}
```

First, `auto` deduces `Estimate<double>`, and `std::println` does not apply the conversion: print `e.value`. Second, `if (*r)` tests the value against zero, not whether `r` holds a result: the derivative of cos x at 0 is exactly 0, a good result that tests false. Clang with `-Wconversion -Werror` rejects that line; GCC accepts it without a warning. Third, `r.value()` is the accessor of `std::expected`, not the estimate's member: it compiles, converts to `double` and works until the first failure, where it throws `std::bad_expected_access` (or terminates a program built without exceptions). Read `r->value` after the test. Fourth, `r.value_or(0.0)` does not compile, since no `double` converts to an `Estimate<double>`; `r->abs_error.value_or(0.0)` does. And `int n = *r` truncates, with a `-Wfloat-conversion` warning on GCC and Clang. The rest of the book reads a result as `d->value` and Ridders' estimate as `*r->abs_error`.

Did it work, and is it right? `if (d)` answers the first question only. A step on the wrong scale (Section 3.5), a function less accurate than ε (Section 3.6) or a break within a step of x (Section 7.8) give a wrong value without a failure; Section 7.5 shows how to check one. Ridders' method also returns a value whose own estimate says it is poor, near a bad point or at the resolution of x. Before a Ridders result is used where it matters, compare its estimate with the error you can accept, in two tolerances:

```
// Did it work, and is the estimate within what you accept:
// atol in the derivative's units, rtol relative? A stencil
// result has no estimate and fails the check.
auto const accept = [](auto const &res, double atol,
                       double rtol) {
    return res && res->abs_error
        && *res->abs_error
               <= atol + rtol * std::abs(res->value);
};
auto const at_T0 = derivative<Ridders>(H, T0);
// 1 mK above the range break of 7.8: a large estimate
auto const at_break = derivative<Ridders>(
    [](double T) { return book::nitrogen_H_joined(T); },
    book::junction_T + 1e-3);
std::println("{} {}", accept(at_T0, 0.0, 1e-8),
    accept(at_break, 0.0, 1e-8)); // true false
```

For a value reported to n significant figures, `rtol` is 10^{-n}; `atol`, the error you can accept in the derivative's units, decides near a zero of f' (Section 7.5). Inside an iteration no check is needed (Section 7.4.1). The check says that the estimate is small, not that the value is right where the estimate is blind, as at a kink (Section 7.8).

7.3 Choosing a method

Table 7.1 is the decision table: find the row that describes your situation. Two rules cover most cases. Inside an iteration, where an error estimate would not be read, use a stencil with the default step: `Central<2>`, or `Central<4>` (medians 5.8×10^{-12} and 6.7×10^{-14} for e^x). For a value that will be reported, use Ridders' method: 14 evaluations for f' (median) against two to six for the stencils, about three more digits than `Central<2>` (14 against 11), as many as `Central<4>`, none over `Central<6>` (Table 10.1), and an estimate of its error. The other rows are the exceptions.

7.4 Worked examples

7.4.1 Newton's method for a temperature

An energy balance often gives the enthalpy of a stream and asks for its temperature: solve $H(T) = H^*$ for T. Newton's method needs the slope $\frac{dH}{dT} = c_p$, which `derivative` supplies. This is the classic use of a derivative inside an iteration, so the method is the default `Central<2>`, applied to the nitrogen enthalpy `H(T)`. A failed slope and a failure to converge are both returned as errors, so a caller cannot take an unconverged temperature for a solution:

```
// Why solve_T returns no temperature.
enum class solve_error { no_slope, no_convergence };

struct Newton
{
    double T;  // the solution, K
    int steps; // the iterations it took
```

Table 7.1: Choosing a method. Relative errors in double: the model $\varepsilon^{P/(P+N)}$ of Chapter 4 where named, then measured values, medians for e^x (Table 10.1) or the nitrogen c_p and this chapter's examples. Evaluations of f' unless the row says otherwise; for Ridders' method the study's median and the cap (Table 10.3). w is the scale on which f varies, η its relative accuracy. Above f', `float` warns (Section 8.6).

Situation	Method	Step	Evaluations	Accuracy to expect	Evidence
Inside an iteration; f smooth and cheap	`Central<2>`, or `Central<4>`	default	2, or 4	model $\varepsilon^{2/3} = 3.7 \times 10^{-11}$, measured median 5.8×10^{-12}; `Central<4>` 6.7×10^{-14}	Fig. 4.2, Table 10.1
A value to report, with an error estimate	`Ridders`	default initial step	14, at most 64	about 14 digits; estimate covers in 99.7 % of the study	Fig. 6.3
f defined on one side of x only (an edge of its domain or its validity range)	`Forward<P>` or `Backward<P>`, P = 2 or 4; with an error estimate, `ForwardRidders` or `BackwardRidders`	default	3, or 5; with the estimate 10, at most 33	1.8×10^{-11} (P = 2); with the estimate 1.7×10^{-13}, which covers in 99.96 % of the study	Fig. 4.4, Sections 4.4 and 6.7
f varies on a scale w much smaller than $\lvert x\rvert$, or $\lvert x\rvert \ll 1$ in your units	`Central<2>`, or `Ridders` from $h = w/4$	$\varepsilon^{1/3} w$	2, or Ridders' 14	5.7×10^{-12}, against 2.8×10^{-5} with the default step	Figs. 3.2 and 3.3
f is the output of an iterative solver, accurate to η only (noisy)	`Central<2>`	$\eta^{1/3} \max(\lvert x\rvert, 1)$	2	model $\eta^{2/3} = 2.2 \times 10^{-7}$, measured 7.0×10^{-8} for $\eta = 10^{-10}$, against 5.5×10^{-5} with the default step; Ridders' estimate is optimistic	Fig. 3.4, Section 7.4.3
A second derivative	`Central<2>` or `Ridders`, with $N = 2$; never float	default	3, or 17	model $\varepsilon^{1/2} = 1.5 \times 10^{-8}$, measured 6.5×10^{-8}; Ridders 3.3×10^{-11} (nitrogen)	Fig. 5.1
A third or fourth derivative	`Ridders`, or `Central<4>`; never float	default	4 per level, or 6 to 7	f'''': Ridders 6.8×10^{-10}, `Central<2>` 1.2×10^{-5}; `Central<4>` in Table 10.1	Fig. 5.2
A wide type (50 digits)	`Central<6>` or `Ridders`	default	6, or 22	7.2×10^{-45}; Ridders 9.9×10^{-48}	Section 7.6
A kink or a range break within a step of x	none: differentiate one side, with `Forward<P>` or `Backward<P>`, or with an estimate `ForwardRidders` or `BackwardRidders`	default	as above	any method across the break returns a wrong value; only Ridders' estimate shows a jump, nothing shows a kink at x	Fig. 7.2

```
};

// The temperature at which H(T) = H_target, by Newton's
// method from T: the slope dH/dT = cp from derivative().
std::expected<Newton, solve_error>
solve_T(double H_target, double T)
{
    for (int steps = 1; steps <= 20; ++steps) {
        auto const cp = nxx::deriv::derivative(H, T);
        if (!cp) {
            return std::unexpected(solve_error::no_slope);
        }
        double const dT = (H(T) - H_target) / cp->value;
        T -= dT;
        if (std::abs(dT) < 1e-9) { // K
            return Newton{ T, steps };
        }
    }
    return std::unexpected(solve_error::no_convergence);
}
```

Started at 800 K for the enthalpy of 1000 K, it converges in 4 iterations to within 1.1×10^{-13} K (measured, Fig. 7.1); the final residual, 3.6×10^{-12} J/mol, is the rounding of H itself. A slope that is off by a relative δ adds a term of about δ times the error to each step (Dennis and Schnabel,[12] Chapter 5), and a δ near 2.4×10^{-11} is far too small to slow the convergence before rounding stops it. Each iteration costs three evaluations of H; Ridders' method would cost 14 per slope and buy nothing the iteration can use.

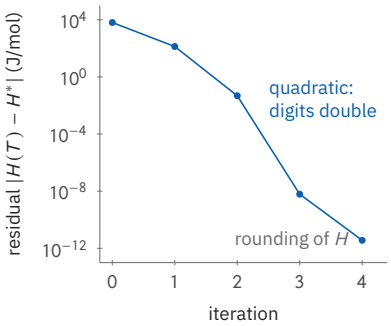


Figure 7.1: Newton's residual $|H(T) - H^*|$ against the iteration, with the slope from `Central<2>`: the correct digits double at each step until the residual reaches the rounding of H.

7.4.2 Parameter sensitivities and a Jacobian

A sensitivity is a derivative of a model output with respect to one input, the others held fixed. Take an Arrhenius rate constant, $k = A\,e^{-E_a/(RT)}$, with the illustrative values $A = 10^{10}$/s, $E_a = 80$ kJ/mol and $T = 600$ K, as a function `rate(A, Ea, T)` of three arguments. `derivative` differentiates a function of one; a lambda that captures the other two makes one:

```
double const A = 1e10, Ea = 80e3, T = 600.0;
// k as a function of one argument, the others held fixed
auto const k_T = [&](double t) { return rate(A, Ea, t); };
auto const k_Ea = [&](double e) { return rate(A, e, T); };
// dk/dT in 1/(s K), dk/dEa in mol/(s J)
auto const dk_dT = derivative<Ridders>(k_T, T);
auto const dk_dEa = derivative<Ridders>(k_Ea, Ea);
if (dk_dT && dk_dEa) {
    std::println("dk/dT = {:.6e} +- {:.1e}", dk_dT->value,
        *dk_dT->abs_error);
}
```

These are values to report, so the method is Ridders'. Against the closed forms $\partial k/\partial T = kE_a/(RT^2)$ and $\partial k/\partial E_a = -k/(RT)$ the measured relative errors are 1.1×10^{-13} and 1.2×10^{-13}, each below its estimate. Engineers usually report the scaled sensitivity $\frac{\mathrm{d}\ln k}{\mathrm{d}\ln E_a} = (E_a/k)\,\partial k/\partial E_a$, the relative change of k per relative change of E_a, which is dimensionless; here it is -16.04, as the exact $-E_a/(RT)$: a small relative error in the activation energy reaches the rate magnified by that factor. The sensitivities to all the parameters are the same call in a loop, whose lambda replaces one parameter and keeps the rest:

```
// The sensitivities dk/dp to the parameters p = (A, Ea):
// one Ridders call each, the other parameter held fixed;
// evals counts the calls of the model.
std::array<double, 2> const p{ A, Ea };
std::array<double, 2> dk_dp{};
int evals = 0;
for (std::size_t j = 0; j < p.size(); ++j) {
    auto const k_j = [&](double v) {
        auto q = p;
        q[j] = v;
        ++evals;
        return rate(q[0], q[1], T);
    };
    auto const d = derivative<Ridders>(k_j, p[j]);
    if (!d) return 1;
    dk_dp[j] = d->value;
```

```
}
```

The cost is the sum over the parameters: 20 evaluations, of which 4 for A (the rate is linear in A, so the first extrapolated entry agrees exactly with its parent and the method stops) and 16 for E_a.

The Jacobian of a set of balances. Newton's method for several unknowns needs the Jacobian of several residuals, which a model usually returns from one call. Take an energy balance $r_1 = nH(T) - Q$ and an ideal-gas pressure $r_2 = nRT/V - P$ in the unknowns T and n, at $T = 1000\,\mathrm{K}$, $n = 2\,\mathrm{mol}$, $V = 1\,\mathrm{m}^3$, with Q and P such that both vanish there. The library differentiates a function of one argument with one value, so it supports one way today: one `derivative` call per entry, on a lambda that evaluates the whole model with one unknown varied, $m \times n$ calls each costing two model evaluations with `Central<2>`: 8 model evaluations here, against 4 by hand (snippet `jacobian-by-calls` in `code/07-using-jacobian.cpp`). For an expensive model, apply the central difference of Chapter 2 to the whole vector yourself, two model calls per unknown, as in the loop of `jacobian(r, x)` in the same file, which fills `J[i][j]` $= \partial r_i/\partial x_j$:

```
for (std::size_t j = 0; j < 2; ++j) {
    double const h = nxx::deriv::default_step(x[j]);
    Vec xp = x, xm = x;
    xp[j] += h;
    xm[j] -= h;
    Vec const rp = r(xp), rm = r(xm);
    for (std::size_t i = 0; i < 2; ++i) {
        J[i][j] = (rp[i] - rm[i]) / (2 * h);
    }
}
```

Here that is 4 model calls, with steps of $0.0061\,\mathrm{K}$ and $1.2 \times 10^{-5}\,\mathrm{mol}$, and every entry is within 2.4×10^{-11} of the closed forms nc_p, H, nR/V and RT/V (measured; 2.4×10^{-11} by the calls). The loop is the formula without the library's checks: it neither detects a non-finite residual nor divides by the distance actually evaluated, so test the residuals it returns.

7.4.3 A function with no formula: a friction factor

The nitrogen enthalpy has a closed-form derivative; most functions in practice do not. The Darcy friction factor λ of turbulent flow in a rough pipe follows from the Colebrook equation, in its common form,[21]

$$\frac{1}{\sqrt{\lambda}} = -2\log_{10}\left(\frac{10^{-4}}{3.7} + \frac{2.51}{Re\,\sqrt{\lambda}}\right) \tag{7.1}$$

for a relative roughness of 10^{-4}, with no closed form: $\lambda(Re)$ is computed by an inner iteration, here bisection on $1/\sqrt{\lambda}$ to a relative tolerance η.

```
// The Darcy friction factor f(Re): the Colebrook equation
// in x = 1/sqrt(f), relative roughness 1e-4, is G(x) = 0.
// G grows with x: bisect on [1, 20] to the tolerance eta.
inline double friction(double Re, double eta)
{
    auto const G = [Re](double x) {
```

```
        return x
            + 2 * std::log10(1e-4 / 3.7 + 2.51 * x / Re);
    };
    double lo = 1.0, hi = 20.0;
    while (hi - lo > eta * hi) {
        double const mid = (lo + hi) / 2;
        if (G(mid) > 0) {
            hi = mid;
        } else {
            lo = mid;
        }
    }
    double const x = (lo + hi) / 2;
    return 1 / (x * x);
}
```

The output of such a solver is accurate to about its tolerance, not to machine epsilon: at $Re = 10^5$ with $\eta = 10^{-10}$ it is 7.2×10^{-11} off, relative, and it changes in steps as Re moves. That is the noisy function of Section 3.6, and its cure applies: the step $\eta^{1/3}s$ instead of the default $\varepsilon^{1/3}s$.

```
double const Re = 1e5, eta = 1e-10;
auto const f = [eta](double re) {
    return book::friction(re, eta);
};
// The default step assumes f accurate to machine epsilon.
auto const d = derivative(f, Re);
// f is accurate to about eta: the step eta^(1/3) Re.
double const h = std::cbrt(eta) * Re;
auto const dh = derivative(f, Re, h);
// Ridders, whose estimate assumes machine epsilon too.
auto const r = derivative<Ridders>(f, Re);
```

The fitted step is the one to use; the default step and Ridders' estimate both assume f accurate to ε. The answer key is the implicit-function derivative of Eq. (7.1) at its root, found to machine precision by Newton's method. With $h = \eta^{1/3}\,Re$, about 46, the relative error is 7.0×10^{-8}, inside the model's $\eta^{2/3} = 2.2 \times 10^{-7}$ (measured); with the default step, 0.61 in units of Re, it is 5.5×10^{-5}, the rounding branch $\eta|\lambda|/h$ at a step much too small. Ridders' estimate happened to cover its error of 1.4×10^{-7} here, but over 300 random points of a function with relative noise 10^{-8} it covered only 90.7 % of the cases (Section 3.6). Here η is known, because the tolerance is on the output itself. When a solver's test is on a residual instead, measure η with the recipe of Section 3.6: on this solver it returns 4.3×10^{-11} relative (the actual deviation is 4.9×10^{-11}) from 8 evaluations.

A solver can also be smooth but wrong by its tolerance, and then no method and no estimate can see it: Ridders' method on a fixed-point iteration stopped at the same tolerance was 2.7×10^{-11} off with a relative estimate of only 1.4×10^{-12} (measured). Converge the inner solver well below the accuracy you want of the derivative, or differentiate the converged equations implicitly.

7.5 Is my derivative good enough?

A stencil returns no error estimate. Four checks, in increasing cost, tell you whether to trust it. The first three use the nitrogen c_p at $T_0 = 1000\,\mathrm{K}$ and count the whole decimal digits on which two values agree, the integer part of $-\log_{10}(|a-b|/|b|)$, with a helper `digits(a, b)`.

Repeat with $2h$ and $h/2$. While truncation dominates, halving the step quarters the error of `Central<2>`; while rounding dominates, it doubles it. Values that agree to many digits at h, $2h$ and $h/2$ are accurate to about that many. `default_step<M, N>(x)` returns the step that the call without one uses (Section 8.2):

```
// Central<2> at its default step h, and at 2h and h/2
double const h = nxx::deriv::default_step(T0);
auto const d = derivative(H, T0, h);
auto const d2 = derivative(H, T0, 2 * h);
auto const d0 = derivative(H, T0, h / 2);
if (!d || !d2 || !d0) return 1;
double const agree = std::min(
    digits(d2->value, d->value),
    digits(d0->value, d->value));
std::println("h, 2h, h/2 agree to {:.0f} digits", agree);
// A physical check: a heat capacity is positive.
if (d->value <= 0) std::println("cp <= 0: wrong");
```

Here they agree to 10 digits, and the value agrees with the closed form to 10 (measured). A jump inside the span does not survive this check: a jump J adds $J/(2h)$ to the central difference, which doubles as the step halves. A kink at x does: every step returns the mean of the two slopes (Section 6.4).

Use a rule of higher order. `Central<4>`, with its own default step and a much smaller error, agrees with `Central<2>` to 10 digits here, which measures the latter's error.

Ask Ridders' method. It gives an independent value with an estimate of its own error:

```
// Ridders: an independent value with an error estimate.
// If that estimate is small, |d - r| is the error of d.
auto const r = derivative<Ridders>(H, T0);
if (r) {
    double const diff = std::abs(d->value - r->value);
    std::println("|d - r| = {:.1e}, Ridders +- {:.1e}",
        diff, *r->abs_error);
}
```

Its estimate, $2.7 \times 10^{-12}\,\mathrm{J/(mol\,K)}$, belongs to Ridders' value, not to the stencil's. But when it is small, the difference $|d-r| = 7.7 \times 10^{-10}\,\mathrm{J/(mol\,K)}$ is the stencil's error to within Ridders' own error, $2.0 \times 10^{-12}\,\mathrm{J/(mol\,K)}$ here, below its estimate; the stencil's actual error is $7.8 \times 10^{-10}\,\mathrm{J/(mol\,K)}$.

Where f' is near zero, judge the absolute error. Near a stationary point of f few digits of f' can agree although the derivative is accurate, because the error of the rule scales with $|f|$, not with $|f'|$. For $\sin x$ just above

$\pi/2$, `Central<2>` gives $f' = -3.673 \times 10^{-6}$ with an absolute error of only 2.7×10^{-12}, in units of f per unit x, while the three values at h, $2h$ and $h/2$ agree in only 6 digits of the small slope (measured, `check-absolute` in `code/07-using-check.cpp`). Compare the spread with $|f|/\max(|x|, 1)$, not with f'.

Finally, a physical check costs nothing: a heat capacity is positive, a reaction rate grows with temperature; a derivative of the wrong sign is wrong whatever the digits say.

7.6 User-defined and multiprecision types

`derivative` works on any type that satisfies the concept `nxx::real`, which includes the `numeric_limits` members the library reads: the built-in floating-point types, Boost.Multiprecision's numbers, or a type of your own (Sidebar 7.1; Section 8.7 gives the list). The algorithms use only +, −, ×, /, comparisons and `numeric_limits`, so a 50-digit type gets 50-digit derivatives:

```
using mp50 = boost::multiprecision::cpp_bin_float_50;
// -> mp50 matters for types with expression templates
// (et_on); cpp_bin_float_50 has none: a safe habit.
auto const f = [](mp50 const &x) -> mp50 { return exp(x); };
mp50 const x = 1;
auto const d = derivative<Central<6>>(f, x);
// A double step converts to mp50: h0 = 1/32.
auto const r = derivative<Ridders>(f, x, 0.03125);
if (!d || !r) return 1;
// Boost's arithmetic needs mp50(*r) or r->value
mp50 const e = exp(mp50(1));
mp50 const error = abs(mp50(*r) - e);
std::println("Central<6>: {}", d->value.str(50));
std::println("Ridders:    {}", r->value.str(50));
std::println("error {} estimate {}", error.str(2),
    r->abs_error->str(2));
```

With `cpp_bin_float_50`, whose machine epsilon is 5.3×10^{-51}, `Central<6>` at its default step is 7.2×10^{-45} off, relative, and Ridders' method 9.9×10^{-48} with a relative estimate of 4.4×10^{-47} after 22 evaluations (measured on e^x at 1). `Central<2>` would stop at 1.5×10^{-35}: a wide type is where the higher orders pay (Table 10.1). Two rules apply with every Boost type. In arithmetic, write `mp50(*r)` or `r->value`: Boost's operators do not see the conversion of an `Estimate`. And give a lambda an explicit `-> mp50`: the library converts a result at once, so an expression over the parameter alone is safe, but one over the lambda's own locals would refer to destroyed values (Sidebar 7.1); the explicit type is always safe.

> **Sidebar 7.1. Expression templates and user-defined real types**
>
> `nxx::real<T>` asks for `std::numeric_limits<T>` specialised as bounded, inexact and non-integer, with the `radix`, `digits`, `epsilon()`, `max()` and `denorm_min()` the algorithms read; construction from `int`, copying, a total order, and +, −, ×, / and unary minus whose results *convert* to T. A type

that lacks one of these fails at the concept, at the call. Section 8.7 quotes a minimal type that provides exactly these members. Exact types such as rationals are rejected, since `numeric_limits` marks them exact (and unbounded): their epsilon is 0 and they have no largest value, so neither the step nor the overflow checks would mean anything.

The results of the operators need not *be* T: Boost.Multiprecision with expression templates (`et_on`) returns from `a + b` an expression object that refers to its operands, which dangles once they are gone. The library never writes `auto` for an arithmetic result and converts the value of `fn` with `static_cast<T>` at once; `-> T` on a lambda does the same in user code. Boost's arithmetic operators are templates that do not consider the conversion of an `Estimate<T>`: `abs(*r - e)` does not compile, with or without expression templates, while `abs(T(*r) - e)` and `r->value - e` do.

7.7 At compile time

`derivative` is `constexpr`: when `fn` can itself be evaluated at compile time, so can its derivative, and the result can feed a `static_assert` or a constant table. With a step that is a power of two, x^3 gives exact results in binary arithmetic, which makes the truncation error of Chapter 2 visible to the compiler:

```
constexpr auto cube = [](double x) { return x * x * x; };
// The truncation error h^2 f'''/6 = 0.0625, exactly:
static_assert(derivative(cube, 2.0, 0.25) == 12.0625);
// Ridders' extrapolation removes it:
static_assert(derivative<Ridders>(cube, 2.0, 0.0625)
              == 12.0);
```

Comparing the result with a value, as in `derivative(cube, 2.0, 0.25) == 12.0625`, compares `value` only, through the conversion, and is false when the result is a failure; the same idiom with `!=` would pass on a failure. Failures are values at compile time too: the nitrogen fit, written to return NaN below its validity range, gives `error::non_finite` for `Central<2>` at 500 K and a value for `Forward<2>`, both checked by `static_assert` in `code/07-using-constexpr.cpp`. The one exception is an overflow of the final quotient, for example a fourth derivative with a tiny step: at run time it is `non_finite`, but GCC rejects it at compile time. Most of `<cmath>`, `exp` and `log` among them, is not `constexpr` in C++23, so such an `fn` is written with arithmetic only; the nitrogen enthalpy is. GCC evaluates some of them in constant expressions as an extension, which Clang and MSVC reject: test compile-time derivatives on every toolchain you ship.

The compiler and the processor compute the same value, provided the compiler neither contracts `a * b + c` into a fused multiply-add nor uses extended precision; the build pins neither, and the identity is verified on the five toolchains of the continuous integration (Section 10.5). How the library avoids that arithmetic is the subject of Sidebar 7.2.

Sidebar 7.2. Overflow guards and constant evaluation

At run time an overflow gives an infinity and a failed check afterwards. In a constant expression it is worse: GCC rejects any floating-point overflow and

any NaN, Clang a NaN and a division by zero, so a failure would become a compile error instead of an error value. The library therefore detects each failure before the arithmetic that would produce it: the input checks keep every point $x + kh$ at least about Kh below Ω, with K the largest node (a NaN x fails them too); every difference and partial sum is tested before it is formed, by comparing $a/\beta + b/\beta$ with Ω/β (β the radix), which is exact and cannot overflow, so the NaN of $\infty - \infty$ never arises; and the rounding floors of Ridders' method saturate at Ω. The exception is the final quotient, with the repeated divisions by h for $N \geq 2$: guarding it exactly would need overflow tests accurate to the last unit, so it is checked afterwards, which GCC rejects in a constant expression.

7.8 Pitfalls

Each pitfall below gives a wrong or missing result without a crash. Symptom first, then the cure.

The step's scale. The default step assumes that f varies on the scale $\max(|x|, 1)$ in your units. *Symptom:* at a large $|x|$ with a narrow feature, Ridders' method or a smaller step disagrees with the default; at a small $|x|$ such as a mole fraction of 10^{-6}, the step is larger than x and the result is `non_finite` or crosses a physical bound. *Cure:* pass $h = \varepsilon^{1/3} w$, with w the scale on which f varies (Figs. 3.2 and 3.3).

A kink or a range break inside the span. A property model is often piecewise: two fits joined at a temperature, a phase boundary, an `if`. The published enthalpy of nitrogen uses one Shomate fit below 500 K and another above, and the two do not meet: H jumps by 0.77 J/mol at 500 K.[5] Every method evaluates it without complaint 1 mK above the break:

```
// H of nitrogen as published: the fit for 100 to 500 K
// below 500 K, the fit for 500 to 2000 K from 500 K on.
auto const H = [](double T) {
    return book::nitrogen_H_joined(T);
};
double const T = book::junction_T + 1e-3; // 1 mK above
auto const c2 = derivative(H, T);              // succeeds
auto const c4 = derivative<Central<4>>(H, T);  // succeeds
auto const r = derivative<Ridders>(H, T);      // succeeds
if (c2 && c4 && r) {
    std::println("{:.4f} {:.4f} {:.4f} +- {:.2f}",
        c2->value, c4->value, r->value, *r->abs_error);
}
```

All three calls succeed. `Central<2>` is 4.3 off, relative (the jump over $2h$), and `Central<4>` 0.041: a higher order does not cure a function that is not smooth. Ridders' value is 0.0019 off, but its estimate, 0.18 J/(mol K) or 0.0062 of the value, is far above its usual size (2.7×10^{-13} of the value away from the break): its extrapolation assumes an error expansion in even powers of h, which the break destroys, so the tableau's entries disagree. At 50 K from the break, outside every span, all three errors are at most 4.3×10^{-12} (measured, Fig. 7.2). Evaluable is not smooth. *Symptom:* a large estimate from Ridders' method, or stencil values that change with the step; a kink at x shows no symptom at all: central rules return the mean of the two slopes (Section 6.4). *Cure:* know the branch boundaries,

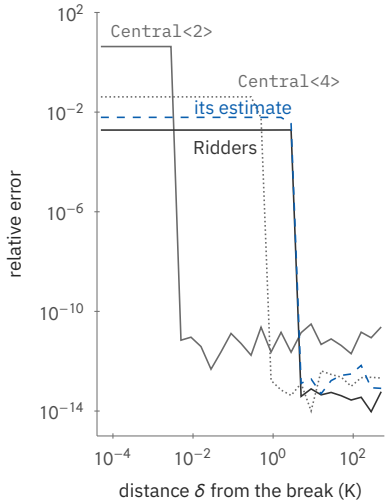


Figure 7.2: Relative error of c_p against the distance δ from the range break at 500 K. While the break lies inside a method's span, the error is a plateau of J/h times the sum of the weights on one side of the break, J the jump in H: $J/(2h)$ for `Central<2>` (4.3 relative, $h = 0.0030$ K); $7J/(12h)$ for `Central<4>` while the break lies within h of x (0.041, $h = 0.37$ K) and $-J/(12h)$ between h and $2h$, the two levels the figure shows. So it falls as $1/h$ across the rules, whatever their order. No rule fails; only Ridders' estimate (dashed) shows a jump.

and differentiate the branch you mean: above a break with `Forward<P>`, or `ForwardRidders` for an error estimate; below it with `Backward<P>` or `BackwardRidders`; or call the branch's own function. At the break itself they return that side's derivative if $f(x)$ belongs to it.

A Ridders result that is a value but poor. Near a bad point or at the resolution of x, Ridders' method can return a value whose own estimate is a large fraction of it. *Symptom:* `*r->abs_error` far above the accuracy you need. *Cure:* the check of Section 7.2; a smaller initial step, or `ForwardRidders` or `BackwardRidders` at an edge.

Aliasing. A function that oscillates on a scale far below $\max(|x|, 1)$ can lie about its slope at every sample: $\sin(64\pi x)$ at $x = 0$, whose first levels sample only zeros (Sidebar 6.3), or $\sin x$ at a large $|x|$, where the default step spans many periods. *Cure:* a step on the oscillation's scale.

A noisy function. A solver's output, or any function whose error changes irregularly from one argument to the next, is accurate to η, not to ε. *Symptom:* the error with the default step is far above $\varepsilon^{2/3}$, and Ridders' estimate is optimistic. *Cure:* the step $\eta^{1/3}s$ (Section 7.4.3, Section 3.6).

A domain edge. Ridders' method restarts below a step at which f is not finite, but needs the nearest bad point a minimum distance away. For $\log x$ with the default step for f', a value is returned down to $x = 2^{-34}$, with an estimate grown to flag the loss; ten correct digits hold only down to $x = 2^{-30}$ (measured, Fig. 6.4); closer, a NaN gives `non_finite`, a pole a flagged wrong value (Section 6.4). Exactly at an edge, use `ForwardRidders` or `BackwardRidders` (Section 6.7), or `Forward<P>` or `Backward<P>` inside an iteration. A fit evaluated outside its range: wrap it to return NaN there, as Section 4.4 does, so that such a sample fails loudly.

Derivatives above the first in float. Few correct digits, and a compile-time warning by every method (Sections 5.4 and 8.6). *Cure:* double.

One step for several orders. The default step differs per method and order, from $\varepsilon^{1/3}s$ for f' to $\varepsilon^{1/6}s$ for f'''' with `Central<2>` (Table 9.9), since the rounding error grows as ε/h^N; a step tuned for f' leaves nothing of f''''. *Cure:* in generic code that varies the order, pass no step.

7.9 What the library does not do

`nxx::deriv` differentiates a function, not a table. Tabulated data must be fitted or interpolated first, and the derivative is then the fit's: a linear interpolant has a piecewise-constant derivative and a kink at every node, so start from a smooth fit or a spline, and use the spline's own derivative when it offers one; numerically, keep the stencil within one interval, or accept the error of a kink (Section 7.8).

Mixed partials and shared evaluations. A mixed partial such as $\partial^2 A/\partial T\,\partial V$ is two nested calls, the outer one in V on a lambda that calls the inner one in T, at the product of their costs. Let the lambda return the inner value, or a NaN when the inner call fails (`d ? d->value : std::numeric_limits<double>::quiet_NaN()`), so that the outer call reports `non_finite`. As a rule of thumb, the inner error, about $\varepsilon^{2/3}$, is the noise level of the outer call: pass it the step $\varepsilon^{2/9}\max(|V|, 1)$ and expect about $\varepsilon^{4/9}$, roughly 10^{-7} in double. Cache an expensive call's last argument and outputs so that its outputs share the evaluations.

Part II

Reference

Chapter 8

API reference

This chapter lists every public name of `nxx::deriv`, and those of the shared header `<numerixx/math.hpp>` that it uses, with its declaration, parameters, preconditions, result, failures and cost, and a pointer to a compiled example. It describes the headers as they are, in their own wording where it summarises a comment of the header. Part I explains the methods, Chapter 7 shows them in use, and Chapters 9 and 10 give the numbers.

8.1 Synopsis

Name	Kind	Section
`derivative`	function template: two overloads, each with a deprecated twin	8.2
`default_step`	function template	8.2
`Estimate<T>`	class template, an aggregate	8.3
`error`	scoped enumeration: `non_finite`, formattable	8.4
`order`	scoped enumeration over `int`: `first` to `fourth`	8.4
`Central<P>`, `Forward<P>`, `Backward<P>`	class templates, empty method tags	8.5
`Ridders`, `ForwardRidders`, `BackwardRidders`	classes, empty method tags	8.5
`NUMERIXX_NO_PRECISION_WARNING`	macro, defined by the user	8.6
`nxx::real`	concept	8.7
`nxx::math::abs`, `isfinite`	function templates	8.7
`nxx::math::root_epsilon`	function template	8.7

Table 8.1: The public names. `deriv.hpp` declares those in `nxx::deriv`, `math.hpp` those in `nxx` and `nxx::math`.

The module is one header, `<numerixx/deriv.hpp>`, which includes `<numerixx/math.hpp>`. Both are header-only, need C++23 and no other library; the CMake target is `numerixx::numerixx`. Table 8.1 lists the public names. Everything in a namespace `detail` is internal: it may change without notice, and the book names it only where the declarations of Section 8.2 do and to generate the tables of Chapter 9. The enumeration `error` may gain values in a later version. The examples of the book name the function, the method tags, `Estimate` and `error` with `using` declarations (`code/07-using-expected.cpp`).

8.2 `derivative`

Declarations. Two overloads, with a step and with the default step.

```
template<typename M = Central<2>, auto N = 1, real T, typename F>
   requires detail::method<M> && detail::valid_order<N>
      && detail::callable<F, T>
      && detail::exact_weights<M, N, T>
[[nodiscard]] constexpr std::expected<Estimate<T>, error>
   derivative(F &&fn, T const &x, std::type_identity_t<T> const &h);

template<typename M = Central<2>, auto N = 1, real T, typename F>
   requires detail::method<M> && detail::valid_order<N>
      && detail::callable<F, T>
      && detail::exact_weights<M, N, T>
[[nodiscard]] constexpr std::expected<Estimate<T>, error>
   derivative(F &&fn, T const &x);
```

Each has a twin with the additional constraint `detail::noisy<N, T>`, marked `[[deprecated]]` (Section 8.6); it returns the same result.

Template parameters.

- `M`, the method: `Central<P>` (by default `Central<2>`), `Forward<P>`, `Backward<P>` (the stencils), or `Ridders`, `ForwardRidders` or `BackwardRidders` (the extrapolation methods). `detail::method<M>` accepts these, the stencils with an accepted order, and nothing else.
- `N`, the derivative order: 1 (the default) to 4, as a value of any integer type except `bool`, or as an `order`. `detail::valid_order<N>` compares an integer in its own type, so a value beyond the range of `int` is not wrapped into 1 to 4.
- `T`, the real type, deduced from `x` alone; it must satisfy `nxx::real`, which includes the `numeric_limits` members of Section 8.7, and hold the method's integer weights exactly, `detail::exact_weights<M, N, T>` (Section 8.5). The step's type, `std::type_identity_t<T>`, is non-deduced, so a `double` step converts to a multiprecision `T`; the conversion is implicit, so for a `float x` write `1e-3f`, since a `double` step is diagnosed under `-Wconversion` unless `float` holds it exactly.
- `F`, the callable. The constraint `detail::callable<F, T>` is `std::invocable<F &, T const &>`, with a result convertible to `T`: `fn` is called as a named lvalue through `std::invoke`, with a `T const` lvalue, so a callable that rejects rvalues works, and so does a pointer to a const member function of `T` itself. For a member function of your own model, capture the object in a lambda.

Parameters. `fn`, the function to differentiate. `x`, the point. `h`, the step of a stencil, or the initial step of an extrapolation method.

The step. A step that is not positive and finite (zero, negative, NaN or infinite) returns `non_finite` before `fn` is called, in every build and in constant expressions. For a stencil, `h` well above ulp(x): a two-node rule fails only when its two points coincide, and near a power of two a step can move x on one side only, so that `Central<2>` divides by a half-ulp distance and returns a finite value that is mostly rounding.

Default steps. For a stencil of accuracy order P, $h = \varepsilon^{1/(P+N)} \max(|x|, 1)$, computed as `root_epsilon<P + N, T>()` times the scale. For Ridders' method the initial step is $\max(|x|, 1)/32$ for f' and $\max(|x|, 1)/4$ for f'' to f''''; for `ForwardRidders` and `BackwardRidders` it is $\max(|x|, 1)/32$ for every order. `default_step` returns the step, so that `derivative<M, N>(fn,`

`x)` and `derivative<M, N>(fn, x, default_step<M, N>(x))` give the same result for every `x` (for a non-finite `x`, `non_finite` from both):

```
template<typename M = Central<2>, auto N = 1, real T>
   requires detail::method<M> && detail::valid_order<N>
[[nodiscard]] constexpr T default_step(T const &x);
```

Pass a step when `fn` varies faster than $\max(|x|, 1)$, or when the derivative is much smaller than the error $\varepsilon^{P/(P+N)}$; use the overload without a step in generic code that varies N (Table 9.9).

Result. An `Estimate<T>` (Section 8.3) whose `abs_error` is filled by the three extrapolation methods and empty for the stencils, or `error::non_finite`.

Failures. `error::non_finite` when no finite estimate exists: `x` is not finite; `fn` returned an infinite or NaN value; a difference, term, sum or quotient would overflow; a point $x + kh$ would overflow; or `h` is below the resolution of `x`: the points of a two-node rule coincide, or, for a rule with more nodes, $x + h$ or $x - h$ rounds to x or neighbouring points round to the same value. For the extrapolation methods, specifically: `fn(x)` is not finite where it is needed (an even N with `Ridders`, every N with the one-sided forms; after one evaluation), or no tableau reached two finite levels. A level at which `fn` is not finite makes the method restart at half that step. An untrusted result, one whose `abs_error` is at least a quarter of |value| and far above rounding, makes it restart too, at half the smallest step it tried; it is nevertheless returned, as a value with its large `abs_error` and not as a failure, when the step reaches the resolution of x or the levels run out during the restarts. Test `abs_error` before relying on such a value (Section 7.2).

Cost. A stencil calls `fn` once per node: $P + N - 1$ times for `Central<P>`, $P + N$ for `Forward<P>` and `Backward<P>`. Ridders' method calls it 2 times per level for f' and f'', 4 times for f''' and f'''', plus `fn(x)` once for f'' and f'''': at most 64, 65, 128 and 129 times. The one-sided forms call it at x once and N times per level, never on the other side of x: at most 33, 65, 97 and 129 times (Table 10.3). `fn` is called only at finite points. No memory is allocated.

Exceptions and constant evaluation. Numerixx throws nothing and catches nothing; an exception thrown by `fn` or by the operations of `T` passes through unchanged, and neither overload is `noexcept`. Both are `constexpr`. Failures are detected before the arithmetic that would produce them, so they are error values in constant expressions too, except an overflow of a final quotient, which GCC rejects at compile time (Sidebar 7.2).

Example. One call of each form, on the nitrogen enthalpy at T_0 (`code/08-api-usage.cpp`):

```
// f' by Central<2> with its default step
auto const a = derivative(H, T0);
// the same rule with the step h = 0.01 K
auto const b = derivative(H, T0, 0.01);
// another method: a template argument
auto const c = derivative<Central<4>>(H, T0);
auto const d = derivative<Forward<2>>(H, T0, 0.1);
// f'': the order as an integer or as an order
```

```
auto const e = derivative<Central<2>, 2>(H, T0);
auto const g =
    derivative<Backward<2>, order::second>(H, T0);
// Ridders: f' and f'', each with an error estimate
auto const r1 = derivative<Ridders>(H, T0);
auto const r2 = derivative<Ridders, order::second>(H, T0);
```

8.3 `Estimate<T>`

```
template<real T> struct Estimate {
   T value;
   std::optional<T> abs_error;
   constexpr operator T() const;
   friend constexpr bool
      operator==(Estimate const &, Estimate const &) = default;
};
```

`value` is the estimate of the derivative. `abs_error` is an estimate of |value – exact|, not a guaranteed bound; it is empty for the stencils. For the extrapolation methods it is the change of the best tableau entry from its parent, but at least the rounding error of the difference it came from (for the one-sided forms, that rounding as the tableau amplifies it) and at least its distance to the latest diagonal entry (Sidebar 6.2). It saturates at `max()` where the estimate would overflow (a large `fn` over a tiny step), and then says only that the method could not estimate its error.

`Estimate` is an aggregate (designated initialisers, structured bindings: `auto const [v, e] = *r`). It converts implicitly to `T`, except where no conversion is considered: `auto`, `std::format` and the arithmetic operator templates of Boost.Multiprecision (its comparisons do accept it); write `e.value` or `T(e)`. It also converts where it may surprise: `if (*r)` tests the value against 0, not whether there is a result (diagnosed by Clang under `-Wconversion`, not by GCC), and `int n = *r` truncates, with a `-Wfloat-conversion` warning (Section 7.2). The defaulted `==` compares both members of two `Estimates`; comparing with a `T`, as `*r == 2.0` or `r == 2.0` on the `std::expected`, compares the value only, and the latter is false when `r` holds an error. `r.value()` is not the member but the accessor of `std::expected`, which throws `std::bad_expected_access` when there is no result (or terminates without exceptions); read `r->value` after testing `r`.

8.4 `error` and `order`

```
enum class error { non_finite };
enum class order : int { first = 1, second = 2, third = 3, fourth = 4 };
```

`error` says why `derivative` returned no estimate; its one value is described in Section 8.2. More values may be added, so a `switch` on it should have a `default`. `deriv.hpp` specialises `std::formatter<error>`: a value formats as its enumerator's name, with the format specification of a string, so `std::format("{:>12}", e)` gives `" non_finite"`. `order` names the derivative orders for readability: `derivative<Ridders, order::second>` is `derivative<Ridders, 2>`. It is scoped, so its names stay out of `nxx::deriv` and it does not convert to `int` elsewhere.

8.5 The methods

```
template<int P> struct Central;                    // even P, 2 <= P <= 10
template<int P> struct Forward;                            // 1 <= P <= 6
template<int P> struct Backward;                           // 1 <= P <= 6
struct Ridders {};
struct ForwardRidders {};
struct BackwardRidders {};
```

Method	Nodes	Evaluations	Default step	Estimate
`Central<P>`	the $P+N-1$ integers around 0; 0 included for even N only	$P+N-1$	$\varepsilon^{1/(P+N)}s$	no
`Forward<P>`	$0, 1, \ldots, P+N-1$	$P+N$	$\varepsilon^{1/(P+N)}s$	no
`Backward<P>`	$-(P+N-1), \ldots, 0$	$P+N$	$\varepsilon^{1/(P+N)}s$	no
`Ridders`	`Central<2>` of order N at $h/2^i$: $\pm h_i$ for $N \le 2$, $\pm h_i, \pm 2h_i$ for $N \ge 3$, and x for even N	2 per level ($N \le 2$), 4 ($N \ge 3$), plus $f(x)$ once for even N; at most 64, 65, 128, 129	$s/32$ ($N = 1$), $s/4$ ($N \ge 2$)	yes
`ForwardRidders`	`Forward<1>` of order N at $h_i = h/2^i$: $x + kh_i$, $k = 0, \ldots, N$	$f(x)$ once, plus N per level; at most 33, 65, 97, 129	$s/32$	yes
`BackwardRidders`	mirrored: $x - kh_i$; value negated for odd N	the same	$s/32$	yes

Table 8.2: The methods for the N-th derivative: nodes in units of h, evaluations of `fn`, default step and error estimate, as `deriv.hpp` documents them. $s = \max(|x|, 1)$. The weights are in Chapter 9.

The methods are empty tags, passed as the first template argument of `derivative`; their weights are internal. P is the accuracy order, the $O(h^P)$ of the error, not the number of points. A tag with an order outside its range is rejected by the constraint of `derivative`; completing the type itself fails a `static_assert` (“`Central<P> needs an even order P from 2 to 10`”, “`Forward<P> needs an order P from 1 to 6`”). A rule whose integer weights the type `T` cannot hold exactly is rejected by the constraint `detail::exact_weights<M, N, T>`, which a `requires`-expression sees; every type with 24 significant bits or more holds them all. The extrapolation methods are checked against the table they extrapolate (Table 8.2). A half-precision type accepts `Central<4>`, for one, but not `Central<10>`.

The two-node rules divide by the distance between the points actually evaluated, `Central<2>` for f'' by its actual offsets (Sidebars 3.1 and 5.2), as do the f' and f'' levels of the one-sided Ridders forms, the other rules by dh and then $N-1$ times by h, in the order that avoids a spurious underflow or overflow (by h first when $dh > 1 > h$). The constants of Ridders’ methods are in Table 9.10.

8.6 The precision warning

A derivative above the first, by any method, in a type whose machine epsilon is above 10^{-7}, tested with integers as $\beta^{\text{digits}-1} < 10^7$ (β the radix: `float`, `decimal32`, `bfloat16`, not `double`), selects the deprecated twin of `derivative`, whose warning reads “derivatives above the first in a type with 7 or fewer significant digits keep few correct digits: use double, or silence this warning at the call or with NUMERIXX_NO_PRECISION_WARNING”. The warning is reported at the caller’s line, so a pragma there silences it (`-Wdeprecated-declarations` on GCC and Clang, C4996 on MSVC), and a system-header include does not hide

it. Defining `NUMERIXX_NO_PRECISION_WARNING` turns it off everywhere; it must be defined alike in every translation unit, best on the command line, since overload resolution would otherwise differ between them. The pragma at the call, for both compiler families:

```
auto const f = [](float x) { return std::exp(x); };
// f'''' in float selects a deprecated overload: its warning
// says it keeps few correct digits. Silence it at the
// call, as here, or for the whole build by defining
// NUMERIXX_NO_PRECISION_WARNING.
#if defined(_MSC_VER) && !defined(__clang__)
#pragma warning(push)
#pragma warning(disable : 4996)
#else
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wdeprecated-declarations"
#endif
auto const d = derivative<Central<2>, 4>(f, 1.0f);
auto const r = derivative<Ridders, 4>(f, 1.0f);
#if defined(_MSC_VER) && !defined(__clang__)
#pragma warning(pop)
#else
#pragma GCC diagnostic pop
#endif
```

The result is the same with or without the warning (accuracy: Section 5.4).

8.7 `nxx::real` and `nxx::math`

```
template<typename T> concept real = /* see below */;

namespace nxx::math {
   template<real T> constexpr T abs(T const &x);
   template<real T> constexpr bool isfinite(T const &x);
   template<int N, real T> constexpr T root_epsilon();
}
```

`real<T>` holds when `std::numeric_limits<T>` is specialised with `is_integer` false, `is_bounded` true and `is_exact` false, so that `max()` is finite and `epsilon()` positive, and with the members the algorithms read: `radix` (at least 2) and `digits` (at least 1) as constant expressions, `epsilon()` and `max()` convertible to `T`, and `denorm_min()` from which `T` can be constructed; when `T` is constructible from `int`, copyable and totally ordered; and when $a+b$, $a-b$, $a\times b$, a/b and $-a$ convert to `T`, which admits expression templates (Sidebar 7.1). The tests check that `float`, `double`, `long double`, `cpp_bin_float_50` with and without expression templates and `cpp_dec_float_50` satisfy it, and that `int`, `unsigned`, `cpp_rational`, a type missing any one of the five members, one with radix 1 and one that cannot be copied do not. A type of your own, here a wrapper of `double` with the arithmetic, a total order and construction from `int` and no default constructor, needs exactly the members below (`code/07-using-usertype.cpp`, which differentiates e^x with it):

```
// numeric_limits with exactly the members derivative()
// uses, none inherited from numeric_limits<double>.
template<>
```

```
struct std::numeric_limits<Real>
{
    using D = std::numeric_limits<double>;
    static constexpr bool is_specialized = true;
    static constexpr bool is_integer = false;
    static constexpr bool is_bounded = true;
    static constexpr bool is_exact = false;
    static constexpr int radix = D::radix;   // 2
    static constexpr int digits = D::digits; // 53
    static constexpr Real epsilon()
    { return Real(D::epsilon()); }
    static constexpr Real max() { return Real(D::max()); }
    static constexpr Real denorm_min()
    { return Real(D::denorm_min()); }
};
```

The helpers use comparisons and $+ - \times /$ only, so they run in constant expressions and accept user types. `abs(x)` returns $|x|$. `isfinite(x)` is false for infinities and NaN: it tests `x == x && abs(x) <= max()`. `root_epsilon<N, T>()` returns $\varepsilon^{1/N}$ for $N \geq 2$ (a `static_assert`), accurate to within an ulp, by Newton's iteration on epsilon scaled into $(\beta^{-N}, 1]$; for `double`, `root_epsilon<3, double>()` is 6.1×10^{-6}, the default step of `Central<2>` for f' at $|x| \leq 1$.

8.8 Rejected calls

What the constraints reject does not compile: the error is reported at the call, not at run time. The example tests each case with a `requires`-expression. A type too narrow for a rule's integer weights is rejected the same way (Section 8.5).

```
constexpr auto sq = [](double x) { return x * x; };

// Whether derivative<M, N>(sq, x) compiles for x of type X.
template<typename M, auto N, typename X = double>
concept accepted =
    requires(X x) { derivative<M, N>(sq, x); };

static_assert(accepted<Central<4>, 1>);
static_assert(!accepted<Central<3>, 1>);  // P must be even
static_assert(!accepted<Central<12>, 1>); // P at most 10
static_assert(!accepted<Forward<7>, 1>);  // P at most 6
static_assert(!accepted<Central<2>, 5>);  // N from 1 to 4
static_assert(!accepted<Central<2>, static_cast<order>(5)>);
static_assert(!accepted<Central<2>, true>); // a bool
static_assert(!accepted<double, 1>);      // not a method
static_assert(!accepted<Central<2>, 1, int>); // x not real
```

A type given explicitly as the second template argument, as in `derivative<Ridders, double>(fn, x)`, does not compile either, because that argument is the order; `T` is deduced from `x`, never from `h`, unless it is given as the third template argument. A callable that cannot be called with a `T const` lvalue, or whose result does not convert to `T`, fails `detail::callable`.

Chapter 9

Tables

The tables of this chapter are the single source of the weights and constants that Part I quotes. Every body was written by the book's data generator from the library itself, from the same internal tables that `derivative` uses (the few constants that are literals of the library's code are transcribed, as Section 9.5 says), and the generator checks each rule against its moment conditions before writing it; the library's tests compare the rules with the published tables of Fornberg[15] and Chapra (Chapter 23).[16]

9.1 How to read the tables

A rule of accuracy order P for the derivative of order N samples f at the nodes $x + n_k h$ and combines the samples with integer numerators c_k over one denominator d:

$$f^{(N)}(x) \approx \frac{1}{d\,h^N} \sum_k c_k\, f(x + n_k h). \tag{9.1}$$

The weights of Chapters 2 and 4 are $w_k = c_k/d$. They are fixed by the moment conditions (Sidebar 4.1): with $M_j = \sum_k c_k n_k^j$,

$$M_N = N!\,d, \qquad M_j = 0 \quad \text{for } j \neq N,\ 0 \le j < N + P. \tag{9.2}$$

The first moment that does not vanish gives the leading error term: the rule's value minus $f^{(N)}(x)$ is about $\pm C h^P f^{(N+P)}(x)$, with the error constant $C = |M_{N+P}|/((N+P)!\,d)$ and the sign given in the last column. The amplification $S = \sum_k |c_k|/d$ multiplies the rounding of the samples: their rounding error reaches the result as about $S\varepsilon|f|/h^N$. Both enter the error model of Chapter 4.

The central rules use the $P + N - 1$ integers around 0, without 0 for odd N, whose weight would be zero. The forward rules use $0, 1, \ldots, P + N - 1$. `Backward<P>` is `Forward<P>` mirrored: its nodes are those of `Forward<P>` negated, and its numerators those of `Forward<P>` times $(-1)^N$, with the same d, S and C; the sign of its leading term changes for odd P. It is not tabulated.

9.2 Central rules

Tables 9.1 to 9.4 give `Central<P>` for $N = 1$ to 4 and $P = 2$ to 10. Each pair of nodes adds two orders; C falls quickly with P and S grows slowly, which is why the central rules of high order pay off in wide types (Table 10.1). The cap at $P = 10$ is a design decision (`docs/deriv/PHASE2_API.md`): by the model's exponent alone, a `Central<12>` would cut the error of f' by a factor of about 1.7 in double and 5.0 in a 50-digit type (for f'''', 3.6 and 62), over a stencil on which f must be smooth that much wider, and the library stops there. The rules with $P = 2$ are the textbook formulas: the

central difference, the three-point second difference, the four-point rule for f''' and the five-point rule for f''''.

Table 9.1: `Central<P>` for the first derivative, f'.

P	Nodes n_k	Numerators c_k	d	S	C	Leading error term
2	$-1, 1$	$-1, 1$	2	1.00	$\frac{1}{6}$	$+\frac{1}{6} h^2 f'''$
4	$-2, -1, 1, 2$	$1, -8, 8, -1$	12	1.50	$\frac{1}{30}$	$-\frac{1}{30} h^4 f^{(5)}$
6	$-3, -2, -1, 1, 2, 3$	$-1, 9, -45, 45, -9, 1$	60	1.83	$\frac{1}{140}$	$+\frac{1}{140} h^6 f^{(7)}$
8	$-4, -3, -2, -1, 1, 2, 3, 4$	$3, -32, 168, -672, 672, -168, 32, -3$	840	2.08	$\frac{1}{630}$	$-\frac{1}{630} h^8 f^{(9)}$
10	$-5, -4, -3, -2, -1, 1, 2, 3, 4, 5$	$-2, 25, -150, 600, -2100, 2100, -600, 150, -25, 2$	2520	2.28	$\frac{1}{2772}$	$+\frac{1}{2772} h^{10} f^{(11)}$

Table 9.2: `Central<P>` for the second derivative, f''.

P	Nodes n_k	Numerators c_k	d	S	C	Leading error term
2	$-1, 0, 1$	$1, -2, 1$	1	4.00	$\frac{1}{12}$	$+\frac{1}{12} h^2 f^{(4)}$
4	$-2, -1, 0, 1, 2$	$-1, 16, -30, 16, -1$	12	5.33	$\frac{1}{90}$	$-\frac{1}{90} h^4 f^{(6)}$
6	$-3, -2, -1, 0, 1, 2, 3$	$2, -27, 270, -490, 270, -27, 2$	180	6.04	$\frac{1}{560}$	$+\frac{1}{560} h^6 f^{(8)}$
8	$-4, -3, -2, -1, 0, 1, 2, 3, 4$	$-9, 128, -1008, 8064, -14350, 8064, -1008, 128, -9$	5040	6.50	$\frac{1}{3150}$	$-\frac{1}{3150} h^8 f^{(10)}$
10	$-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5$	$8, -125, 1000, -6000, 42000, -73766, 42000, -6000, 1000, -125, 8$	25200	6.83	$\frac{1}{16632}$	$+\frac{1}{16632} h^{10} f^{(12)}$

Table 9.3: `Central<P>` for the third derivative, f'''.

P	Nodes n_k	Numerators c_k	d	S	C	Leading error term
2	$-2, -1, 1, 2$	$-1, 2, -2, 1$	2	3.00	$\frac{1}{4}$	$+\frac{1}{4} h^2 f^{(5)}$
4	$-3, -2, -1, 1, 2, 3$	$1, -8, 13, -13, 8, -1$	8	5.50	$\frac{7}{120}$	$-\frac{7}{120} h^4 f^{(7)}$
6	$-4, -3, -2, -1, 1, 2, 3, 4$	$-7, 72, -338, 488, -488, 338, -72, 7$	240	7.54	$\frac{41}{3024}$	$+\frac{41}{3024} h^6 f^{(9)}$
8	$-5, -4, -3, -2, -1, 1, 2, 3, 4, 5$	$205, -2522, 14607, -52428, 70098, -70098, 52428, -14607, 2522, -205$	30240	9.25	$\frac{479}{151200}$	$-\frac{479}{151200} h^8 f^{(11)}$
10	$-6, -5, -4, -3, -2, -1, 1, 2, 3, 4, 5, 6$	$-479, 6840, -46296, 198760, -603315, 764208, -764208, 603315, -198760, 46296, -6840, 479$	302400	10.7	$\frac{59}{79200}$	$+\frac{59}{79200} h^{10} f^{(13)}$

Table 9.4: `Central<P>` for the fourth derivative, $f^{(4)}$. The largest numerator of all the tables, 6,222,216, is the centre weight of `Central<10>`; every numerator fits in 24 bits, so every rule's integer weights are exact in `float`.

P	Nodes n_k	Numerators c_k	d	S	C	Leading error term
2	$-2, -1, 0, 1, 2$	$1, -4, 6, -4, 1$	1	16.0	$\frac{1}{6}$	$+\frac{1}{6} h^2 f^{(6)}$
4	$-3, -2, -1, 0, 1, 2, 3$	$-1, 12, -39, 56, -39, 12, -1$	6	26.7	$\frac{7}{240}$	$-\frac{7}{240} h^4 f^{(8)}$
6	$-4, -3, -2, -1, 0, 1, 2, 3, 4$	$7, -96, 676, -1952, 2730, -1952, 676, -96, 7$	240	34.1	$\frac{41}{7560}$	$+\frac{41}{7560} h^6 f^{(10)}$
8	$-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5$	$-82, 1261, -9738, 52428, -140196, 192654, -140196, 52428, -9738, 1261, -82$	15120	39.7	$\frac{479}{453600}$	$-\frac{479}{453600} h^8 f^{(12)}$
10	$-6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6$	$479, -8208, 69444, -397520, 1809945, -4585248, 6222216, -4585248, 1809945, -397520, 69444, -8208, 479$	453600	44.0	$\frac{59}{277200}$	$+\frac{59}{277200} h^{10} f^{(14)}$

9.3 Forward rules

Tables 9.5 to 9.8 give `Forward<P>` for $N = 1$ to 4 and $P = 1$ to 6. A one-sided rule gains one order per node, and both its constants are larger than those of the central rule of the same order: S grows to 27.7 for f' with $P = 6$ and to 4019 for f''''. The cap at $P = 6$ is a design decision (`docs/deriv/PHASE2_API.md`): beyond it the amplification grows about as fast as the higher order pays, by the model, so the library stops there.

9.4 Default steps and model accuracy

Table 9.9 gives, for every stencil and order, the exponent of the default step $h = \varepsilon^{1/(P+N)} \max(|x|, 1)$, the step itself for `double` at $|x| \le 1$ (for a larger $|x|$, multiply by $|x|$), the model's relative error $\varepsilon^{P/(P+N)}$ at that step, and the

Table 9.5: `Forward<P>` for the first derivative, f'. `Backward<P>`: nodes negated, the same numerators times -1.

P	Nodes n_k	Numerators c_k	d	S	C	Leading error term
1	$0, 1$	$-1, 1$	1	2.00	$\frac{1}{2}$	$+\frac{1}{2}\,h^1\,f''$
2	$0, 1, 2$	$-3, 4, -1$	2	4.00	$\frac{1}{3}$	$-\frac{1}{3}\,h^2\,f'''$
3	$0, 1, 2, 3$	$-11, 18, -9, 2$	6	6.67	$\frac{1}{4}$	$+\frac{1}{4}\,h^3\,f^{(4)}$
4	$0, 1, 2, 3, 4$	$-25, 48, -36, 16, -3$	12	10.7	$\frac{1}{5}$	$-\frac{1}{5}\,h^4\,f^{(5)}$
5	$0, 1, 2, 3, 4, 5$	$-137, 300, -300, 200, -75, 12$	60	17.1	$\frac{1}{6}$	$+\frac{1}{6}\,h^5\,f^{(6)}$
6	$0, 1, 2, 3, 4, 5, 6$	$-147, 360, -450, 400, -225, 72, -10$	60	27.7	$\frac{1}{7}$	$-\frac{1}{7}\,h^6\,f^{(7)}$

Table 9.6: `Forward<P>` for the second derivative, f''. `Backward<P>`: nodes negated, the same numerators.

P	Nodes n_k	Numerators c_k	d	S	C	Leading error term
1	$0, 1, 2$	$1, -2, 1$	1	4.00	1	$+\,h^1\,f'''$
2	$0, 1, 2, 3$	$2, -5, 4, -1$	1	12.0	$\frac{11}{12}$	$-\frac{11}{12}\,h^2\,f^{(4)}$
3	$0, 1, 2, 3, 4$	$35, -104, 114, -56, 11$	12	26.7	$\frac{5}{6}$	$+\frac{5}{6}\,h^3\,f^{(5)}$
4	$0, 1, 2, 3, 4, 5$	$45, -154, 214, -156, 61, -10$	12	53.3	$\frac{137}{180}$	$-\frac{137}{180}\,h^4\,f^{(6)}$
5	$0, 1, 2, 3, 4, 5, 6$	$812, -3132, 5265, -5080, 2970, -972, 137$	180	102	$\frac{7}{10}$	$+\frac{7}{10}\,h^5\,f^{(7)}$
6	$0, 1, 2, 3, 4, 5, 6, 7$	$938, -4014, 7911, -9490, 7380, -3618, 1019, -126$	180	192	$\frac{363}{560}$	$-\frac{363}{560}\,h^6\,f^{(8)}$

Table 9.7: `Forward<P>` for the third derivative, f'''. `Backward<P>`: nodes negated, the same numerators times -1.

P	Nodes n_k	Numerators c_k	d	S	C	Leading error term
1	$0, 1, 2, 3$	$-1, 3, -3, 1$	1	8.00	$\frac{3}{2}$	$+\frac{3}{2}\,h^1\,f^{(4)}$
2	$0, 1, 2, 3, 4$	$-5, 18, -24, 14, -3$	2	32.0	$\frac{7}{4}$	$-\frac{7}{4}\,h^2\,f^{(5)}$
3	$0, 1, 2, 3, 4, 5$	$-17, 71, -118, 98, -41, 7$	4	88.0	$\frac{15}{8}$	$+\frac{15}{8}\,h^3\,f^{(6)}$
4	$0, 1, 2, 3, 4, 5, 6$	$-49, 232, -461, 496, -307, 104, -15$	8	208	$\frac{29}{15}$	$-\frac{29}{15}\,h^4\,f^{(7)}$
5	$0, 1, 2, 3, 4, 5, 6, 7$	$-967, 5104, -11787, 15560, -12725, 6432, -1849, 232$	120	455	$\frac{469}{240}$	$+\frac{469}{240}\,h^5\,f^{(8)}$
6	$0, 1, 2, 3, 4, 5, 6, 7, 8$	$-2403, 13960, -36706, 57384, -58280, 39128, -16830, 4216, -469$	240	956	$\frac{29531}{15120}$	$-\frac{29531}{15120}\,h^6\,f^{(9)}$

Table 9.8: `Forward<P>` for the fourth derivative, $f^{(4)}$. `Backward<P>`: nodes negated, the same numerators.

P	Nodes n_k	Numerators c_k	d	S	C	Leading error term
1	$0, 1, 2, 3, 4$	$1, -4, 6, -4, 1$	1	16.0	2	$+2\,h^1\,f^{(5)}$
2	$0, 1, 2, 3, 4, 5$	$3, -14, 26, -24, 11, -2$	1	80.0	$\frac{17}{6}$	$-\frac{17}{6}\,h^2\,f^{(6)}$
3	$0, 1, 2, 3, 4, 5, 6$	$35, -186, 411, -484, 321, -114, 17$	6	261	$\frac{7}{2}$	$+\frac{7}{2}\,h^3\,f^{(7)}$
4	$0, 1, 2, 3, 4, 5, 6, 7$	$56, -333, 852, -1219, 1056, -555, 164, -21$	6	709	$\frac{967}{240}$	$-\frac{967}{240}\,h^4\,f^{(8)}$
5	$0, 1, 2, 3, 4, 5, 6, 7, 8$	$3207, -21056, 61156, -102912, 109930, -76352, 33636, -8576, 967$	240	1741	$\frac{89}{20}$	$+\frac{89}{20}\,h^5\,f^{(9)}$
6	$0, 1, 2, 3, 4, 5, 6, 7, 8, 9$	$4275, -30668, 99604, -192624, 244498, -210920, 123348, -47024, 10579, -1068$	240	4019	$\frac{4523}{945}$	$-\frac{4523}{945}\,h^6\,f^{(10)}$

number of evaluations. The steps are `nxx::math::root_epsilon<P + N, double>()`; `default_step<M, N>(x)` returns them times the scale. The model error drops the constants C and S and assumes that f varies on the scale $\max(|x|, 1)$, $|f| \sim s^N |f^{(N)}|$; it is a rule of thumb, and the measured accuracies are in Table 10.1.

9.5 Ridders' constants

Table 9.10 collects the constants of Ridders' method and of its one-sided forms, the same for every derivative order except where the value depends on N; Chapter 6 and Sidebar 6.2 explain them, and the design record in the repository (`docs/deriv/PHASE3_API.md` to `docs/deriv/PHASE5_API.md`) gives the evidence for each choice. The level cap, the initial-step divisors and the one-sided evaluations per level are read from the library; the ratio, the stall and the two factors of the untrusted-result test are literals of its code, transcribed by the generator.

Method	P	N	Exponent	Step	Model error	Evaluations
`Central<2>`	2	1	1/3	6.1×10^{-6}	3.7×10^{-11}	2
`Central<4>`	4	1	1/5	7.4×10^{-4}	3.0×10^{-13}	4
`Central<6>`	6	1	1/7	0.0058	3.8×10^{-14}	6
`Central<8>`	8	1	1/9	0.018	1.2×10^{-14}	8
`Central<10>`	10	1	1/11	0.038	5.9×10^{-15}	10
`Central<2>`	2	2	1/4	1.2×10^{-4}	1.5×10^{-8}	3
`Central<4>`	4	2	1/6	0.0025	3.7×10^{-11}	5
`Central<6>`	6	2	1/8	0.011	1.8×10^{-12}	7
`Central<8>`	8	2	1/10	0.027	3.0×10^{-13}	9
`Central<10>`	10	2	1/12	0.050	9.0×10^{-14}	11
`Central<2>`	2	3	1/5	7.4×10^{-4}	5.5×10^{-7}	4
`Central<4>`	4	3	1/7	0.0058	1.1×10^{-9}	6
`Central<6>`	6	3	1/9	0.018	3.7×10^{-11}	8
`Central<8>`	8	3	1/11	0.038	4.1×10^{-12}	10
`Central<10>`	10	3	1/13	0.062	9.1×10^{-13}	12
`Central<2>`	2	4	1/6	0.0025	6.1×10^{-6}	5
`Central<4>`	4	4	1/8	0.011	1.5×10^{-8}	7
`Central<6>`	6	4	1/10	0.027	4.1×10^{-10}	9
`Central<8>`	8	4	1/12	0.050	3.7×10^{-11}	11
`Central<10>`	10	4	1/14	0.076	6.6×10^{-12}	13
`Forward<1>`	1	1	1/2	1.5×10^{-8}	1.5×10^{-8}	2
`Forward<2>`	2	1	1/3	6.1×10^{-6}	3.7×10^{-11}	3
`Forward<3>`	3	1	1/4	1.2×10^{-4}	1.8×10^{-12}	4
`Forward<4>`	4	1	1/5	7.4×10^{-4}	3.0×10^{-13}	5
`Forward<5>`	5	1	1/6	0.0025	9.0×10^{-14}	6
`Forward<6>`	6	1	1/7	0.0058	3.8×10^{-14}	7
`Forward<1>`	1	2	1/3	6.1×10^{-6}	6.1×10^{-6}	3
`Forward<2>`	2	2	1/4	1.2×10^{-4}	1.5×10^{-8}	4
`Forward<3>`	3	2	1/5	7.4×10^{-4}	4.1×10^{-10}	5
`Forward<4>`	4	2	1/6	0.0025	3.7×10^{-11}	6
`Forward<5>`	5	2	1/7	0.0058	6.6×10^{-12}	7
`Forward<6>`	6	2	1/8	0.011	1.8×10^{-12}	8
`Forward<1>`	1	3	1/4	1.2×10^{-4}	1.2×10^{-4}	4
`Forward<2>`	2	3	1/5	7.4×10^{-4}	5.5×10^{-7}	5
`Forward<3>`	3	3	1/6	0.0025	1.5×10^{-8}	6
`Forward<4>`	4	3	1/7	0.0058	1.1×10^{-9}	7
`Forward<5>`	5	3	1/8	0.011	1.6×10^{-10}	8
`Forward<6>`	6	3	1/9	0.018	3.7×10^{-11}	9
`Forward<1>`	1	4	1/5	7.4×10^{-4}	7.4×10^{-4}	5
`Forward<2>`	2	4	1/6	0.0025	6.1×10^{-6}	6
`Forward<3>`	3	4	1/7	0.0058	2.0×10^{-7}	7
`Forward<4>`	4	4	1/8	0.011	1.5×10^{-8}	8
`Forward<5>`	5	4	1/9	0.018	2.0×10^{-9}	9
`Forward<6>`	6	4	1/10	0.027	4.1×10^{-10}	10

Table 9.9: Default steps, model accuracy and cost per stencil and derivative order, for `double`. Step at $|x| \le 1$; the model error is relative.

The level cap counts every level attempted, restarts included; with the evaluations per level it gives the maximum evaluations of the last row.

Constant	`Ridders`	`ForwardRidders`, `BackwardRidders`
Ratio of the steps of consecutive levels	2	2
Initial step, divisor of $\max(\|x\|, 1)$	32 (f'), 4 (f'' to f'''')	32
Column divisor of the tableau, column j	$4^j - 1$	$2^j - 1$
Rounding floor of an entry	that of its level	carried through the tableau
Levels without improvement before the stop	2	2
Untrusted result: estimate at least $\|A\|/4$ and more than 64 times the floor	4, 64	4, 64
Restart after a failed level or an untrusted result	at half the smallest step tried	the same
Level cap (restarts included)	32	32
Evaluations per level	2, 2, 4, 4	1, 2, 3, 4
Evaluations of $f(x)$, once	0, 1, 0, 1	1, 1, 1, 1
Maximum evaluations	64, 65, 128, 129	33, 65, 97, 129

Table 9.10: The constants of Ridders' method and, in the last column, of its one-sided forms (Section 6.7; `BackwardRidders` is `ForwardRidders` mirrored). They differ in the column divisor, since the error of the forward difference has every power of h, not only the even ones, and in the rounding floor, which the one-sided forms carry through the tableau. Where a value depends on the derivative order, it is given for f', f'', f''' and f'''' in turn. The rows for the column divisor, the floor and the restart transcribe the header's documented rules.

Chapter 10

Accuracy, cost, limits and guarantees

This chapter states what to expect of `nxx::deriv` and on what evidence: the accuracy per method, order and type; the cost in evaluations; where the methods stop working; what the library guarantees and under which hypotheses; and how all of it is tested. Each statement is marked as measured, derived or a rule of thumb, and points to its source.

10.1 Accuracy

Table 10.1 gives the measured relative error of seven methods at their default steps (`BackwardRidders` is the mirror of `ForwardRidders`), for the first to the fourth derivative, in three types. The function is e^x, whose derivatives all equal the function, so that the orders compare; each cell is the median over 11 points $x = 0.5, 0.55, \ldots, 1$, which share the scale $s = 1$ and so the step. The median matters: the rounding part of the error at a single point is a matter of luck by about a factor of ten, as `Central<2>` for f' shows with 7.9×10^{-13} at $x = 1$ against a median of 5.8×10^{-12} (measured).

Table 10.1: Measured accuracy: the median relative error of the derivative of e^x over 11 points in $[0.5, 1]$, at the default steps, by method, derivative order N and type. * Every float derivative above the first warns; these were computed with the warning silenced (Section 8.6). 1.4e−5 stands for 1.4×10^{-5}.

	`float`				`double`				`cpp_bin_float_50`			
Method	$N = 1$	2	3	4	$N = 1$	2	3	4	$N = 1$	2	3	4
`Central<2>`	5.0e−6	1.4e−4*	4.0e−4*	0.014*	5.8e−12	8.1e−9	1.4e−7	1.2e−5	6.0e−35	7.8e−26	4.6e−21	2.5e−17
`Central<4>`	4.1e−7	2.0e−5*	6.3e−5*	9.3e−4*	6.7e−14	1.6e−11	4.6e−10	4.9e−8	2.8e−41	4.0e−34	2.7e−29	1.6e−25
`Central<6>`	3.4e−7	9.8e−6*	3.7e−5*	4.3e−4*	8.0e−15	8.6e−13	2.7e−11	2.3e−9	2.6e−44	3.0e−38	2.1e−34	5.2e−30
`Forward<2>`	1.2e−5	7.2e−4*	0.0073*	0.071*	1.8e−11	2.8e−8	2.3e−6	3.5e−5	1.3e−34	1.0e−25	3.4e−20	1.8e−16
`Forward<4>`	4.1e−6	1.9e−4*	0.0043*	0.030*	4.4e−13	1.6e−10	3.6e−8	5.5e−7	7.8e−41	2.2e−33	4.0e−28	9.4e−24
`Ridders`	5.1e−6	2.7e−5*	1.1e−4*	0.0019*	1.3e−14	5.3e−13	2.1e−11	6.8e−10	7.2e−48	2.5e−44	3.5e−41	2.0e−38
`ForwardRidders`	7.6e−6	0.0022*	0.042*	5.7*	1.7e−13	4.1e−11	1.3e−7	4.7e−6	7.5e−46	4.3e−41	5.3e−36	6.3e−32

The error model of Chapter 4 explains the table: a rule of accuracy order P at its default step has an error of about $\varepsilon^{P/(P+N)}$, relative (Table 9.9). Against the double column, the central rules lie below the model by up to a factor of 6.3 for f' to f''' and up to 5.6 times above it for f''''; `Forward<2>` lies up to 5.8 times above it, and `Forward<4>` 32 and 37 times above it for f''' and f'''', by its larger amplification S, which the model drops (measured). In round numbers, `Central<2>` in double gives 11 correct digits for f' and 5 for f''''; Ridders' method 14 and 9. `ForwardRidders`, whose columns gain one order rather than two, loses more with each order: on the random study of Section 10.2 its median error is 14, 212, 1261 and 3007 times Ridders' (measured) for f' to f''''. Higher orders pay most in the wide type, where the step can grow far before truncation matters; the float columns show why every float derivative above the first warns, whatever the method.

Two caveats apply to every figure. First, the model gives an absolute error, $\varepsilon|f|/h^N$ in its rounding part, and the relative figures presuppose

$|f| \sim s^N |f^{(N)}|$: true for e^x on $[0.5, 1]$, and nearly so for the nitrogen c_p, where $|H|/(s|c_p|) = 0.66$ (Chapter 3). Near a zero of $f^{(N)}$ the relative error grows without bound while the absolute error stays (Section 7.5). Second, the default steps assume that f varies on the scale $\max(|x|, 1)$ and is accurate to ε; where it varies faster or is noisier, the figures do not hold until the step is fitted (Figs. 3.2 and 3.4).

10.2 Ridders' error estimate

Ridders' method returns an estimate of its error, not a bound. Its quality was measured on a random study (Fig. 6.3): 9 smooth functions with closed-form derivatives (e^x, $\sin x$, $\log x$, $\sqrt{x}$, $1/x$, $1/(1 + 25x^2)$, x^5, e^{-x^2}, $\tanh x$), each at 300 random points in $[0.2, 3]$, in double, with the default initial steps and no noise. For f' no singularity lies within the initial step; for f'' to f'''' the initial step $s/4$ (and the span $2h_0$ of f''' and f'''') reaches past zero for x below 0.25 (0.5), which are the restarts of Table 10.2. The table gives the results per derivative order. The coverage is the fraction of cases whose estimate is at least the actual error; the ratio is actual error over estimate, so a ratio below 1 is covered.

The estimate is built so that it errs on the safe side: an extrapolated entry's estimate is its change from its parent in the tableau, but never less than the rounding floor of its level (for the one-sided forms, the floor carried through the tableau); the result's `abs_error` is also at least its distance to the latest diagonal entry. The method stops when an entry equals its parent, after two levels that improve nothing, or at the resolution of x; it restarts at half the smallest step it tried after a step at which f is not finite and after an untrusted result, meaning an estimate of at least a quarter of the value and more than 64 times the rounding floor; and it attempts at most 32 levels in all (Table 9.10, Sidebar 6.2). The library's tests accept a result whose actual error is at most six times its estimate, and whose estimate is within twice that of an independent prototype for the same case.

The one-sided `ForwardRidders` was measured on the same cases, with its default initial step $s/32$: its estimate covered 99.96 %, 99.96 %, 99.96 % and 99.9 % of them for f' to f'''', with worst ratios of 1.3, 1.01, 1.6 and 1.7, and median errors of 1.5×10^{-13}, 1.0×10^{-10}, 4.1×10^{-8} and 3.5×10^{-6} (measured; Section 6.7). A pole ahead of x, inside the span the forward rule samples, lies outside that study. On $1/(t - a)$ at 0, with 4000 poles from 2^{-30} to 2^{-3} ahead, f' and f'' were always covered, f''' missed 3 times (at worst 1.02 times the estimate), and f'''' 10 times, 10 of them neither covered nor flagged: 57 to 65 times off, with an error up to 5.0 times the estimate, within the tests' factor of six. The same poles behind x gave 0 uncovered results (measured). Near a known singularity ahead of x, repeat a one-sided f'''' at a second initial step.

Order	Coverage	Uncovered	Ratio		Evaluations	Restarts
	(%)		median	worst	min, median, max	
f'	99.7	9	0.14	1.8	8, 14, 18	0
f''	99.9	4	0.059	1.4	7, 17, 21	11
f'''	99.96	1	0.015	1.001	12, 32, 44	61
f''''	100.0	0	0.0041	0.33	9, 29, 41	61

Table 10.2: Ridders' method on the random study: 2700 cases per derivative order, measured. Restarts are those after a value of f that is not finite, counted from a log of the calls; one after an untrusted result continues the halving of the step and cannot be told apart.

The estimate assumes that f is accurate to about ε. With a relative noise of 10^{-10} and 10^{-8} on e^x, the coverage over 300 points fell to 89.3 % and 90.7 %, with worst ratios of 67 and 42 (measured, Section 3.6). It does not see a jump inside its span either, except as a large estimate (Fig. 7.2), nor a kink at x at all (Section 6.4).

10.3 Cost

The cost of a derivative is counted in evaluations of f, which dominate whenever f is more than a few arithmetic operations; the library's own work is one weighted sum per stencil and one tableau row per level of Ridders' method. Table 10.3 gives the counts. For a stencil they are exact: one evaluation per node. For Ridders' method and `ForwardRidders` they depend on f and x; the table gives the median and the range of the random study and the cap, which no call exceeds. `BackwardRidders` costs what `ForwardRidders` costs on the mirrored function. For f' in double, Ridders' 14 evaluations (median) against a stencil's 2 to 6 buy about three more digits than `Central<2>` (14 against 11), as many as `Central<4>`, none over `Central<6>` (Table 10.1), and an estimate of the error.

10.4 Limits

Orders. The derivative order is 1 to 4. The caps on P are design decisions (`docs/deriv/PHASE2_API.md`). `Central<P>` stops at $P = 10$: by the model's exponent, a `Central<12>` would cut the error of f' by a factor of about 1.7 in double and 5.0 in a 50-digit type (3.6 and 62 for f''''), on a still wider stencil. The one-sided rules stop at $P = 6$, where the rounding amplification S is already 27.7 for f' and 4019 for f'''' (Tables 9.5 and 9.8) and grows about as fast as a higher order pays.

Types. The integer weights must be exact in `T`, which every type with 24 significant bits or more satisfies; a narrower type is rejected at the call by a constraint. A derivative above the first in a type with an epsilon above 10^{-7} keeps few correct digits, by every method, and warns at compile time (Section 8.6).

The scale. The default steps assume that f varies on the scale $\max(|x|, 1)$ in the units of x. That is a rule of thumb, not a property of f; Section 7.8 shows the symptoms when it fails and the cure.

Domain edges. Ridders' method needs the nearest point at which f is not finite a minimum distance away, because the levels it spends restarting come out of its budget of 32. For $\log x$ at $x = 2^{-k}$ with the default first-derivative step it returned a value down to $k = 34$, with an estimate grown

Method	$N = 1$	2	3	4
`Central<2>`	2	3	4	5
`Central<4>`	4	5	6	7
`Central<6>`	6	7	8	9
`Forward<2>`	3	4	5	6
`Forward<4>`	5	6	7	8
`Ridders`, measured (median, range)	14 (8–18)	17 (7–21)	32 (12–44)	29 (9–41)
`Ridders`, at most	64	65	128	129
`ForwardRidders`, measured (median, range)	10 (7–11)	17 (15–21)	22 (13–28)	29 (21–37)
`ForwardRidders`, at most	33	65	97	129

Table 10.3: Evaluations of f per derivative, by method and derivative order N. Stencils: exact. `Ridders` and `ForwardRidders`: measured on the random study (median, with the range), and the cap; `BackwardRidders` as `ForwardRidders`.

to flag the loss, but ten correct digits only down to $k = 30$; it used its whole budget from $k = 27$ (measured, Fig. 6.4). For ten digits, keep the nearest bad point at least $2^{-30}\max(|x|, 1)$ away. The design record derives the first limit, $2^{-34}\max(|x|, 1)$, from the initial step $s/32$ and the cap. The same derivation gives $2^{-31}\max(|x|, 1)$ for f'', whose rule reaches h from the initial step $s/4$, and $2^{-30}\max(|x|, 1)$ for f''' and f'''', whose rules reach $2h$; the record's 2^{-31} for these two is a halving too close. On $\log x$, in half halvings, a value was returned down to $2^{-31.5}$ for f'' and $2^{-30.5}$ and $2^{-30.5}$ for f''' and f'''' (measured).

A pole, where f is finite on both sides, behaves otherwise. The restart after an untrusted result now descends as after a non-finite value, so a pole close to x is resolved: at the library's test poles, 10^{-3} ahead of x and 10^{-4} behind it, far inside the limits above, 8 of 8 results for f' to f'''' were covered, with errors from 1.6×10^{-15} to 3.8×10^{-9} (measured). Closer, a pole does not make the method fail: it returns a wrong value. Over 2328 poles 1.25 to 1.75 times 2^{-k} from x, $k = 4$ to 100, ahead and behind, f' to f'''', none was `non_finite` and every one of the 1586 uncovered results was flagged, its estimate at least a quarter of its value (measured). Near a pole, test `abs_error`.

Exactly at an edge, the one-sided methods apply: `Forward<P>` and `Backward<P>`, or with an error estimate `ForwardRidders` and `BackwardRidders` (Section 6.7). They have a reach of their own: for $\log x$ at $x = 2^{-k}$, the estimate of `ForwardRidders` covered its error up to $k = 36$, 34, 33 and 32 for f' to f'''', and flagged the result, at least a quarter of its value, from $k = 38$, 35, 34 and 33 to the end of the study, $k = 45$; between them, f' at $k = 37$ was 0.22 off with an estimate of 0.19, relative. The band between the two is narrow but not always a single point: on $\log x$, $\sqrt{x}$ and $\sqrt[3]{x}$ at four points per halving, 12 of 528 results for f' were neither covered nor flagged ($\log x$ (1), $\sqrt{x}$ (7), $\sqrt[3]{x}$ (4)), each within 2.2 times its estimate (measured).

Aliasing. A function that oscillates on a scale far below the step can lie at every sample; samples that happen to lie on a polynomial fool every sampling method, Ridders' included (Sidebar 6.3).

Noise and kinks. A function less accurate than ε needs a larger step (Section 3.6); a kink or a jump within a step of x gives a wrong value from every method that samples across it (Section 7.8).

Overflow. A final quotient that overflows, such as a fourth derivative with a tiny step, is `non_finite` at run time and a compile error under GCC in a constant expression (Sidebar 7.2).

10.5 Guarantees

Each guarantee below is stated with the hypothesis it rests on and with how it is verified.

- *No exceptions.* The library throws nothing and catches nothing; failures are `std::expected` values. An exception thrown by `fn` or by the operations of `T` passes through unchanged. Verified by inspection: the headers contain no `throw` and no `try`.
- *Where `fn` is called.* Only at finite points: the overflow checks come before any point is formed. A stencil calls `fn` once per node, `Forward<P>`

never below x and `Backward<P>` never above; a step that is not positive and finite returns `non_finite` before any call. Ridders' method calls it at most 64, 65, 128 and 129 times for f' to f''''; `ForwardRidders` never below x and `BackwardRidders` never above, restarts included, at most 33, 65, 97 and 129 times. Verified by tests that count and log the calls.

- *No shared state.* The headers hold no global, static or thread-local variable other than compile-time constants, and `fn` is called on the calling thread, one call at a time. Concurrent calls of `derivative` are therefore as safe as the concurrent calls of `fn` they make. Verified by inspection.
- *No allocation.* Only fixed-size arrays and `std::optional` on the stack; a `T` that allocates does so in its own operations. Verified by inspection.
- *Failures are values, also at compile time.* Every failure is detected before the arithmetic that would produce it, so it is an error value in a constant expression too, with one exception: an overflow of the final quotient, which GCC rejects at compile time. Verified by `static_asserts` on overflowing, non-finite and saturating cases.
- *Exact weights.* The integer weights are exact in every type with 24 significant bits or more; the largest is 6,222,216, below 2^{24}. A larger value would make the generator's constant evaluation fail, and a narrower type is rejected by a constraint of `derivative`. Verified at every compilation.
- *Exact on polynomials.* With dyadic x and a power-of-two h, a stencil is exact for polynomials of degree up to $N + P - 1$, provided every intermediate is exact in the type. Verified by `static_asserts` that also check the bit budget of each case.
- *Compile time equals run time.* The same call gives the same bits at compile time and at run time, provided the compiler neither contracts `a * b + c` into a fused multiply-add nor uses extended precision. The build pins neither. Verified on the five toolchains of the continuous integration (GCC, Clang, MSVC, clang-cl, Emscripten), by tests that compare constant and run-time results, and by `code/07-using-constexpr.cpp`.
- *`root_epsilon`* is within an ulp of $\varepsilon^{1/N}$. Verified against references computed to 80 digits for selected N from 2 to 2000 in `float` and `double`, and for the multiprecision types against 60-digit references.
- *Types.* `derivative` is tested on `float`, `double`, `cpp_bin_float_50` with and without expression templates, and `cpp_dec_float_50`, the `float` derivatives above the first with the warning silenced at the call; `long double` satisfies the concept and is tested in `root_epsilon` only.

Not guaranteed: the accuracy, which is a model and a measurement (Section 10.1); Ridders' error estimate, which is an estimate (Section 10.2); and the default step, which is a rule of thumb.

10.6 How it is verified

The library's tests are one doctest executable and four build tests, listed in Table 10.4. Their reference values for the derivatives come from closed forms, from the published tables of Fornberg[15] and Chapra (Chapter 23)[16] or from an independent prototype, never from Numerixx itself; the ceilings on `abs_error` for the multiprecision types are twice the estimates of the first run, regression baselines that `docs/deriv/design.md` records as such, not independent references. Each tolerance is justified by an error estimate in a comment. The stencils, for example, are held to $(C + 2S)\,\varepsilon^{P/(P+N)}$ relative on e^x, truncation plus the rounding of the samples and of the points, for a selection of rules and orders, two-node rules included; the tests' comment calls it an estimate, not a bound, and reports measured errors 4 to 16 times below it on its three points. They run on the five toolchains of the continuous integration with warnings as errors. Each phase was reviewed by specialised review agents, Phases 4 and 5 also by another vendor's model (OpenAI Codex); the findings and decisions are recorded in the design record (`docs/deriv/PHASE3_API.md` to `docs/deriv/PHASE5_API.md`, `docs/deriv/design.md`). The book's own examples build and run as tests, and its data generator checks every table against its moment conditions and the nitrogen function against an independent exact evaluation each time it runs.

File	What it checks
`test_math.cpp`	the concept `real` on accepted and rejected types; `abs`, `isfinite`; `root_epsilon` against 80-digit roots, the same at compile time and run time
`test_deriv.cpp`	`Central<2>` with the default step on float, double and multiprecision types; two calls of `fn`; non-finite results as errors
`test_stencils.cpp`	stencils of each family with their default step against $(C + 2S)\,\varepsilon^{P/(P+1)}$; one-sided rules never cross x; one call per node; failures; multiprecision
`test_callables.cpp`	`fn` called through `std::invoke` with a `T const` lvalue: member pointers, callables that reject rvalues
`test_ridders.cpp`	exact results on polynomials with the floor as the estimate; accuracy and coverage against closed forms and the prototype; restarts; failures, untrusted results and the level cap; multiprecision
`test_orders.cpp`	orders 1 to 4: the generated tables against the moment conditions and published rows; exactness on polynomials; steps outside the range of h^N; overflow; Ridders' evaluations and accuracy, poles close to x
`test_onesided.cpp`	`ForwardRidders` and `BackwardRidders`: exact on polynomials; accuracy and coverage against closed forms and the prototype; domain edges, a pole behind x, a kink at x; `fn` called on one side only, restarts included; the mirror; failures and cost; multiprecision
`precision_warning.cpp`	four builds: a float f'' warns through each overload, and a pragma or the macro silences it

Table 10.4: The library's tests, in `tests/`.

Appendix A

Floating-point essentials

This appendix collects the facts about floating-point arithmetic that Chapter 3 and the sidebars use. The standard references are Higham's book and Goldberg's survey;[8,9] the formats and the rounding rule are fixed by IEEE 754.[22]

A.1 Representation

A binary floating-point number is

$$x = \pm m \cdot 2^{e}, \qquad 1 \le m < 2,$$

where the *significand* m has p binary digits and the *exponent* e is an integer in a fixed range. For `float` $p = 24$, for `double` $p = 53$ (Table A.1). Every integer up to 2^{p} is exact, which is why the library's integer weights, all below 2^{24}, are exact in every type with at least 24 significant bits. Between two powers of two the numbers are evenly spaced; at each power of two the spacing doubles.

A.2 Machine epsilon, the unit roundoff and the ulp

Machine epsilon $\varepsilon = 2^{1-p}$ is the distance from 1 to the next larger number: 2^{-52} for `double`. The spacing at a general x is its *unit in the last place*, $\operatorname{ulp}(x) = \varepsilon \cdot 2^{e}$ for $2^{e} \le |x| < 2^{e+1}$. It is proportional to $|x|$ within a factor of two: $\operatorname{ulp}(1) = 2.2 \times 10^{-16}$, just below 1 the spacing is 1.1×10^{-16}, and $\operatorname{ulp}(1000) = 1.1 \times 10^{-13}$.

IEEE 754 arithmetic rounds the exact result of each operation $+$, $-$, $\times$, $/$ and $\sqrt{\ }$ to the nearest representable number (ties to an even last digit). The error is at most half a spacing, so

$$\operatorname{fl}(a \circ b) = (a \circ b)(1 + \delta), \qquad |\delta| \le \varepsilon/2,$$

as long as the result is neither too large nor too small for the format. The bound $\varepsilon/2$ is the *unit roundoff*, which many texts call u. This book speaks of ε only, and the library's rounding model uses it: a value of f that is the result of many operations is rarely accurate to half an ulp, and $\varepsilon|f|$ is a more honest estimate of its error.

	`float`	`double`
significant bits p (`digits`)	24	53
decimal digits (`digits10`)	6	15
machine epsilon ε	1.2×10^{-7}	2.2×10^{-16}
largest finite Ω (`max`)	3.4×10^{38}	1.8×10^{308}
smallest normal (`min`)	1.2×10^{-38}	2.2×10^{-308}
smallest subnormal (`denorm_min`)	1.4×10^{-45}	4.9×10^{-324}

Table A.1: The constants of `float` and `double`, from `std::numeric_limits` on the build machine (IEEE 754 formats). $\varepsilon = 2^{1-p}$; decimal digits is `digits10`, the number of decimal digits that survive a round trip; below the smallest normal number the subnormals continue to the smallest subnormal.

A.3 The ends of the range

The largest finite number is Ω. A result beyond it rounds to infinity, and infinity minus infinity is NaN, not a number. The library checks what could overflow (before it is formed; the final quotient after) and reports `error::non_finite` instead; it never calls `fn` at a point that is not finite.

At the other end, below the smallest normal number (2.2×10^{-308} for `double`) the exponent can fall no further, and the numbers continue with a fixed spacing equal to the smallest subnormal, 4.9×10^{-324}. These *subnormal* numbers fill the gap to zero gradually, but their relative accuracy falls as they shrink: rounding there is absolute, not relative. The library's rounding model accounts for that: the rounding error of a nonzero value of f is $\varepsilon|f|$, but never less than the smallest subnormal.

A.4 Why $x + h$ is not $x + h$

A finite difference evaluates f at $x + h$, and that sum is rounded like any other. The function therefore sees $\mathrm{fl}(x + h)$, which differs from $x + h$ by up to half the spacing at the result: $\mathrm{ulp}(x)/2$, or $\mathrm{ulp}(x)$ when $x + h$ crosses a power of two (the two-sided spacing of Sidebar 3.3). Two consequences follow, both met in Chapter 3. A step below $\mathrm{ulp}(x)/2$ does not move x. If neither $x + h$ nor $x - h$ moves, the library reports `error::non_finite`. If only one moves, as can happen near a power of two, a two-node rule for f' returns a finite value that is mostly rounding, and every other rule reports `non_finite` (Sidebar 3.3 and Section 8.2). Hence the requirement: a step well above $\mathrm{ulp}(x)$. And for any step the distance between the two points of the central difference is not exactly $2h$, which is why the two-node rules divide by the distance actually evaluated (Sidebar 3.1).

A.5 Cancellation

The subtraction of two nearby numbers of the same sign is exact: if $a, b > 0$ and $a/2 \le b \le 2a$, then $\mathrm{fl}(a - b) = a - b$ (Sterbenz's lemma, Theorem 4.3.1; likewise for $a, b < 0$).[10] The danger is not in the subtraction but in its inputs. If a and b carry errors δ_a and δ_b from earlier rounding, the difference carries $\delta_a - \delta_b$, and its relative error is

$$\frac{|\delta_a - \delta_b|}{|a - b|} \le \frac{|\delta_a| + |\delta_b|}{|a - b|},$$

which is large when a and b are close: an error that was small compared with a and b is large compared with their difference. This is *cancellation*, and it is the whole reason for the rounding branch of the error model: the values $f(x + h)$ and $f(x - h)$ come closer as h shrinks, their rounding errors do not, and the division by $2h$ magnifies what is left (Section 3.2).

A.6 Other types

Nothing above is special to binary: decimal types round to decimal digits, with their own ε, and multiprecision types such as `cpp_bin_float_50` of Boost.Multiprecision ($p = 168$ bits, $\varepsilon = 5.3 \times 10^{-51}$) follow the same rules with more digits. What the library needs from a type, its arithmetic, its comparisons and a few members of `std::numeric_limits`, is listed in Section 8.7; its error model needs only ε.

Appendix B

Derivations

This appendix proves what the sidebars of Chapters 4 to 6 only cite: why the moment conditions decide a rule's order and its error constant, why the Lagrange formula produces the weights, why Richardson's tableau works and what its error estimate measures, and where the best step and its constant come from. Throughout, f has as many continuous derivatives as the expansions use, and one more bounded derivative where the error model's $O(h^{P+1})$ is claimed (Section B.1); the nodes n_k are distinct integers, and the weights w_k are the rational numbers of Eq. (4.1); with the library's integer numerators $c_k = d w_k$ every moment is multiplied by d.

B.1 Taylor expansions and the moment conditions

Taylor's theorem with the Lagrange remainder (Burden et al.,[7] Section 1.1) gives, for each node, with $x_k = x + n_k h$,

$$f(x_k) = \sum_{j=0}^{m-1} \frac{f^{(j)}(x)}{j!}\, n_k^j h^j + \frac{f^{(m)}(\xi_k)}{m!}\, n_k^m h^m, \qquad \xi_k \text{ between } x \text{ and } x_k .$$

Multiply by w_k and sum over the nodes. The powers of h collect into the moments $M_j = \sum_k w_k n_k^j$ of Eq. (4.2):

$$\sum_k w_k f(x_k) = \sum_{j=0}^{m-1} \frac{M_j}{j!}\, h^j f^{(j)}(x) + \frac{h^m}{m!} \sum_k w_k n_k^m f^{(m)}(\xi_k). \tag{B.1}$$

Suppose the moment conditions hold up to $m-1 = N+P-1$: $M_N = N!$ and $M_j = 0$ for every other j below m. Then the sum on the left is $h^N f^{(N)}(x)$ plus the remainder, and dividing by h^N gives

$$\frac{1}{h^N} \sum_k w_k f(x_k) = f^{(N)}(x) + \frac{h^P}{(N+P)!} \sum_k w_k n_k^{N+P} f^{(N+P)}(\xi_k).$$

The remainder is $O(h^P)$, which is the accuracy order, and since each ξ_k tends to x with h and $f^{(N+P)}$ is continuous, the remainder is $M_{N+P}\, h^P f^{(N+P)}(x)/(N+P)! + o(h^P)$ for every rule, and the correction is $O(h^{P+1})$ when f has one more bounded derivative, which the error model assumes throughout. This asymptotic form is what the error model uses, with the error constant

$$C = \frac{|M_{N+P}|}{(N+P)!}, \qquad \text{or } \frac{|M_{N+P}|}{(N+P)!\, d} \text{ with the integer numerators,} \tag{B.2}$$

as Chapter 9 tabulates it.

The sharper statement with a single intermediate point, as in Eq. (4.3) and Eq. (5.1), needs one more argument, and which argument depends on the rule. If the products $w_k n_k^{N+P}$ all have one sign, the intermediate-value theorem collapses the sum above to one point: the remainder is

$M_{N+P}\, h^P f^{(N+P)}(\xi)/(N+P)!$ for some ξ in the span of the stencil. That is the case for Eq. (5.1), whose products are $1, 0, 1$, but not for Eq. (4.3), whose products $-\frac{8}{3}, \frac{2}{3}, \frac{2}{3}, -\frac{8}{3}$ have mixed signs, nor for any central rule for f' above order 2, nor for some one-sided rules. For the first derivative the single-point form follows instead from the interpolation error formula, as Burden et al.[7] (Section 4.1) derive the five-point formulas: with x itself among the nodes (added with the weight 0 where the rule omits it, which changes nothing), $f(x+th) - p(t)$ equals $f^{(m)}(\xi(t)) \prod_j (t - n_j)\, h^m/m!$, and differentiating once at $t = 0$, where the node product vanishes, leaves $f'(x) - p'(0)/h = f^{(m)}(\xi)\, h^{m-1} \prod_{n_j \neq 0}(-n_j)/m!$ with one ξ. For `Central<4>` the product over the nodes $-2, -1, 1, 2$ is 4 and $m = 5$, which gives the $+h^4 f^{(5)}(\xi)/30$ of Eq. (4.3); the one-sided rules, which contain x as a node, are covered by the same formula. For the higher derivatives and mixed signs the book uses only the asymptotic form.

For a symmetric stencil, nodes in pairs $\pm n$ with $w_{-n} = (-1)^N w_n$, every moment M_j with $j - N$ odd vanishes identically, so the expansion of a central rule contains only the powers $h^P, h^{P+2}, \ldots$: an even expansion in h. This is what Sidebar 2.1 states for the central difference and what the extrapolation of Section B.3 relies on for every order. It also explains why a central rule needs one node fewer than a one-sided rule of the same order: of the $N + P$ conditions, those with $j - N$ odd hold by symmetry and cost no weight.

Rounding enters Eq. (B.1) through the samples. If each $f(x_k)$ is off by at most $\varepsilon|f(x_k)|$, the weighted sum is off by at most $\sum_k |w_k|\, \varepsilon|f(x_k)|$, which for samples of about the size of $|f(x)|$ is $S\varepsilon|f|$ with $S = \sum_k |w_k|$; the division by h^N turns it into the rounding term $S\varepsilon|f|/h^N$ of Eq. (5.2). Rounding of the points, x_k off by up to $\varepsilon|x_k|/2$, moves each sample by about $\varepsilon|xf'|/2$ and adds the third term named in Section 4.3.

B.2 The Lagrange formula for the weights

Let p be the polynomial of degree below the number of nodes m that interpolates the samples, written in the scaled variable t with $x + th$: $p(t) = \sum_k f(x_k) L_k(t)$, where the Lagrange basis polynomial $L_k(t) = \prod_{j \neq k}(t - n_j)/(n_k - n_j)$ is 1 at $t = n_k$ and 0 at every other node. Differentiating p N times at $t = 0$ and dividing by h^N gives an estimate of $f^{(N)}(x)$ of the form Eq. (4.1) with

$$w_k = L_k^{(N)}(0) = N!\, [t^N]\, L_k(t), \tag{B.3}$$

the coefficient of t^N in L_k times $N!$, which is what the library's generator computes with exact integer arithmetic (Burden et al.,[7] Section 4.1). These weights satisfy the moment conditions, for the following reason. Interpolation on m nodes reproduces every polynomial of degree below m: if $f(x + th) = t^j$ with $j < m$, then $p(t) = t^j$ exactly, since both sides are polynomials of degree below m that agree at m points. For that f the samples are $f(x_k) = n_k^j$, so $\sum_k w_k n_k^j = p^{(N)}(0) = (t^j)^{(N)}(0)$, which is $N!$ for $j = N$ and 0 for every other $j < m$. These are exactly the moment conditions up to M_{m-1}, so the Lagrange weights are the unique solution of that linear system, and the rule's order is at least $m - N$. For the one-sided rules $m = P + N$ and the order is exactly P, since M_m does not vanish; for the central rules $m = P + N - 1$ and the symmetry argument of Section B.1 supplies the missing condition $M_m = 0$.

The library's own worked example is `Central<4>` for f' on the nodes $-2, -1, 1, 2$. For the node $n_k = -1$, the numerator of L_k is $(t+2)(t-1)(t-2) = t^3 - t^2 - 4t + 4$ and its denominator $(-1+2)(-1-1)(-1-2) = 6$, so $w_{-1} = 1! \cdot (-4)/6 = -2/3$; the other three nodes give $1/12$, $2/3$ and $-1/12$. Over the common denominator 12 the numerators are $1, -8, 8, -1$, the table the tests check against Fornberg's.[15] The same computation for Chapra's backward rule for the fourth derivative on the nodes $-5, \ldots, 0$ gives $-2, 11, -24, 26, -14, 3$ over 1, the row the tests compare with the published one (Chapra, Chapter 23).[16]

B.3 Richardson extrapolation

Let $D(h)$ be a base rule with an even expansion, $D(h) = f^{(N)}(x) + a_2 h^2 + a_4 h^4 + \cdots$; by Section B.1 every `Central<2>` rule has one. With $h_i = h_0/2^i$ and $D_i = D(h_i)$, define $A_{i,0} = D_i$ and

$$A_{i,j} = \frac{4^j A_{i,j-1} - A_{i-1,j-1}}{4^j - 1} = A_{i,j-1} + \frac{A_{i,j-1} - A_{i-1,j-1}}{4^j - 1}. \tag{B.4}$$

The claim is that column j has an expansion starting at h^{2j+2} (Burden et al., [7] Section 4.2). By induction on j: suppose $A_{i,j-1} = f^{(N)} + c\, h_i^{2j} + O(h_i^{2j+2})$ with a coefficient c that does not depend on i, which holds for $j = 1$ with $c = a_2$. The parent one level up has the step $2h_i$, so $A_{i-1,j-1} = f^{(N)} + c\, 4^j h_i^{2j} + O(h_i^{2j+2})$. In the combination $4^j A_{i,j-1} - A_{i-1,j-1}$ the h^{2j} terms cancel exactly and the constant term is $(4^j - 1) f^{(N)}$; dividing by $4^j - 1$ leaves $f^{(N)} + O(h_i^{2j+2})$, again with a leading coefficient independent of i. The two forms in Eq. (B.4) are equal by a line of algebra, and the second is the one the library computes (Sidebar 6.1).

The a posteriori estimate follows from the same expansion. With $A_{i,j-1} - f^{(N)} = c\, h_i^{2j} + \cdots$ and $A_{i-1,j-1} - f^{(N)} = 4^j c\, h_i^{2j} + \cdots$,

$$A_{i,j} - A_{i-1,j-1} = \left(A_{i,j} - f^{(N)}\right) - \left(A_{i-1,j-1} - f^{(N)}\right) = -4^j c\, h_i^{2j} + O(h_i^{2j+2}),$$

so the change of an entry from its parent equals the parent's error to leading order, and the parent's error is of a lower order in the step than the entry's own, h_i^{2j} against h_i^{2j+2}. While truncation dominates, the change is therefore a conservative estimate of the entry's error. Once rounding dominates it is not: the entries of a converged column are the limit plus independent rounding of about the size of the rounding floor, and the smallest of several such differences underestimates the error. This is why Eq. (6.3) takes the larger of the change and the floor, and why Ridders' method stops once two levels bring no smaller estimate.

The one-sided forms of Section 6.7 extrapolate the `Forward<1>` rule of order N, whose nodes $0, \ldots, N$ have no symmetry: its expansion $f^{(N)} + b_1 h + b_2 h^2 + \cdots$ has every power of h. The same induction with $A_{i,j-1} = f^{(N)} + c\, h_i^j + O(h_i^{j+1})$ and the parent $f^{(N)} + c\, 2^j h_i^j + \cdots$ shows that $(2^j A_{i,j-1} - A_{i-1,j-1})/(2^j - 1)$ cancels the term in h^j and leaves $O(h_i^{j+1})$: the divisor is $2^j - 1$, and each column gains one order (Burden et al., [7] Section 4.2). Unrolled, an entry is a combination $\sum_k \gamma_k D_k$ of the base values, so it inherits at most $\sum_k |\gamma_k| r_k$ of rounding, to first order; as the γ_k alternate in sign, the recursion $r_{i,j} = (2^j r_{i,j-1} + r_{i-1,j-1})/(2^j - 1)$ computes that sum exactly.

B.4 The error model and the best step

The error model of Eq. (5.2) is the sum of the leading truncation term and the rounding term of Section B.1, with $F = |f^{(P+N)}(x)|$ written for short:

$$E(h) = C\,h^{P}F + S\,\varepsilon\frac{|f|}{h^{N}}.$$

It is convex in $h > 0$, and its derivative vanishes where $PC\,h^{P-1}F = NS\varepsilon|f|/h^{N+1}$, that is at

$$h^{*} = \left(\frac{NS\,\varepsilon|f|}{PC\,F}\right)^{1/(P+N)}. \tag{B.5}$$

At the minimum the two terms stand in the ratio $N : P$ (truncation to rounding), so $E(h^{*}) = (1 + N/P)\,S\varepsilon|f|/h^{*N}$, which is of order $\varepsilon^{1-N/(P+N)} = \varepsilon^{P/(P+N)}$ times a factor built from $|f|$, F and the constants. The default step assumes that f varies on the scale s, meaning $|f^{(k)}| \approx |f|/s^{k}$, so that $|f|/F \approx s^{P+N}$; then $h^{*} \approx \kappa\,\varepsilon^{1/(P+N)}s$ with the constant

$$\kappa = \left(\frac{NS}{PC}\right)^{1/(P+N)}, \tag{B.6}$$

which the library drops: its step is $\varepsilon^{1/(P+N)}s$ (Eq. (5.3)).

The cost of dropping κ is bounded by the model. At a step $h = h^{*}/\kappa$ the truncation term is κ^{-P} times its value at h^{*} and the rounding term κ^{N} times its value, and since those values are the fractions $N/(P+N)$ and $P/(P+N)$ of $E(h^{*})$,

$$\frac{E(h^{*}/\kappa)}{E(h^{*})} = \frac{N\,\kappa^{-P} + P\,\kappa^{N}}{P+N}. \tag{B.7}$$

For the first derivative κ lies between 1.4 (`Central<2>`, where it is $3^{1/3}$) and 2 (`Forward<1>`, $(2\cdot 2)^{1/2}$), and Eq. (B.7) gives at most 1.63, reached by `Central<10>` with $\kappa = 1.8$; for `Central<2>` and f' it is 1.12. For the higher orders κ is 2.6 for f'' and 2.4 for f'''' with `Central<2>`, and the cost by Eq. (B.7) is 3.54 and 11.2; for `Central<4>` and f'''' it is 15.1, the largest of the four rules the design record quotes. These agree with the design record's figures of 3.5, 11 and 15 (`docs/deriv/PHASE4_API.md`); the library's documentation gives the bound for f' as about 1.6, and 1.63 for `Central<10>`. They are model figures for a function that varies on the scale s exactly. The design record's prototype (`docs/deriv/PHASE4_API.md`) reports the opposite experience on nine test functions that vary faster than s: half the default step was better for every order, by a factor 1.2 to 1.8 for `Central<2>`, which is why the library keeps the plain convention of Eq. (5.3) for every order and states its sensitivity instead (Sidebar 3.2, Sidebar 5.1).

Notation and glossary

Notation

Mathematics is set in the sans-serif face; variables are italic and operators upright. Code names such as `Central<4>` and `abs_error` appear in monospace and never inside mathematics. Primes are used up to f'''; the fourth derivative is $f^{(4)}$ or, where the library's own wording is quoted, f''''.

Symbol	Meaning
x, f, $f(x)$	the point and the function to differentiate; `x` and `fn` in code
h	the step; for Ridders' method the initial step h_0, with $h_i = h_0/2^i$ at level i
$\mathrm{d}f/\mathrm{d}x$, $f^{(N)}$	the first and the N-th derivative
N	the derivative order, 1 to 4 (the template argument `N`)
P	the accuracy order of a rule, error $O(h^P)$ (the template argument `P`)
$O(h^P)$	of the order of h^P as $h \to 0$
n_k, w_k	the nodes of a rule (integer offsets: the samples are at $x + n_k h$) and its rational weights, $f^{(N)} \approx \sum_k w_k f(x + n_k h)/h^N$
c_k, d	the integer numerators and their common denominator, $w_k = c_k/d$, as the library stores them and Chapter 9 tabulates them
x_k, f_k	$x + n_k h$ and $f(x_k)$, the k-th point and sample
K	the largest $\lvert n_k \rvert$ of a rule; the rule reaches at most Kh from x
M_j	the j-th moment of a rule, $\sum_k w_k n_k^j$ (with c_k: $\sum_k c_k n_k^j$)
C	the error constant, the magnitude of the leading truncation term $\pm C\, h^P f^{(P+N)}$, whose sign is that of M_{N+P} (Chapter 9 gives the term signed)
S	the rounding amplification, $\sum_k \lvert w_k \rvert$
ε	machine epsilon of the type, `std::numeric_limits<T>::epsilon()`: 2^{-52} for `double`; the book uses ε, not the unit roundoff $\varepsilon/2$
$\mathrm{ulp}(x)$	unit in the last place: the spacing of the floating-point numbers at x
$\mathrm{fl}(\cdot)$	the rounded result of an operation
Ω	the largest finite value of the type, `std::numeric_limits<T>::max()`
s	$\max(\lvert x \rvert, 1)$, the scale of the default steps
w	the scale on which f varies; s is the library's guess for it
η	the noise level: the relative accuracy of f when it is worse than ε
$E(h)$, $E_\mathrm{t}(h)$, $E_\mathrm{r}(h)$	the error model and its truncation and rounding parts
h^*	the best step of the error model
$D(h)$, D_i	the base difference (the `Central<2>` rule of order N) at step h; $D_i = D(h_i)$

Symbol	Meaning
$A_{i,j}$	Ridders' tableau entry at level i, column j; $A_{i,0} = D_i$
r_i	the rounding floor of level i of the tableau
$e_{i,j}$, $\hat{e}$	the estimate of a tableau entry, and the returned error estimate `abs_error`
n_f	the number of evaluations of `fn`
ξ	the intermediate point of Taylor's remainder
$H(T)$, c_p, T_0	the nitrogen example: the molar enthalpy of nitrogen in J/mol relative to 298.15 K, its heat capacity $\mathrm{d}H/\mathrm{d}T$ in J/(mol K), and the point 1000 K
R	the gas constant

Glossary

accuracy order P The power of h in a rule's leading truncation error, $O(h^P)$; the number in `Central<P>`, `Forward<P>` and `Backward<P>`. Not the number of points.

aggregate A C++ class with public members and no user-written constructor, which can be initialised member by member in braces, `Estimate{value, abs_error}`, and by *designated initialisers*, `Estimate{.value = 1.0}` (Section 8.3).

aliasing Samples that lie: a function that oscillates faster than the step can look constant, or like a polynomial, at every sample (Sidebar 6.3).

amplification S The sum of the absolute weights of a rule, $\sum_k |w_k|$, by which the rounding of the samples reaches the result (Section 4.2).

cancellation The loss of leading digits when two nearly equal numbers are subtracted: their difference keeps only the digits in which they differ, rounding errors included (Section 3.2).

coverage The fraction of cases in a study whose error estimate is at least the actual error.

derivative order N Which derivative is computed, 1 to 4; the template argument `N` or a value of `order`.

error estimate The `abs_error` of a result, filled by Ridders' method and empty for the stencils: an estimate of the absolute error, not a bound.

estimate A returned value of a derivative; the `value` of an `Estimate<T>`.

evaluation One call of `fn`; the unit of cost.

kink A point where f or one of its derivatives jumps: a range break between two fits, a phase boundary, an `if`. A rule whose span contains one returns a wrong value without failing.

level, column, entry, tableau Ridders' method halves the step once per level; the extrapolated values form a triangular tableau whose column j is free of the error terms up to h^{2j}; an entry is one of its values.

lvalue In C++, an expression that names an object, such as a variable; `fn` is called with a `T const` lvalue (Section 8.2).

machine epsilon ε The distance from 1 to the next number of the type.

moment conditions The equations that fix a rule's weights: $\sum_k w_k n_k^j$ is $N!$ for $j = N$ and 0 for every other j below $N + P$ (Section 4.1).

node An integer offset n_k of a rule; the sample is taken at $x + n_k h$.

noise level η The relative accuracy of f when it is worse than ε (a solver's tolerance, for instance); it replaces ε in the rounding branch of the error model.

non-finite An infinite or NaN value; the one failure of `derivative`, `error::non_finite`.

requires-expression A C++ expression `requires (X x) { e; }`, true when the statement `e` would compile for an `x` of type `X`; the book's test of what the library rejects (Section 8.8).

restart Ridders' method discarding its tableau and starting again from a smaller step, after a non-finite value or an untrusted result.

Richardson extrapolation Combining two estimates of known error orders so that the leading error term cancels: $(4D(h/2) - D(h))/3$ for the central difference, repeated in Ridders' tableau (Section 6.1).

rounding error The part of the error that comes from the rounding of the values of f (and of the points); it grows as the step shrinks.

rounding floor The rounding error of a base difference of Ridders' tableau, below which no entry of that level can claim an estimate (Section 6.3).

rule A formula for a derivative: a stencil with its weights, such as the central difference.

sample A value $f_k = f(x + n_k h)$ at a node.

scale s $\max(|x|, 1)$, the length on which the default steps assume f to vary.

stencil The pattern of nodes of a rule, and the family of rules that share it (`Central<P>`); a stencil method, as against Ridders', samples a fixed pattern of points.

step h The distance between x and the points at which f is sampled, in the units of x; the default step is the library's choice when none is passed.

structured binding The C++ form `auto const [value, abs_error] = *r` that names the members of an aggregate at once (Section 8.3).

truncation error The error of a rule in exact arithmetic, from truncating the Taylor series; it shrinks with the step.

unit roundoff Half the machine epsilon, $\varepsilon/2$, the largest relative error of one rounding; many texts call it u, this book uses ε (Section A.2).

untrusted result A Ridders result whose estimate is at least a quarter of its value and far above the rounding floor; the method restarts below it and returns it only if the budget runs out (Sidebar 6.2).

References

[1] Griewank, A.; Walther, A. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed.; Society for Industrial and Applied Mathematics: Philadelphia, 2008; DOI: 10.1137/1.9780898717761.

[2] Squire, W.; Trapp, G. Using complex variables to estimate derivatives of real functions. *SIAM Review* **1998**, *40*, 110–112, DOI: 10.1137/S003614459631241X.

[3] Martins, J. R. R. A.; Sturdza, P.; Alonso, J. J. The complex-step derivative approximation. *ACM Transactions on Mathematical Software* **2003**, *29*, 245–262, DOI: 10.1145/838250.838251.

[4] Lyness, J. N.; Moler, C. B. Numerical differentiation of analytic functions. *SIAM Journal on Numerical Analysis* **1967**, *4*, 202–210, DOI: 10.1137/0704019.

[5] Linstrom, P. J., Mallard, W. G., Eds. *NIST Chemistry WebBook*; NIST Standard Reference Database Number 69; National Institute of Standards and Technology: Gaithersburg, MD, DOI: 10.18434/T4D303, Nitrogen, gas phase thermochemistry data, Shomate equation (new parameter fit January 2009, data from Chase 1998), https://webbook.nist.gov/cgi/cbook.cgi?ID=C7727379&Mask=1, retrieved 23 October 2026.

[6] Chase, M. W., Jr. *NIST-JANAF Thermochemical Tables*, 4th ed.; Journal of Physical and Chemical Reference Data, Monograph 9; American Chemical Society and American Institute of Physics for the National Institute of Standards and Technology: Woodbury, NY, 1998; pp 1–1951, Two parts: Part I, Al–Co; Part II, Cr–Zr. Nitrogen (N_2): Part II, p. 1621.

[7] Burden, R. L.; Faires, J. D.; Burden, A. M. *Numerical Analysis*, 10th ed.; Cengage Learning: Boston, MA, 2016.

[8] Higham, N. J. *Accuracy and Stability of Numerical Algorithms*, 2nd ed.; Society for Industrial and Applied Mathematics: Philadelphia, 2002; DOI: 10.1137/1.9780898718027.

[9] Goldberg, D. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys* **1991**, *23*, 5–48, DOI: 10.1145/103162.103163.

[10] Sterbenz, P. H. *Floating-Point Computation*; Prentice-Hall Series in Automatic Computation; Prentice-Hall: Englewood Cliffs, NJ, 1974.

[11] Nocedal, J.; Wright, S. J. *Numerical Optimization*, 2nd ed.; Springer Series in Operations Research and Financial Engineering; Springer: New York, 2006; DOI: 10.1007/978-0-387-40065-5.

[12] Dennis, J. E., Jr.; Schnabel, R. B. *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; Classics in Applied Mathematics 16; Society for Industrial and Applied Mathematics: Philadelphia, 1996; DOI: 10.1137/1.9781611971200.

[13] Moré, J. J.; Wild, S. M. Estimating derivatives of noisy simulations. *ACM Transactions on Mathematical Software* **2012**, *38*, 19:1–19:21, DOI: 10.1145/2168773.2168777.

[14] Moré, J. J.; Wild, S. M. Estimating computational noise. *SIAM Journal on Scientific Computing* **2011**, *33*, 1292–1314, DOI: 10.1137/100786125.

[15] Fornberg, B. Generation of finite difference formulas on arbitrarily spaced grids. *Mathematics of Computation* **1988**, *51*, 699–706, DOI: 10.1090/S0025-5718-1988-0935077-0.

[16] Chapra, S. C.; Canale, R. P. *Numerical Methods for Engineers*, 7th ed.; McGraw-Hill Education: New York, 2015.

[17] Ridders, C. J. F. Accurate computation of F'(x) and F'(x)F"(x). *Advances in Engineering Software* **1982**, *4*, 75–76, DOI: 10.1016/S0141-1195(82)80057-0.

[18] Richardson, L. F. The approximate arithmetical solution by finite differences of physical problems involving differential equations, with an application to the stresses in a masonry dam. *Philosophical Transactions of the Royal Society of London. Series A* **1911**, *210*, 307–357, DOI: 10.1098/rsta.1911.0009.

[19] Richardson, L. F.; Gaunt, J. A. The deferred approach to the limit. *Philosophical Transactions of the Royal Society of London. Series A* **1927**, *226*, 299–361, DOI: 10.1098/rsta.1927.0008.

[20] Sidi, A. *Practical Extrapolation Methods: Theory and Applications*; Cambridge Monographs on Applied and Computational Mathematics 10; Cambridge University Press: Cambridge, 2003; DOI: 10.1017/CBO9780511546815.

[21] Colebrook, C. F. Turbulent flow in pipes, with particular reference to the transition region between the smooth and rough pipe laws. *Journal of the Institution of Civil Engineers* **1939**, *11*, 133–156, DOI: 10.1680/ijoti.1939.13150.

[22] *IEEE Standard for Floating-Point Arithmetic*; IEEE Std 754-2019 (Revision of IEEE 754-2008); Institute of Electrical and Electronics Engineers: Piscataway, NJ, 2019; DOI: 10.1109/IEEESTD.2019.8766229.

Quick reference

Include. `#include <numerixx/deriv.hpp>`, namespace `nxx::deriv`; header-only, C++23; CMake target `numerixx::numerixx` (Section 8.1).

The call (Section 8.2).

- `derivative(fn, x)`: `Central<2>`, f', default step;
- `derivative(fn, x, h)`: your step, `h > 0`;
- `derivative<M>(fn, x)`: the method `M`;
- `derivative<M, N>(fn, x)`, with or without `h`: the derivative order `N`, 1 to 4 or `order::second` and so on.

`fn` takes a `T`; `x` fixes `T`, any `nxx::real` type.

The result. A `std::expected<Estimate<T>, error>`: `d->value` is the estimate, `*d->abs_error` the error estimate of the three Ridders methods (empty for the stencils), `d.error()` the reason for a failure. `d.value()` is `std::expected`'s accessor and throws on a failure: test, then read `d->value`.

Two checks (Section 7.2). *Did it work?* `if (d)`. *Is it right?* A wrong step scale, a noisy f or a break near x return a wrong value without failing: if in doubt, repeat at $2h$ and $h/2$. For Ridders, also require `r->abs_error && *r->abs_error <= atol + rtol * std::abs(r->value)`: `rtol` = 10^{-n} for n figures, `atol` in the derivative's units decides near $f' = 0$; a stencil has no estimate to test. Inside an iteration no check is needed.

Failures. `non_finite`: `x` not finite; `fn` infinite or NaN at a sample; an overflow; a step below the resolution of `x`; a step that is not positive and finite. A float derivative above the first warns at compile time, by every method.

Methods (relative accuracy in double at the default step).

- `Central<P>`, P = 2, 4, …, 10: $P + N - 1$ evaluations; about $\varepsilon^{P/(P+N)}$; with P = 2 about 11 digits for f' and 5 for f''''.
- `Forward<P>`, `Backward<P>`, P = 1 to 6: $P + N$ evaluations, never below (above) x; larger errors than `Central<P>`.
- `Ridders`: a median of 14 evaluations for f', at most 64, 65, 128 and 129 for f' to f''''; about 14 digits for f'; an error estimate. Near a NaN it returns a value down to $2^{-34}\max(|x|, 1)$ away (ten digits down to $2^{-30}\max(|x|, 1)$), then `non_finite`; closer to a pole, a wrong value whose estimate flags it.
- `ForwardRidders`, `BackwardRidders`: Ridders on one side of x only, for an edge, with an error estimate; a median of 10 evaluations for f', at most 33, 65, 97 and 129; about 1.7×10^{-13} for f'.

Choosing a method (Table 1.1; the full table is Table 7.1).

Situation	Method and step	Cost and accuracy
Inside an iteration; f smooth and cheap	`Central<2>`, the default step	2 evaluations; about 11 digits in `double`
A value to report, with an error estimate	`Ridders`, the default initial step	typically 14, at most 64; about 14 digits
f defined on one side of x only	`Forward<2>` or `Backward<2>`, the default step; with an error estimate, `ForwardRidders` or `BackwardRidders`	3 evaluations; about 1.8×10^{-11} relative on e^x. With the estimate: typically 10, at most 33; about 1.7×10^{-13}
f varies on a scale w far from $s = \max(\|x\|, 1)$	`Central<2>`, $h = \varepsilon^{1/3}w$	2 evaluations; as in the first row
f accurate to $\eta > \varepsilon$ only (a solver output)	`Central<2>`, $h = \eta^{1/3}s$	2 evaluations; about $\eta^{2/3}$ relative

Your own step. `default_step<M, N>(x)` is the one `derivative` uses; `h = nxx::math::root_epsilon<3, double>() * w`; for $\varepsilon^{1/3}w$ (`Central<2>`, f'; in general `root_epsilon<P + N, T>()`), and `h = std::cbrt(eta) * std::max(std::abs(x), 1.0);` for a noisy f.

No error estimate? Check it (Section 7.5). Repeat at $2h$ and $h/2$ and count the agreeing digits, or compare with `Central<4>` or with Ridders' method. Where $f' \approx 0$, judge the absolute error. Check the sign against the physics.

The pitfalls that matter most (Section 7.8).

- *The step's scale is in your units.* The default step assumes that f varies on the scale $\max(|x|, 1)$; at a large $|x|$ with a narrow feature, or at $|x| \ll 1$, pass h.
- *Evaluable is not valid.* A fit sampled outside its range returns numbers: make it return NaN there, and use `Forward<P>` or `Backward<P>` at the edge, or `ForwardRidders` or `BackwardRidders` for an error estimate.
- *Evaluable is not smooth.* A kink or a range break within a step of x gives a wrong value from every method that samples across it, without failing; Ridders' estimate flags a jump, nothing flags a kink at x: differentiate the branch you mean.
- *`auto` and `.value()`.* `auto e = *d` is an `Estimate`, and `d.value()` throws on a failure; write `d->value`.

Where to look. Names: Chapter 8; tables: Chapter 9; accuracy: Table 10.1; cost: Table 10.3; guarantees: Section 10.5.